Building LLMs from Scratch: A Source-Grounded Companion

Public Lectures Edition: 22 of 43 lectures

Compiled by Mahima Arora

2026-10-11

Table of contents

About this book 1
How claims are grounded . . . 1
Lectures not covered . . . 1

Introduction 2

1 Foundations and Course Roadmap 4
1.1 Why learn LLMs from scratch . . . 5
1.2 LLMs, generative AI, and the broader AI stack . . . 6
1.3 What modern LLMs do and how they differ from earlier NLP . . . 7
1.4 Applications and motivation . . . 9
1.5 Open and closed models, and learning access . . . 11
1.6 Transformers as the key modern architecture . . . 11
1.7 Tokenization, embeddings, and sequence representation . . . 13
1.8 Attention and long-range dependencies . . . 14
1.9 BERT, GPT, and other model families . . . 15
1.10 The build pipeline: stage one to stage three . . . 17
1.11 Prerequisites, delivery, and current evidence gaps . . . 19

2 From Text to Model Inputs 21
2.1 Why preprocessing exists . . . 22
2.2 Word-based tokenization foundations . . . 23
2.3 Building a simple word tokenizer in Python . . . 25
2.4 Limits of word tokenization . . . 26
2.5 Unknown and boundary tokens in a word tokenizer . . . 27
2.6 Character and subword tokenization . . . 29
2.7 Byte pair encoding as compression . . . 30
2.8 Byte pair encoding for subword vocabularies . . . 31
2.9 Using the GPT-2 tokenizer with tiktoken . . . 32
2.10 Why token embeddings are needed . . . 34
2.11 What embeddings capture . . . 35
2.12 Embedding matrices and lookup layers . . . 37
2.13 Embedding layers versus one-hot plus linear layers . . . 39
2.14 Positional embeddings and input embeddings . . . 40
2.15 Creating next-token training examples . . . 42

3 Attention and Transformer Building Blocks 45
3.1 From tokens to embeddings and context . . . 46
3.2 Simplified self-attention intuition . . . 47
3.3 Normalization and softmax for attention weights . . . 49
3.4 Computing context vectors . . . 51
3.5 Vectorizing self-attention over all tokens . . . 52
3.6 Why trainable Q, K, V are needed . . . 54
3.7 Causal attention and masking . . . 57
3.8 Dropout, batching, and implementation details for causal attention . . . 59
3.9 Efficient multi-head attention with weight splits . . . 61
3.10 Layer normalization in transformers . . . 65
3.11 Shortcut connections and gradient flow . . . 68

4 Assembling and Pretraining a GPT Model 73
4.1 What the model is and how the full GPT pipeline is assembled . . . 74
4.2 Configuration, tokenizer, and prerequisite vocabulary concepts . . . 75
4.3 Inputs, batches, embeddings, and dimensions . . . 77
4.4 Dropout, layer normalization, and transformer block internals . . . 78
4.5 Stacking blocks, final normalization, output head, and logits . . . 80
4.6 Coding the model and parameter accounting . . . 82
4.7 Next-token prediction tasks, targets, and basic generation from logits . . . 85
4.8 Loss function: from target probabilities to cross-entropy and perplexity . . . 86
4.9 Preparing data for pre-training . . . 89
4.10 Backpropagation, optimizer steps, and the training loop . . . 90
4.11 Evaluation mode, monitoring progress, and overfitting . . . 92
4.12 Decoding logits into text: greedy, temperature, and top-k sampling . . . 94
4.13 Interpreting decoding behavior and a gap in the course narrative . . . 96

5 Using Pretrained GPT-2 and Fine-Tuning for Classification 98
5.1 Why load GPT-2 weights and what prerequisites are assumed . . . 99
5.2 Installing tools and downloading released GPT-2 files . . . 100
5.3 Understanding GPT-2 release files, settings, and tokenization metadata . . . 101
5.4 The parameter dictionary and how GPT-2 weights map onto the architecture . . . 102
5.5 Updating the scratch GPT model and assigning pretrained weights . . . 104
5.6 Verifying generation and identifying what can be changed . . . 105
5.7 Preparing the spam-classification dataset . . . 107
5.8 Tokenization, padding, truncation, and dataset design for classification . . . 107
5.9 Building and checking the data loaders . . . 111
5.10 Adapting GPT for classification and making predictions . . . 112
5.11 Accuracy and loss before training . . . 113
5.12 Training loop for fine-tuning . . . 116
5.13 Results, overfitting checks, and what was trained . . . 117
5.14 Inference on new messages and saving the fine-tuned model . . . 119

6 Instruction Fine-Tuning and Evaluation **121**
6.1 Why instruction fine-tuning is needed and what data it uses . . . 122
6.2 Formatting instruction data into Alpaca-style prompts . . . 122
6.3 Tokenization and variable-length sequences . . . 124
6.4 Batch padding with the end-of-text token . . . 125
6.5 Shifted targets for next-token prediction . . . 126
6.6 Ignoring padding in the loss with -100 . . . 128
6.7 The full batching recipe and optional masking choices . . . 130
6.8 Loading pretrained GPT-2 weights for fine-tuning . . . 131
6.9 Downloading and mapping GPT-2 parameters into the model . . . 132
6.10 Checking the pretrained model before fine-tuning . . . 133
6.11 Evaluating a fine-tuned model: goals, examples, and response extraction . 134
6.12 Evaluation frameworks for instruction-tuned LLMs . . . 137
6.13 Using Ollama and Llama 3 as an automated evaluator . . . 139
6.14 Improving results and extending the approach . . . 141

Appendices **143**

A Sources and coverage **143**
A.1 Coverage map . . . 143
A.2 Where each section comes from . . . 145
A.3 Points not included . . . 148

B Evaluation report **158**
B.1 Planted claims by type . . . 159
B.2 Planted claims the checker missed . . . 159

About this book

This book is generated from the lecture playlist Building LLMs from scratch and covers lectures L01, L02, L04, L06, L08, L10, L12, L14, L16, L18, L20, L22, L24, L26, L28, L30, L32, L34, L36, L38, L40, L42.

This is the **Public Lectures Edition**: it covers 22 of the playlist's 43 lectures. The others are listed below with the reason each one could not be used; nothing in the book is written from them.

How claims are grounded

Every factual sentence in the chapters was written from the automatic captions of the lectures and cites the caption lines it relies on. Each chapter names the lectures it is based on, and the Sources appendix lists, for every section, the minutes of each lecture it draws on, linked to that moment in the video. A separate model then checked every sentence against the lines it cites; unsupported text was rewritten once, and anything still uncertain was sent to a human reviewer.

Measured for this edition: 97.9% of 2237 factual sentences were judged supported by their cited captions; the checker caught 106 of 120 deliberately planted false sentences. This is a measured rate, not a guarantee of zero errors; see the evaluation report appendix.

An independent evaluator, using a different model and never seeing the checker's verdicts, then judged 200 randomly sampled sentences 3 times each: 96.5% were supported by their captions, with 1.5 material errors per 100 sentences. Every sentence flagged along the way was decided by a human reviewer.

Lectures not covered

- members-only on YouTube: L03, L05, L07, L09, L11, L13, L15, L17, L19, L21, L23, L25, L27, L29, L31, L33, L35, L37, L39, L41, L43

Introduction

Large language models and generative AI are transforming everything around us, with startups forming around them, companies switching to them for various tasks, and jobs in the field rising rapidly. That is why learning to build an LLM from scratch matters: many people jump straight to applications and running code, but very few understand how large language models actually work or the nuts and bolts that make them powerful. Building one yourself develops confidence in the subject, and after that the application layer becomes much easier to approach.

The field is worth learning because current models are far more capable than earlier systems. Open models also make the subject newly accessible, because key information is now available publicly and the gap with closed models is decreasing. Generative AI is broader than language alone, including video, audio, and 3D models, and its practical impact already reaches real applications and a fast-growing job market.

It fits readers who want to understand LLMs from the very basics, build an entire model from scratch, and be able to answer detailed questions with confidence.

The book opens with foundations. Chapter 1 introduces what LLMs are, why building them from scratch is valuable, how generative AI and the broader AI stack relate to them, what modern systems do, and why transformers are central. It also lays out the build pipeline from stage one to stage three, while making explicit where prerequisite coverage is partial and where evidence gaps remain. Chapter 2 then turns raw text into model inputs by moving from word tokenization to character and subword methods, bringing in byte pair encoding, embeddings, positional information, and the full preprocessing pipeline needed to create next-token training examples. Chapter 3 builds directly on those representations and develops the transformer's core mechanics: self-attention, causal masking, multi-head attention, layer normalization, residual pathways, and the implementation details that make contextual representations possible.

With those building blocks in place, the book assembles complete models and then adapts them to practical settings. Chapter 4 constructs a full GPT-style model, defines next-token prediction, trains it in a pretraining loop, and shows how generation changes under greedy decoding, temperature, and top-k sampling. Chapter 5 moves from training from scratch to using pretrained GPT-2 weights, mapping released parameters into the architecture and then fine-tuning the model for spam classification with padded datasets, dataloaders, and accuracy-based evaluation. Chapter 6 extends that workflow to instruction fine-tuning, covering prompt formatting, variable-length batching, shifted targets, masking choices in the loss, loading pretrained weights again, and evaluating

instruction-following behavior with both examples and broader benchmarking ideas. By the end, each chapter has built on the one before it: from concepts, to inputs, to transformer internals, to GPT pretraining, to transfer, and finally to instruction-following systems.

1 Foundations and Course Roadmap

Based on L01: Lecture 1: Building LLMs from scratch: Series introduction; L02: Lecture 2: Large Language Models (LLM) Basics; L04: Lecture 4: What are transformers?; L06: Lecture 6: Stages of building an LLM from Scratch.

This chapter begins by setting the standard for the rest of the book: the goal is not merely to use large language model applications, but to understand how large language models work at the nuts-and-bolts level and to build one from scratch with confidence. That emphasis matters because many learners jump straight to applications, Colab notebooks, copied code, or LLM app tools, even though real depth is what builds confidence, helps you answer detailed interview questions, and keeps knowledge from remaining superficial.

As the opening chapter, this one establishes the conceptual map before the implementation work begins. It starts with why learning from scratch matters, then defines large language models, generative AI, and their place in the broader AI stack, before turning to what modern LLMs do, why they are useful, and how application excitement can obscure foundational understanding. From there, it introduces transformers and the key representation ideas that later chapters will unpack in detail, surveys major model families, lays out the stage-by-stage build pipeline, and closes by making the prerequisite picture explicit, including the current gap that neural networks and basic deep learning are only partially introduced here.

At a glance

- A large language model is a neural network designed to understand, generate, and respond to human-like text.
- The word large refers to parameter count, and these models typically have billions of parameters, with some reaching trillions.
- Generative AI is broader than large language models because it includes language, video, audio, and 3D models.
- In the AI stack, artificial intelligence is the broadest umbrella, machine learning is inside it, deep learning is inside machine learning, and large language models sit inside deep learning.
- A rule-based chatbot can count as AI, but it is not machine learning if it does not learn from user responses and behaves the same way for different users.

1.1 Why learn LLMs from scratch

Why learn large language models from scratch instead of starting with applications and ready-made notebooks? The need is pressing because large language models and generative AI are transforming everything around us, startups are forming in this space, companies are switching to large language models for various tasks, and jobs in this field are rising a lot. A sound way to learn in this setting is to begin from the basics, assume nothing, and study the nuts and bolts of every concept. Many learners make the opposite mistake: they jump directly to applications, run a Google Colab notebook, follow a video, and execute code without really understanding how large language models work. When that happens, very few people know the nuts and bolts that make large language models so powerful. The same wrong pattern appears when students become fascinated by applications, download code, run it, make small changes, and then write on a resume that they know about LLMs. The right approach is to understand the foundations, make the basics clear, and understand the nuts and bolts of how LLMs work.

Many courses focus on building LLM apps or on large language model concepts, but they do not teach how to build a large language model from scratch. One course is fairly quick. A learner seeking real understanding wants one course that teaches the foundations in a lot of detail and depth. A learner seeking to build an LLM from scratch wants detailed foundations so that the learner's skill set improves and the learner feels confident about generative AI and LLM job interviews. One playlist has 18 chapters, but each chapter is only 10 to 15 minutes long. Other material is too complex for beginners because it starts in the middle of a concept and is not meant for beginners. Still other courses are not explained well from scratch, so they do not satisfy the need for a deep beginner-friendly treatment.

A from-scratch path changes what the learner can do with the subject. Building a large language model completely from scratch would make a learner feel very confident about the subject. The aim is a very comprehensive path that shows everything about building an LLM from scratch. It is intended to teach everything from the basics as a beginner, without assuming anything, and to lead to building an LLM from scratch successfully by yourself by the end. Once that foundation is in place, the later application parts start seeming extremely easy. The planned material is extensive: detailed lecture notes are being made and shared, with a massive set of notes of maybe 200 to 300 pages and 50 to 60 videos. Within that material, nothing will be assumed, everything will be spelled out, and it is meant for serious people who want to transition to large language models and understand LLMs the right way.

Application frameworks can help build LLM apps, but they are not useful if you do not know how to build an LLM yourself. Starting by deploying apps is not the right way to learn about LLMs; the right way is to know how to build an entire large language model from scratch. One reason is interview readiness: if someone asks a detailed question about key, query, and value, or about positional embedding or positional encoding, you should be able to say that you built this from scratch and know the details. The students

who will really contribute and make an impact are the ones who know details such as how to write the Transformer code, how key-query-value works, and what positional encoding is. Without understanding concepts like Transformer code, key-query-value, and positional encoding, a student's knowledge remains superficial.

1.2 LLMs, generative AI, and the broader AI stack

How do large language models fit into the wider landscape of AI, and why are they only one part of generative AI? Generative AI is broader than large language models, and it includes language, video, audio, and 3D models. Examples shown for generative AI include realistic videos that are made by artificial intelligence rather than shot on camera. A large language model, by contrast, is a neural network designed to understand, generate, and respond to human-like text. At the core, LLMs are neural networks built for generic text-related applications such as understanding, generating, and responding to human-like text. They can also be described as deep neural networks trained on massive amounts of data to do tasks such as understanding, generating, and responding to human-like text.

To make that definition concrete, it helps to unpack what a neural network is. A neural network takes input data, feeds it into a layer of neurons, stacks a number of these layers together, and produces an output layer. The name "neural network" comes from the idea that these neurons represent the circuitry in the brain in a very symbolic sense. Deep neural networks have many applications, including image detection, text generation, self-driving cars, and detecting whether a patient has a brain tumor. LLMs are one specific application of these deep learning techniques, focused on processing and generating human-like text. They are called language models because they deal with language rather than other modalities such as images or video.

The word "large" adds another part of the definition. In "large language model," "large" refers to the number of parameters in the model. LLMs typically have billions of parameters, and some have trillions of parameters. A table of GPT-3 model sizes ranges from 125 million parameters through 350 million, 760 million, 13 billion, and up to 175 billion parameters, and GPT-4 is said to have even more parameters. As models progress from GPT-1 to GPT-2 to GPT-3, they become better and their parameter counts increase tremendously. The same comparison also shows other quantities increasing as the models grow: GPT-1 has 12 decoders, GPT-2 has 48, GPT-3 has 96, and token size increases from 512 to 1024 to 2048. A separate graph from 1950 to 2022 shows model parameter counts reaching around 100 million, 1 billion, and almost 1 trillion by around 2020, and the orange symbols have the largest number of parameters. These models are called large language models because their number of parameters is unprecedentedly huge.

The broader AI stack becomes clearer when the categories are nested. Artificial intelligence is the broadest umbrella, machine learning is a subset of artificial intelligence, deep learning is a subset of machine learning, and large language models are a subset of deep learning.



Figure 1.1: Three increasing blocks compare GPT-1, GPT-2, and GPT-3 by parameter count, decoder count, and token size. Redrawn from the board in L02 9:02.

Artificial intelligence is the broad category that includes any machine showing even remotely human-like behavior or some sort of intelligence. A rule-based chatbot can be an example of artificial intelligence without being machine learning. It is not machine learning because it does not learn from user responses and behaves the same way for different users. Machine learning consists of machines that learn and adapt based on how users interact with them.

Deep learning narrows the focus further. Deep learning usually involves neural networks, whereas machine learning includes neural networks and also methods that are not neural networks, such as decision trees. A decision tree used to predict heart disease from patient data is an example of machine learning but not deep learning because it does not use neural networks. A convolutional neural network for image detection is an example of deep learning because it uses a neural network to classify an input image. A neural network for handwritten digit classification is also an example of deep learning. Deep learning is therefore a subset of machine learning because machine learning includes neural network methods and also methods like decision trees that are not neural networks. Large language models are then a subset of deep learning because deep learning also includes image-based work, while LLMs deal only with text.

This also explains where generative AI sits relative to LLMs. Generative AI can be thought of as a mixture of large language models and deep learning because it handles multiple modalities such as image, sound, and video. More specifically, generative AI uses deep neural networks to create new content such as text, images, and other media, whereas LLMs deal only with text. So although the terms are often confused, LLMs are a specific text-focused application within deep learning, while generative AI extends across several media types.

1.3 What modern LLMs do and how they differ from earlier NLP

Why are modern language models so much more useful than the NLP systems that came before them? Large language models work on language, and they perform a wide range of natural language processing tasks such as question answering, translation, and sentiment analysis. Natural language processing existed long before large language models. Earlier NLP models were designed for very specific tasks. For example, one earlier NLP model

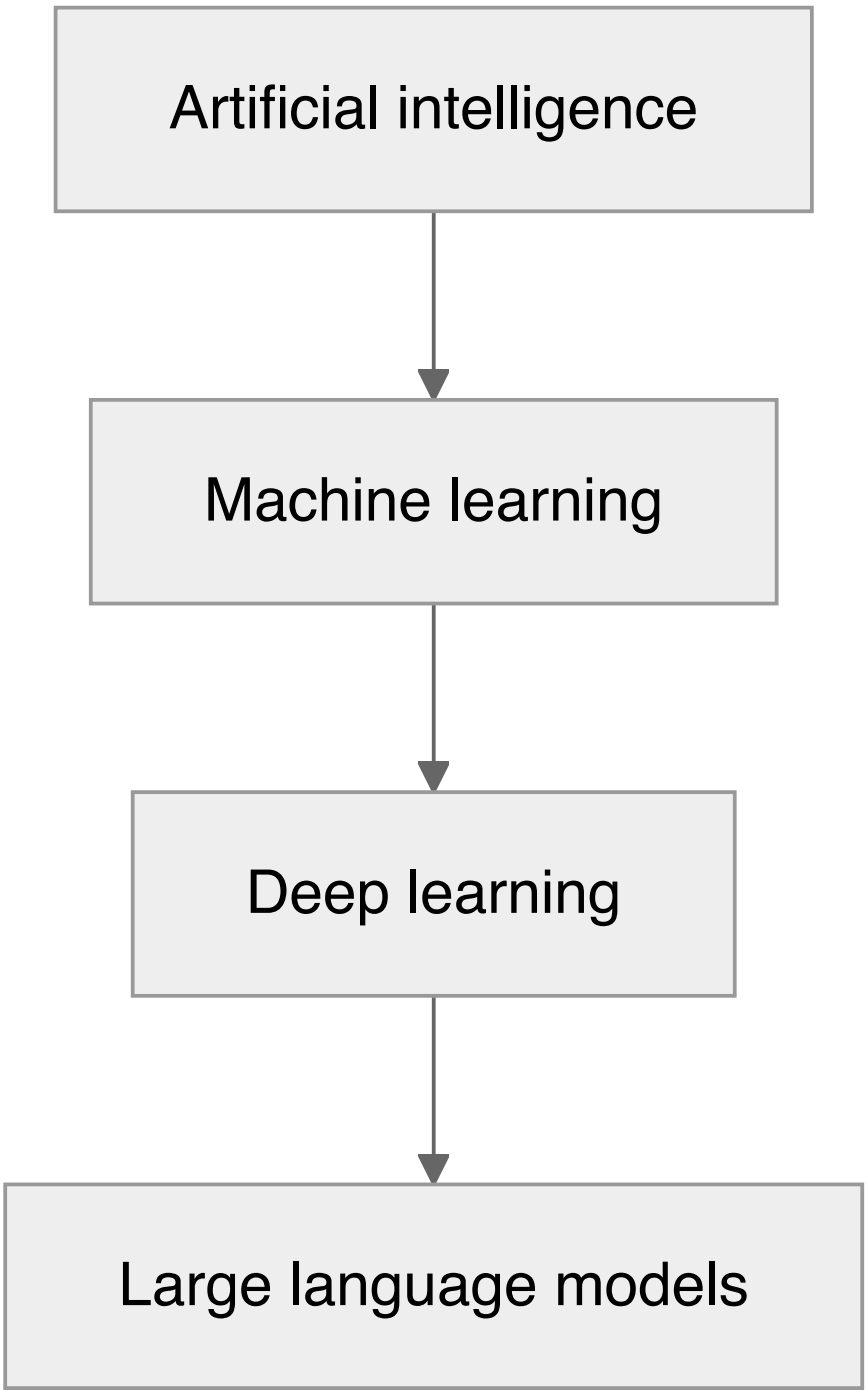


Figure 1.2: Artificial intelligence contains machine learning, which contains deep learning, which contains large language models.

might be designed for language translation, while another specific NLP model might be for sentiment analysis. Before large language models, natural language processing typically required training a separate algorithm for each specific task.

Modern LLMs can do a wide range of NLP tasks. A GPT trained for text completion can also work well for language translation. That is a basic difference between earlier NLP systems and modern LLMs: earlier models were built for specific tasks, whereas modern LLMs can handle many tasks. Compared with earlier NLP models, LLMs are broader, they can do a wide range of generic tasks, and they have many more applications. Large language models have transformed natural language processing by advancing the generation, understanding, and translation of human language.

Writing an email from custom instructions was very difficult for earlier language models. For a modern LLM such as ChatGPT, the same task is almost trivial. For example, given an instruction to draft an email to a colleague or a friend to book movie tickets, ChatGPT starts drafting the email. This kind of easy, generic behavior is part of why modern LLMs have so many more applications than earlier NLP models.

What makes these models generic if they are trained in such a simple way? Large language models are only trained to predict the next word. Even so, training for next-word prediction can produce emergent properties. Because of these emergent properties, a model that is not only good at predicting the next word can also be useful for multiple-choice questions, text summarization, emotion classification, and language translation. Although they are only trained for next-word prediction, LLMs can show abilities such as classifying text, translating text from one language into another, and summarizing texts. The pre-training stage works so well that LLMs develop additional useful properties, which makes them impactful for a wide range of tasks.

1.4 Applications and motivation

Why do large language models matter so much now, and why are they attracting so much attention across applications and careers? A simple contrast shows how far natural language processing has come. One early chatbot from 50 to 60 years ago was “Elisa”, one of the first chatbots humans developed, and its conversation quickly proceeds nowhere. By contrast, “chat GPT” can answer a request about learning large language models with a response that is extremely useful, to the point, and filled with resources such as books, online courses, and research papers. Research on natural language processing and large language models is at a stage where LLMs such as GPT are very powerful. Generative AI is broader than language alone and includes video, audio, and 3D models. Some videos can look incredibly realistic even though they are made entirely by artificial intelligence rather than shot on a camera.

The same shift appears in interactive text generation. A prompt such as “plan a relaxing day” can ask for a day focused on rejuvenation and request that the system first ask

about favorite forms of relaxation. “chat GPT” replies conversationally, asking whether the user prefers spending time outdoors, reading, yoga, or creative pursuits. If the answer is “reading a book,” it generates a reading-focused plan with items such as morning meditation, breakfast, reading time, and a nature walk. This kind of demonstration shows that large language models understand, generate, and respond to human-like text, and that they also respond like humans. They are sounding more and more like humans.

These capabilities lead directly to practical applications. The applications of large language models can be divided into five main categories, even though there are several more applications beyond these. One category is creating new content that may not already exist. For example, a model can write a poem about the solar system in the format of a detective story, and it can also be used to write books and generate media articles. Another category is chatbot or virtual-assistant use, where a person can interact with the model in an ongoing conversation. Chatbots are a huge and major application for airlines, hotel reservation desks, banks, movie and restaurant services, and customer support.

The remaining categories include machine translation, new text generation, and sentiment analysis. Machine translation means that text can be translated to any language. A model can be asked to translate a poem to French, and it can do that immediately. The output can be translated very quickly into many languages, and there is some support for regional languages, with research being done to improve those outputs. Another application is new text generation, including poems, books, media articles, and news articles that did not exist before. Large language models can also be used for sentiment analysis by taking paragraphs and identifying the sentiment, which can be useful for hate speech detection on social media.

To illustrate these applications, consider a portal created for school teachers with tools such as a YouTube generator, MCQ generator, text summarizer, text rewriter, and worksheet generator. In one application, selecting the MCQ generator, entering a topic such as “gravity”, and clicking generate causes the LLM powering the application to produce multiple-choice questions within seconds. In another application, a lesson plan on “gravity” can be generated and aligned with the CBSE curriculum of India. The generated lesson plan includes the objective, assessment, key points, opening, introduction to new material, guided practice, and independent practice. This can save teachers a great deal of time, and it is presented as something that was not possible five years earlier but is possible now with LLMs. A similar tool can generate questions on “World War II” at hard, medium, and easy levels, along with the correct answers and explanations, and it does so within seconds.

The broader motivation is not only technical but also practical. Large language models and generative AI are changing every single thing around us, including applications provided to teachers. The projected global generative AI job market is expected to grow about five to six times in the next five years. Knowledge of generative AI and large language models is an extremely useful skill, and the need for this skill is only going to increase in the future. If you have knowledge about LLMs, the sky is the limit

for building wonderful applications. The examples here are only the tip of the iceberg, because LLMs can be used for many more research and industrial applications.

1.5 Open and closed models, and learning access

Why does the distinction between open-source and closed-source models matter for learning large language models now? An open-source model is one whose architecture is available to the public, so anyone can see the architecture. By contrast, models released by OpenAI are described here as usually closed source. A closed-source model is one for which the weights, the architecture, and many other details about the model itself are not released publicly.

This distinction matters because access affects what can be inspected and studied directly. The comparison described for 2022 says that, when the field of large language models was booming, most things were pretty closed source. GPT-4 is presented as a closed-source model released in 2023. In the 2024 comparison, the gap between open-source and closed-source models is described as slowly decreasing. When Llama 3.1 is released in that comparison, it is shown as performing at the same level as GPT-4. All the information needed to learn is available right now through open-source models.

That makes the learning path less dependent on access to closed systems. A strong reference for learning large language models is Sebastian Raschka's book, which is described as one of the best books on the topic. That book is also used as the reference material for this course.

1.6 Transformers as the key modern architecture

What makes large language models work so well, and why does that matter before looking at tokens and embeddings? The specific secret sauce behind large language models is the Transformer architecture. People often use the terms large language models and Transformers interchangeably, and their similarities and differences can be examined. Most modern large language models rely on the Transformer architecture. A Transformer is a deep neural network architecture introduced in the 2017 paper "Attention Is All You Need." That paper has more than 100,000 citations, described as occurring in about five years and also in six to seven years, because it led to many later breakthroughs. "Attention Is All You Need" is a dense paper, and understanding it takes a great deal of time and effort.

The original Transformer and GPT are closely related, but they are not the same architecture. The GPT architecture that underlies ChatGPT originated from the ideas in "Attention Is All You Need," but GPT is not exactly the same as the original Transformer architecture. The original Transformer was proposed for machine translation rather

than text completion, especially for translating English text into German and French. A Transformer architecture predominantly consists of two main blocks: an encoder block and a decoder block. The original Transformer has both an encoder and a decoder, whereas GPT uses only the decoder and not the encoder. The Transformer paper came in 2017 with an encoder plus decoder, and GPT came in 2018 with only the decoder architecture. The source also states that even GPT-4 uses only the decoder and not an encoder. It further places GPT in a short timeline: GPT arrived in 2018, GPT-2 in 2019, and GPT-3 in 2020 with 175 billion parameters, which changed the field because models of that size had not been seen before.

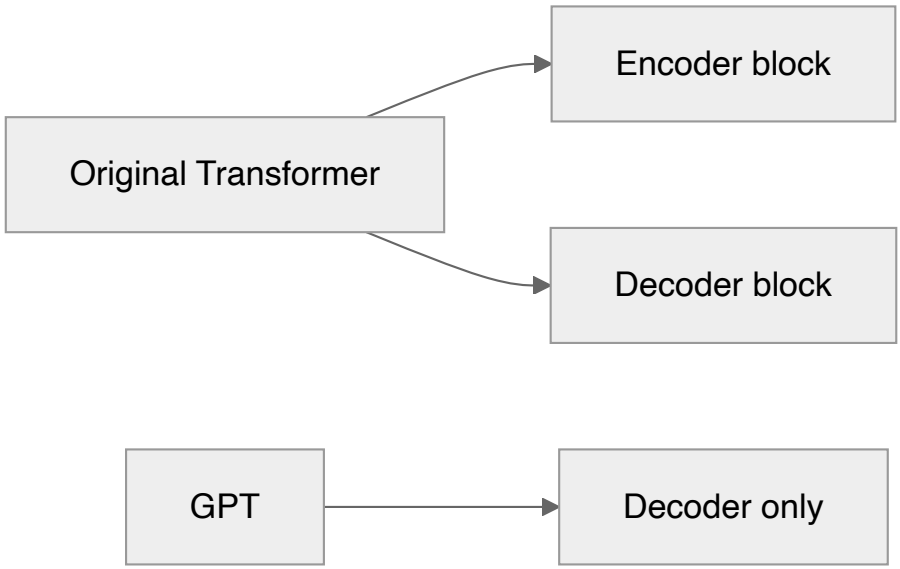


Figure 1.3: The original Transformer uses an encoder and a decoder, while GPT uses only the decoder.

To see how this architecture works, start with the encoder side. The encoder is one of the most important building blocks of the Transformer architecture. The encoder takes preprocessed tokens and performs vector embedding. The main purpose of the encoder is to take input text or tokens and convert them into vector embeddings. Vector embedding represents tokens in a way that captures semantic meaning between words by converting words into vectorized representations. For example, tokenization may assign random IDs to “dog” and “puppy,” but the information that these words are related must be encoded somewhere. On the left side, these four steps take the input sentences and convert them into vector embeddings so that semantic meaning is captured between the words.

These embeddings are fed into the decoder on the right-hand side of the Transformer architecture. The main purpose of the decoder is to generate output text from the embedding vectors and from the partial output it has received. The model generates its output one word at a time. During decoding, the model has access to the partial

output text that it has already generated. In the translation example, the partial output is described as “Das s”, and that partial text is available to the model before it predicts the next word. The decoder receives both the vector embeddings and the partial output text, and it predicts the next word based on that information. It generates the translated text one word at a time until the final output is produced. A Transformer can be viewed as a neural network trained with a loss function and optimized parameters, including feed-forward layers with weights and parameters.

The key idea behind the Transformer architecture is the attention mechanism. Self-attention is the heart of why Transformers work so well, which is why the paper is called “Attention Is All You Need.” In a nutshell, the attention mechanism gives an LLM selective access to the whole input sequence when generating output one word at a time. It allows the model to weigh the importance of different words relative to each other and capture long-range dependencies. When predicting the next word, self-attention lets the model look at past context and weigh which words matter more for the prediction. The attention mechanism also lets an LLM determine the importance of words not only in the current sentence but also in previous sentences, because context matters for predicting the next word. More generally, attention allows an LLM to access the entire context and assign weight to the words that are important for predicting the next word.

1.7 Tokenization, embeddings, and sequence representation

How does raw text become a form a model can process while still preserving meaning and word order? Tokenization is a preprocessing step in which sentences are broken into simpler words or tokens, because the entire sentence cannot be fed into the model directly. A token can be thought of as a unit of a sentence. For simplicity, it is useful to imagine that tokenization breaks a sentence into individual words, although one word is generally not equal to one token. There is a particular way of doing tokenization. In the simplified workflow, the input text is first taken and then preprocessed into tokens. Tokenization can be understood procedurally as breaking sentences into individual tokens and assigning each token a unique numerical ID. For example, the sentence “fine tuning is fun for all” is broken into smaller pieces, and each piece is assigned a unique ID. In preprocessing, data such as Reddit posts or Wikipedia text is broken into words or collected subwords from each sentence in the dataset.

Token IDs are only the start, because the model also needs a representation that captures relationships between words. Numerical token IDs alone do not capture semantic relationships between words. If tokenization assigns random IDs to words like “dog” and “puppy”, those IDs do not express that the words are related. Vector embedding is the process that represents tokens in a way that captures semantic meaning between words. After tokenization, each word is transformed into a very high-dimensional vector space so that semantic meaning between words is captured. In the simplified transformer workflow, token IDs are passed into the encoder, and the encoder converts them into

embeddings. These embeddings project words into high-dimensional vector space in a way that captures semantic relationships between words.

In the embedding examples, words such as “king”, “man”, and “woman” are closer together, “apple”, “banana”, and “orange” are closer together, and “football”, “golf”, and “tennis” are closer together. Words with similar meanings should lie closer together than unrelated words. Creating useful vector embeddings is difficult, because vectors cannot just be placed randomly. There is a detailed procedure for this, and neural networks are trained for the embedding step so that related words are closer than unrelated ones. The examples also show text converted first into token IDs and then into three-dimensional vector representations by an embedding model. Alongside tokenization and embeddings, positional encoding provides information about the order in which words appear in a sentence.

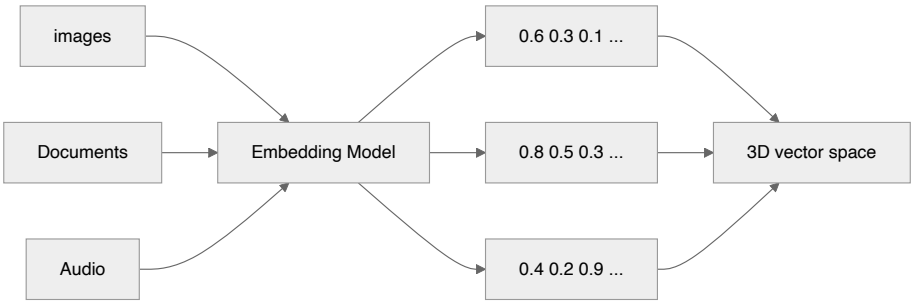


Figure 1.4: Images, documents, and audio are input to an embedding model, which outputs numeric vector representations visualized as points in a 3D space. Redrawn from the board in L04 13:02.

Listing 01.07.1: This code builds a vocabulary dictionary that assigns each token in ‘all_words‘ a unique integer ID using ‘enumerate‘. Source: rasbt/LLMs-from-scratch, `ch02/01_main-chapter-code/ch02.ipynb`, Apache-2.0, commit cdbd33e.

```
vocab = {token:integer for integer,token in enumerate(all_words)}
```

1.8 Attention and long-range dependencies

How can a model use earlier context when the words that matter may be far away from the current prediction? In this setting, attention is a technical term. A key part of the transformer architecture is the self-attention mechanism. Attention mechanisms have become an integral part of sequence modeling because they allow modeling dependencies without regard to distance in the input or output sequences. Put more concretely, attention lets the model relate different words even when those words are close together or far apart. Self-attention is an attention mechanism that relates different positions

of a single sequence in order to compute a representation of that sequence. One useful intuition is that attention allows the model to weigh the importance of different words or tokens relative to each other.

For example, suppose earlier sentences mention that Harry Potter is on a station or platform number something and that Harry Potter wants to board the train. When predicting the next word in a fourth sentence, the context from sentences one, two, and three is important for understanding that fourth sentence and predicting its next word. That is the meaning of long-range dependencies: when predicting the next word, the model needs to know the importance of previous words. It needs to know how much attention to give to previous words and which earlier words should receive more attention, perhaps "Harry," perhaps "platform," or perhaps "train." Self-attention captures these long-range dependencies so the model can look far behind, including sentences closer to the current one, to identify the next word.

The mechanism becomes easier to picture if you think in terms of scores over words. For next-word prediction, self-attention maintains an attention score that tells the model which word should receive more attention. That attention score is a matrix that tells the model which words should be given more importance in relation to other words. Attention appears in attention blocks such as multi-head attention and masked multi-head attention. These attention blocks help capture long-range dependencies in sentences, which is why the paper is called "attention is all you need."

There is also a helpful human analogy. People naturally keep context in mind while reading a story and remember what was written on the previous page. Self-attention allows a model to capture long-range dependencies, which helps it make the next-word prediction accurately. In ChatGPT, the model gives attention to every sentence in the input when predicting the next word. It does not just look at the sentence immediately before the current one, because previous words may be more important. Another way to think about self-attention is as parallel attention to different parts of a paragraph or to different sentences.

1.9 BERT, GPT, and other model families

How should you think about model families such as BERT and GPT, and how do they relate to Transformers and large language models more broadly? BERT and GPT are later variations of the Transformer architecture. BERT stands for Bidirectional Encoder Representations from Transformers. GPT stands for Generative Pre-trained. GPT is described as pre-trained because it is a pre-trained or foundational model. Both BERT and GPT are based on the Transformer architecture.

BERT predicts hidden or missing words in a sentence, which are also called masked words, whereas GPT generates a new next word. In BERT, words are randomly masked during training. The text is pre-processed by converting it into token IDs, then passed

to the encoder for embeddings, and the output fills in the missing words. For example, from the incomplete text “this is an Dash or question mark of how llm Dash perform”, BERT outputs the missing words “example” and “can”, giving “this is an example of how llm can perform”.

GPT handles incomplete text in a different way. GPT receives incomplete text where the next word is unknown, pre-processes it into token IDs, and uses a decoder-only model to predict that next word. For example, given “this is an example of how llm can Dash”, the decoder predicts the unknown word “perform” to produce “this is an example of how llm can perform”. GPT models learn to generate one word at a time. GPT is a left-to-right model: it uses the information on the left and predicts the rightmost unknown token or next word. Because GPT produces one word at a time, it can complete missing text. GPT does not have an encoder and only has a decoder. BERT, by contrast, only has the encoder.

In BERT, random words can be masked, so the model has to pay attention to different parts of the sentence. BERT is called bidirectional because it looks at the sentence from both directions, paying attention from the left and the right, unlike GPT which looks left to right. By looking at the entire sentence from both directions, BERT can capture nuances and relationships between words that are important for understanding meaning and context. It can also capture the meanings of different words and how they relate to each other very well. For example, BERT can distinguish “bank” as a financial institution from “bank” as a river bank by looking at the surrounding words. That is why BERT does very well in sentiment analysis and is very commonly used for it. GPT can also be used for sentiment analysis, but BERT is especially suited to it.

Not all Transformers are LLMs. Transformers can be used for tasks other than language, including computer vision. Vision Transformers can be used for image recognition and image classification. They can also be used to detect potholes on roads and to classify tumors from images. Compared to convolutional neural networks, Vision Transformers achieve remarkable results while using substantially fewer computational resources for pre-training, and they show a generally weaker bias. When thinking about image classification, many people think first of convolutional neural networks, but Vision Transformers are a newer method that is also gaining popularity.

However, it is also important to distinguish the two terms carefully. Not all LLMs are Transformers. LLMs can be based on recurrent or convolutional architectures as well. Before Transformers, recurrent neural networks were introduced in 1980 and long short-term memory networks were introduced in 1997. Both architectures can do sequence modeling and text completion tasks, so they can also be called language models. An RNN maintains a feedback loop, which is why it can incorporate memory. An LSTM incorporates two separate paths, one for short-term memories and one for long-term memories. Using both paths, LSTMs can make predictions of what comes next. Recurrent neural networks, long short-term memory networks, and some convolutional architectures can also be large language models. For that reason, Transformers and LLMs should not be used interchangeably, because they are different things.



Figure 1.5: A three-step timeline shows Recurrent Neural Networks (1980), Long Short-Term Memory Networks (1997), and Transformers (2017), with dotted arrows progressing from left to right and annotations 'Bam!', 'Double Bam!!', and 'Triple Bam!!!'. Redrawn from the board in L04 33:34.

1.10 The build pipeline: stage one to stage three

How does the full build pipeline of a large language model fit together, from the first data steps to task-specific applications? The roadmap is divided into three stages: stage one, stage two, and stage three. Stage one focuses on the building blocks needed before training a large language model: data preprocessing and sampling, the attention mechanism, and the LLM architecture. In this first stage, data is collected from different datasets, processed, and sampled. The attention mechanism is built from scratch in Python, including key, query, value, attention scores, positional encoding, and vector embeddings. Stage one also covers the LLM architecture: how to stack layers and where the attention head goes, with the goal of understanding the basic mechanism behind a large language model. The discussion of the Transformer attention mechanism includes multi-head attention, positional encoding, input embedding, and output embedding.

A next-word prediction task gives a bunch of words and predicts the next word. Context means how many words should be taken for training to predict the next output. The data preparation work includes constructing batches from a large dataset and feeding those batches to the large language model. Batching the data into different sets makes computation much more efficient. A data batching sequence is implemented before the dataset is given to the large language model for pre-training.

Stage two is pre-training: after assembling the data and constructing the LLM architecture, code is written to train the model on the underlying dataset. Pre-training builds a foundational model on unlabeled data. The training procedure breaks the training dataset into epochs, computes the gradient of the loss in each epoch, updates the parameters, and generates sample text toward the end for visual inspection. The process also includes model evaluation and loading pretrained weights. Functions are implemented to save and load large language model weights so the model can be reused or training can continue later. Saving and loading weights saves a lot of computational cost. The model also loads pretrained weights from OpenAI into its own large language model. Taken together, stage two consists of the training loop, model evaluation, and loading pretrained weights to build a foundational model.

Modern large language models are trained in two main steps: first, they are pre-trained on unlabeled data, and then they are fine-tuned. Pre-training requires very large datasets, typically billions of words, and it is expensive. One stated example is that pre-training

GPT-3 cost about 4.6 million dollars. In pre-training, the dataset is unlabeled and autoregressive because the sentence structure itself creates the labels by predicting the next word. Fine-tuning means providing the model with a specific, usually smaller labeled dataset and training it further to build a better model. In fine-tuning, a labeled dataset is used, such as labels for spam versus not spam or good versus bad answers.

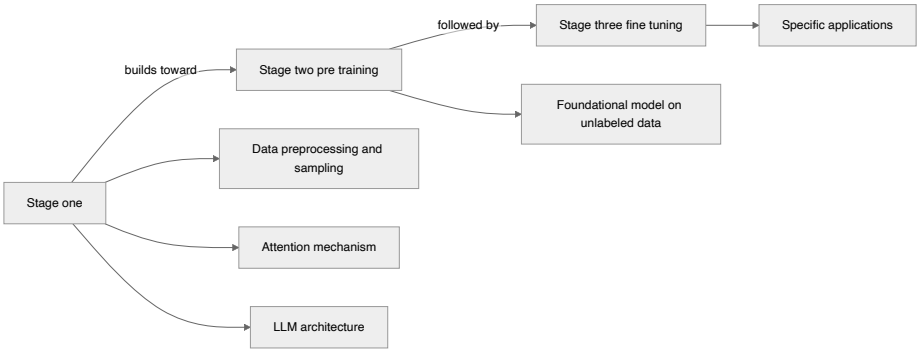


Figure 1.6: The roadmap moves from building blocks and data preparation, to pre-training a foundational model, to fine-tuning for applications.

Stage three focuses on fine-tuning the large language model to build specific applications. Two fine-tuned applications are built: a classifier and a personal assistant. A classifier can label emails such as "you are a winner you have been specially selected to receive th000 cash now" as spam and "hey just wanted to check if we are still on for dinner tonight let me know" as not spam. A pretrained foundational model alone is not enough for spam classification, because the model must be trained with labeled data. To build an email classification application, the foundational model is given additional labeled data that marks examples as spam or not spam. Another fine-tuned application is a chatbot that takes an instruction and an input and produces an output.

Why do all three stages matter? All the stages are equally important, and many students focus only on stage three and use LangChain or tools to build applications without understanding what is going on in stage one and stage two. When companies deploy LLMs into production level, they fine-tune the pre-trained LLM; nobody deploys the pre-trained one directly. Fine-tuning is generally needed for building production-ready LLM applications. Fine-tuned LLMs can outperform LLMs that are only pretrained on specific tasks. If you want to build a specific application on your own dataset and take it to production level, fine-tuning is needed.

Listing 01.10.1: This code selects one input vector and defines the input and output embedding dimensions for computing attention components step by step. Source: rasbt/LLMs-from-scratch, `ch03/01_main-chapter-code/ch03.ipynb`*, Apache-2.0, commit cdbd33e.*

```
x_2 = inputs[1] # second input element
d_in = inputs.shape[1] # the input embedding size, d=3
d_out = 2 # the output embedding size, d=2
```

Listing 01.10.2: This code saves the model parameter state dictionary to a file named "model.pth" using PyTorch. Source: rasbt/LLMs-from-scratch, `ch05/01_main-chapter-code/ch05.ipynb`, Apache-2.0, commit cdbd33e.

```
torch.save(model.state_dict(), "model.pth")
```

1.11 Prerequisites, delivery, and current evidence gaps

What background do you need before the material turns to implementation, and how will the teaching move from concepts into code? A lack of familiarity with ChatGPT is not a problem, because the playlist builds its own GPT and explains it along the way. The book serves as reference material for the course, and the plan is to convert every aspect of its 48 pages into a set of videos. The playlist is expected to contain about 35, 40, or maybe 50 videos. To teach from the book in detail, detailed whiteboard notes are being prepared from the book so that understanding can be transferred in an interesting and fundamental way. LangChain is described as a tool that helps build LLM apps.

That teaching plan also explains the pacing of the early part of the series. The first six lectures are introductory and are meant to give a feel for the whole series while building conceptual understanding at a fundamental level. From lecture 7 onward, the material dives deeply into code and starts stage one. The approach combines whiteboard explanation with coding so that details and code are learned together, because theory and practical implementation are both treated as important. The next stage starts with data preparation and sampling. It begins by working with text data, including looking at datasets, loading a dataset, counting characters, and breaking data into tokens. From the next lecture onward, Jupyter notebooks are shared so coding can begin in parallel.

Key takeaways

- Large language models and generative AI are transforming everything around us, startups are forming in this space, companies are switching to large language models for various tasks, and jobs in this field are rising a lot.
- The chapter argues that the right way to learn is from the basics and the nuts and bolts, not by only running notebooks, copying code, or jumping straight to applications.
- The roadmap is meant to teach everything from scratch so that by the end a beginner can build a large language model alone, and with that foundation later application work becomes much easier.

- Generative AI is broader than LLMs because it covers several media types, while an LLM is a deep neural network for understanding, generating, and responding to human-like text.
- LLMs are part of the hierarchy from artificial intelligence to machine learning to deep learning, and the word "large" refers to their very high parameter counts, often in the billions or more.
- Modern LLMs differ from earlier NLP models because earlier NLP models were designed for specific tasks, whereas modern LLMs can do a wide range of NLP tasks. Much of this power comes from the Transformer architecture and its attention mechanism.
- The course roadmap has three stages: first the building blocks such as preprocessing, attention, and architecture, then pre-training a foundational model, and finally fine-tuning it into task-specific applications that are needed for production use.

2 From Text to Model Inputs

Based on L08: Lecture 8: The GPT Tokenizer: Byte Pair Encoding; L10: Lecture 10: What are token embeddings?; L12: Lecture 12: The entire Data Preprocessing Pipeline of Large Language Models (LLMs).

The previous chapter established what large language models are and where transformers fit; this chapter turns to the practical question of how raw text becomes something a model can actually consume. Data preprocessing matters because it is presented as one of the most fundamental building blocks of a strongly performing large language model, and because text such as paragraphs, documents, or a PDF file cannot be given directly as input and must be processed first. The core pipeline runs from tokenization to token IDs to token embeddings to positional embeddings and finally to input embeddings, which makes the data ready to be fed into the model architecture or pre-training process.

We begin with why preprocessing exists, then build intuition with word-based tokenization, a simple Python tokenizer, and the limits of that approach, including unknown and boundary tokens. From there the chapter broadens to character and subword tokenization, introduces byte pair encoding and its role in modern LLM tokenizers, and uses the GPT-2 tokenizer through "tiktoken" while noting that the mechanics of subword BPE are only introduced indirectly here. The closing sections move from token IDs to token embeddings, embedding matrices and lookup layers, positional information, and the creation of next-token training examples, setting up the attention machinery of the next chapter.

At a glance

- Preprocessing is the procedure of processing text data before it is fed into a large language model, and it is described as a fundamental building block for strong model performance.
- A model input pipeline starts by breaking input text into tokens, converting those tokens into token IDs, and then converting the token IDs into token embeddings used to train the model.
- There are three tokenization families here: word-based, subword-based, and character-based tokenizers.
- A word-based tokenizer usually treats each word as one token, but a simple split on whitespace is not enough because punctuation should often be separated and whitespace tokens may need to be removed.
- A vocabulary maps each unique token to a unique integer token ID, typically

> by removing duplicate tokens and sorting the remaining tokens in ascending or alphabetical order.

2.1 Why preprocessing exists

How does raw text become something a large language model can actually take as input, and why is that transformation so important? Data processing is one of the fundamental building blocks of a strongly performing large language model. Data preprocessing is the procedure of processing data before it is fed as input to a large language model. The data is text such as paragraphs and documents. A big PDF file cannot be given directly as input to a large language model and needs further processing first.

The preprocessing path can be understood as a small sequence of transformations. Before a sentence is in a form good enough to be an input for a large language model, four fundamental steps are used: tokenization, token embeddings, positional embeddings, and creating input embeddings. The final input embeddings then serve as inputs to the large language model. The complete pipeline runs from tokenization to token IDs to token embeddings to positional embeddings and finally to input embeddings. This pipeline makes the data ready to be fed into the LLM architecture or the LLM pre-training process.



Figure 2.1: The preprocessing pipeline runs from tokenization through positional embeddings to input embeddings, which are then fed to the large language model.

It helps to make that flow concrete with a simple example. For an input text such as “this is an example”, the input text is first broken down. In a word-based tokenizer, “this”, “is”, “an”, and “example” are four tokens, one word per token. After tokenization, each token is converted into a token ID. Those token IDs are then converted into token embeddings. These token embeddings then serve as the input to training a large language model such as GPT. The term “token embeddings” is preferred over “word embeddings” because tokens can be words, subwords, or even characters. A fuller treatment of tokenizers and byte pair encoding is separate from this overview, but the key point here is that text must pass through these preprocessing steps before it can be used by the model.

Listing 02.01.1: This code passes token ID inputs through a token embedding layer and prints the resulting token embedding tensor shape. Source: rasbt/LLMs-from-scratch, `ch02/01_main-chapter-code/ch02.ipynb`*, Apache-2.0, commit cdbd33e.*

```
token_embeddings = token_embedding_layer(inputs)
print(token_embeddings.shape)

# uncomment & execute the following line to see how the embeddings look like
# print(token_embeddings)
```

Output (executed):

```
torch.Size([8, 4, 256])
```

Listing 02.01.2: This code creates input embeddings by adding token embeddings and positional embeddings and then prints the resulting tensor shape. Source: rasbt/LLMs-from-scratch, `ch02/01_main-chapter-code/ch02.ipynb`, Apache-2.0, commit cdbd33e.

```
input_embeddings = token_embeddings + pos_embeddings
print(input_embeddings.shape)

# uncomment & execute the following line to see how the embeddings look like
# print(input_embeddings)
```

Output (executed):

```
torch.Size([8, 4, 256])
```

2.2 Word-based tokenization foundations

How does a simple tokenizer turn raw sentences into the numbered symbols a model can use? A simple tokenization scheme takes sentences, converts them into tokens, arranges the tokens in ascending order as a vocabulary, and assigns each token a numerical token ID. In that simple tokenizer, every word is treated as a unique token, along with special characters such as commas, full stops, and exclamation marks. More generally, tokenization can be broken down into three types: word-based tokenization, subword-based tokenization, and character-based tokenization.

Word-based tokenization converts the entire dataset into chunks of words. At the sentence level, word-based tokenization means breaking a sentence down into individual words. In a word-based tokenizer, each word in a sentence is usually one token. For example, if the sentence is “my hobby is playing cricket” and the tokens are the individual words “my”, “hobby”, “is”, “playing”, and “cricket”, that is a word-based tokenizer. The same idea appears in the sentence “the fox chased the dog”, where the tokens are “the”, “fox”, “chased”, “the”, and “dog”, so every word is one token. In this word-based example,

tokens also include special symbols such as commas, exclamation marks, full stops, and quotation marks.

Once tokens have been identified, they need a lookup structure. A vocabulary is a dictionary containing a collection of tokens, and every token corresponds to a token ID. Equivalently, the vocabulary is a dictionary in which every unique token is mapped to a unique integer called the token ID. Each token is associated with an integer token ID. The tokens are sorted in ascending order before token IDs are assigned. In the example shown, the first token receives token ID 0, the second receives token ID 1, and this continues through the vocabulary.

Listing 02.02.1: This code builds a vocabulary dictionary that maps each token in ‘all_words‘ to a numeric ID using ‘enumerate‘. Source: rasbt/LLMs-from-scratch, ch02/01_main-chapter-code/ch02.ipynb, Apache-2.0, commit cdbd33e.

```
vocab = {token:integer for integer,token in enumerate(all_words)}
```

- *Checked by running the code:* `vocab[list(vocab.keys())[0]]` = 0 (the text says 0)
- *Checked by running the code:* `vocab[list(vocab.keys())[1]]` = 1 (the text says 1)

Listing 02.02.2: This code prints the first entries of the vocabulary mapping so you can see tokens paired with their assigned integer IDs. Source: rasbt/LLMs-from-scratch, ch02/01_main-chapter-code/ch02.ipynb, Apache-2.0, commit cdbd33e.

```
for i, item in enumerate(vocab.items()):
    print(item)
    if i >= 50:
        break
```

Output (executed):

```
('Gallery', 36)
('Gideon', 37)
('Gisburn', 38)
('Gisburns', 39)
('Grafton', 40)
('Greek', 41)
('Grindle', 42)
('Grindles', 43)
('HAD', 44)
('Had', 45)
('Hang', 46)
```

```
('Has', 47)
('He', 48)
('Her', 49)
('Hermia', 50)
```

2.3 Building a simple word tokenizer in Python

How does a raw document become a list of word tokens and then integer IDs that Python can work with? To build intuition, it helps to construct a word-based tokenizer from scratch before moving to the subword-based tokenizer used in GPT models. For this demonstration, a single book is enough to show the preprocessing steps, even though real large language models process millions of articles and hundreds of thousands of books. The code begins by opening the document, reading the full contents into a variable called "raw text", and then checking that the read succeeded by printing the total number of characters and a sample of the text. In the example shown here, the file has 20479 characters, and the first 100 characters are printed to verify that they match the book text.

The next problem is deciding where to split the text into tokens. Using Python's regular expression library, text can be split wherever whitespace characters occur. But splitting only on whitespace is not enough, because punctuation stays attached to words and whitespace tokens may remain in the result. Whether whitespace should be kept as a token is debatable, since it can matter in cases such as Python indentation, but for this demonstration whitespace tokens are omitted. A better approach is to split not only on whitespace but also on punctuation and other special characters, and then remove the whitespace tokens afterward. The regular expression is updated to split on characters such as commas, full stops, colons, semicolons, question marks, underscores, exclamation marks, quotation marks, brackets, slashes, and whitespace, and the resulting list is filtered so that only non-whitespace items remain. After this processing, words and punctuation become separate tokens and whitespace is no longer kept as a token.

Once the splitting rule works on a small example, the same idea can be applied to the full text. The same splitting and whitespace-removal steps are applied to the entire dataset to create a preprocessed list containing all tokens from the essay. To check the tokenization, the code prints the first 30 tokens in the list. Those tokens show that the essay has been broken into individual words, while special symbols such as the dash also appear as separate tokens. The preprocessed list contains 4690 tokens in total, including duplicates.

To create token IDs, all tokens are deduplicated, the unique tokens are sorted in ascending or alphabetical order, and IDs are assigned starting from zero. For example, with the text "the quick brown fox jumps over the lazy dog," the unique tokens are arranged in alphabetical order before IDs are assigned. This mapping is the vocabulary: a dictionary

in which every unique token is paired with a unique integer token ID. In code, constructing the vocabulary means taking the preprocessed token list, converting it into a set to remove duplicates, and then sorting it in ascending order. The vocabulary size shown here is 1130. After sorting, Python's "enumerate" function is used to assign each token an integer token ID. Because the tokens are enumerated in ascending order, the first token gets ID 0, the second gets ID 1, and so on.

A tokenizer class in Python takes the vocabulary and provides an "encode" method and a "decode" method. The "encode" method converts words or input text into token IDs, and the "decode" method converts token IDs back into tokens or text. The class keeps both a string-to-integer dictionary and a reverse integer-to-string dictionary. In "encode", the input text is split on the chosen characters, whitespace is removed, and the remaining strings are looked up in the vocabulary to return a list of token IDs. In "decode", a list of token IDs is mapped back to strings with the integer-to-string dictionary, and those strings are joined to reconstruct the text. When decoding, an extra space usually appears before punctuation after joining, so that space has to be removed.

With the class defined, you can instantiate it and test both directions of the mapping. The tokenizer instance is created by passing in the previously defined vocabulary. After that, the instance can call the "encode" and "decode" methods. A sample input text is given to the tokenizer. Encoding that sentence splits it, removes whitespace, converts each token to a token ID, and produces a list of token IDs for the sentence. Decoding those token IDs returns the same sentence.

Listing 02.03.1: This code builds a vocabulary dictionary by assigning each token in 'all_words' an integer ID using 'enumerate'. Source: rasbt/LLMs-from-scratch, `ch02/01_main-chapter-code/ch02.ipynb`, Apache-2.0, commit cdbd33e.

```
vocab = {token:integer for integer,token in enumerate(all_words)}
```

- *Checked by running the code:* `min(vocab.values())` = 0 (the text says 0)

Listing 02.03.2: This code encodes 'text' into token IDs with 'tokenizer.encode' and immediately decodes those IDs back into text with 'tokenizer.decode'. Source: rasbt/LLMs-from-scratch, `ch02/01_main-chapter-code/ch02.ipynb`, Apache-2.0, commit cdbd33e.

```
tokenizer.decode(tokenizer.encode(text))
```

2.4 Limits of word tokenization

Why does a simple word tokenizer break down once it meets real user input? A main problem with word-based tokenization is handling words that are not present in the vocabulary. These words are called out-of-vocabulary words. A vocabulary is a dictionary with a collection of tokens arranged in ascending order. If a user inputs a word that

was not present in that vocabulary, a word-based tokenization scheme can run into an error. For example, if the available data contains only a small case such as “my hobby is playing Cricket” and a new word such as “football” is not present in that data set, these schemes usually run into an error. The same issue appears when trying to encode text containing words such as “hello” that are not in the vocabulary: the tokenizer returns an error because it does not know how to encode them. Out-of-vocabulary words are like preparing for an exam and then being asked a completely different question in the exam.

Another major problem with word-based tokenization is that similar words such as “boy” and “boys” are treated as separate words despite their similarity. In a word-based scheme, different forms of related words get separate token IDs, so similarity based on a shared root word is not captured. The token IDs for such words may be far apart, and in either case the tokenization itself does not capture that they are very similar.

There is a trade-off here. Word-based tokenization produces shorter tokenized sequences because each word is one token. For example, “dinosaur” is just one token in word-based tokenization. Its disadvantages are that out-of-vocabulary words are problematic and the vocabulary size is pretty large. A major reason for the large vocabulary is that a model would need tokens for a huge number of words in the language. The number of words in English is estimated to be around 600,000 to 1 million. That makes the required vocabulary size really very high for a word-based tokenizer.

2.5 Unknown and boundary tokens in a word tokenizer

How does a word tokenizer turn text into something a computer and a language model can actually use, especially when the text contains words the vocabulary has never seen? After text has been converted into tokens, those tokens still need numerical representations. Computers do not understand words directly, so the tokens must be converted into token IDs. A vocabulary is a dictionary in which every unique token is mapped to a unique integer, called the token ID. For practical purposes, the LLM cares about token IDs rather than the original tokens, because token IDs have a numerical representation.

The difficulty appears as soon as the input contains a word that is not present in the vocabulary. A simple way to handle this is to augment the existing vocabulary with two special tokens: an unknown token and an end-of-text token. The unknown token represents a word in the input text that the vocabulary does not know, and that word is mapped to the unknown token’s ID. The end-of-text token is used when working with different input documents to specify where one document ends and another starts. When multiple text sources are combined, such as news articles, Reddit posts, blogs, and newspaper clippings, an end-of-text token is inserted between them to mark the boundary from one source to the next. These end-of-text tokens act as markers for segment boundaries, and this leads to more effective processing and understanding by the

language model. During preprocessing for ChatGPT, end-of-text tokens are also added to text resources.

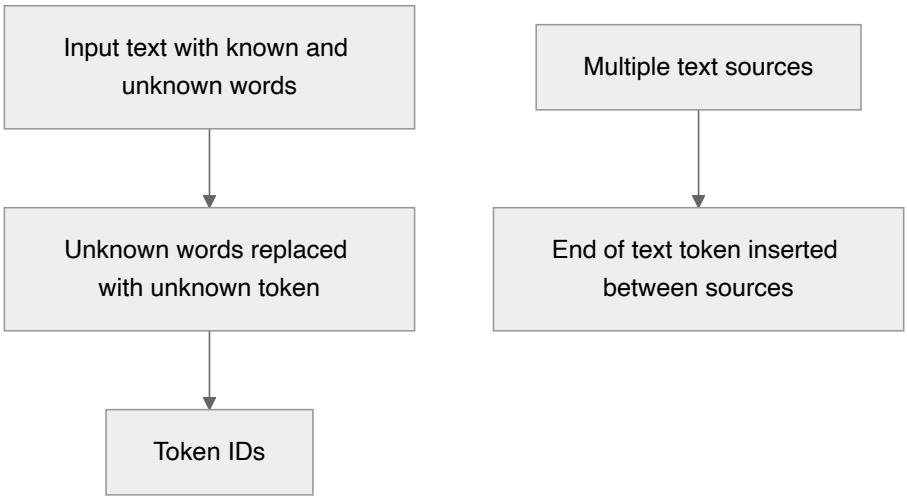


Figure 2.2: An augmented tokenizer maps unknown words to an unknown token and inserts an end-of-text token between documents.

With these two special tokens in place, the tokenizer can treat unseen words and document boundaries explicitly. The modified tokenizer uses the unknown token whenever it encounters a word that is not part of the vocabulary, and it inserts an end-of-text token between unrelated texts. After adding the end-of-text token and the unknown token to the token list, the token-to-integer vocabulary mapping is created again with these two additional entries. The vocabulary length then becomes 1132 instead of 1130. In this augmented vocabulary, the end-of-text token has token ID 1130 and the unknown token has token ID 1131. To modify the tokenizer, the decode method stays the same, but the encode method is changed so that any item not found in the string-to-integer dictionary is replaced with the unknown token before being mapped to an ID.

A concrete example shows how the revised tokenizer behaves. Take the two texts “hello do you like t” and “in the sunlight Terraces of the palace” and merge them with an end-of-text token between them. This placement of the end-of-text token indicates when the first text ends and the second text starts, and it is described as how this is done in GPT. With the modified tokenizer, encoding no longer raises an error when “hello” is not in the vocabulary, because “hello” is replaced with the unknown token. In that encoded result, the unknown token receives ID 1131, while the end-of-text token receives ID 1130. Decoding produces text in which unseen words appear as unknown tokens, as in “unknown do you like t end of text in the sunlight Terraces of the unknown”. Here, both “hello” and “palace” are treated as unknown words, and augmenting the vocabulary with the additional tokens allows encoding and decoding even unknown words successfully.

Special tokens are not limited to these two. Depending on the language model, researchers

may also use special tokens such as beginning-of-sequence, end-of-sequence, and padding tokens. For GPT, however, the tokenizer does not use an unknown token, but it does use an end-of-text token. The end-of-text token marks that one text has ended and another has started, and it is part of the GPT-2 vocabulary.

2.6 Character and subword tokenization

How can tokenization avoid huge word vocabularies without throwing away everything useful about words? At one end of the spectrum is character-based tokenization, where characters rather than words are used as tokens. For the sentence "my hobby is playing cricket," this means tokens are individual characters such as "m", "y", "h", "o", and "b" instead of whole words. This leads to a very small vocabulary because every language has a fixed number of characters. For English, the character count is stated as around 256, while the number of words is about 170,000 to 200,000. Because any new sentence can always be broken into characters already in the vocabulary, character-based tokenization avoids the out-of-vocabulary problem. It also removes the large-vocabulary problem of word-based tokenization, and the small vocabulary makes it computationally and memory efficient.

This tradeoff is severe. When text is broken into individual characters, the meaning associated with words is completely lost. Shared roots are lost as well: "boy" and "boys" no longer reveal their common root. The same issue appears with word-based tokenization: "tokenization" and "modernization" both contain the common "ization," whose meaning is lost, and it does not capture the common root in "old" and "older" or the common root in "finest" and "lowest." Another major problem is that the tokenized sequence becomes much longer than the initial raw text. For example, "dinosaur" is one token with word-based tokenization but becomes eight separate character tokens with character-based tokenization. The same pattern appears with "hobby," which becomes five character tokens: "h", "o", "b", "b", and "y." Character-level tokenization has a very small vocabulary and completely solves the out-of-vocabulary problem, but it produces much longer token sequences and loses meaning completely, while the main problem with word-based tokenization is that it needs a large vocabulary.

A middle ground is subword-based tokenization. Subword-based tokenization is another tokenization algorithm, and byte pair encoding is an example of a subword-based tokenization algorithm. It is presented as a best-of-both-worlds approach between word tokenization and character tokenization. The first rule is that frequently used words in the dataset should not be split into smaller subwords. The second rule is that rare words should be split into smaller meaningful subwords, and this splitting can go down to the character level if needed. That is why subword tokenization combines features of word tokenization and character tokenization: frequent words are kept whole, while rare words can be broken down further. It captures some root words that occur in many other words, such as treating "boy" as the common root in related forms.

The core idea becomes clear in simple examples. If “boy” appears multiple times in the dataset, it is kept as a token and is not split further. But “boys” can be split into the meaningful subwords “boy” and “s.” The main essence of subword tokenization is that words are sometimes broken into smaller elements because those smaller elements appear very frequently. In this example, “boys” is broken into “boy” and “s” because “boy” appears more frequently and “s” is also a token that appears very frequently. Breaking one word into subwords has advantages, and that is what is mainly done in subword tokenization. Subword splitting helps the model learn that words such as “token”, “tokens”, and “tokenizing” share the same root and are similar in meaning. It also helps the model learn that “tokenization” and “modernization” have different roots but share the suffix “isation,” which appears in the same syntactic situations. A word like “tokenization” can therefore be broken into subword pieces that include a suffix token such as “ization” along with root-related tokens, so the model learns that “ization” is a common suffix that appears in multiple words.

Byte pair encoding makes this subword view concrete. The tokens produced by byte pair encoding are subwords rather than full words or single characters, and they can encode root representations. For example, the process can construct tokens such as “old” and “est”. A repeated part can be kept as its own token, while a less reusable word is split into reusable subwords and characters. That is why subword encoding keeps the out-of-vocabulary advantage of character-level tokenization while also capturing root meanings and avoiding the very large vocabulary of word-level encoding.

2.7 Byte pair encoding as compression

Why is a compression method relevant when the goal is to build token vocabularies for language models? Byte pair encoding, or BPE, was introduced in 1994 as a data compression algorithm. The core idea is simple: scan the data, find the most common pair of consecutive bytes, replace that pair with a byte that does not occur in the data, and keep doing this iteratively. Put another way, the algorithm scans the data from left to right, identifies the byte pairs that occur most often, replaces them with a byte not present in the data, and repeats until no byte pair occurs more than once.

A concrete sequence makes the mechanics clear. Take the original data “a a a b d a a a b a c”. When this sequence is scanned from left to right, the most common byte pair is “AA”. “AA” also occurs the most, but it is not considered here because it is not a byte pair. In this example, “AA” occurs four times. The most frequent byte pair is then replaced with Z, since Z does not occur in the data. After replacing each occurrence of “AA”, the new sequence becomes “z a b d z a b a c”.

The process then repeats on the compressed sequence. Looking again for the most frequent byte pair, the example replaces “AB” with “Y” because “Y” does not occur anywhere in the data. After those replacements, the compressed data is “z y d z y a”. Replacing common byte pairs with another symbol is called encoding, which is why the

method is described as an encoder. In the example, “AC” appears only once, so it is not encoded further. The example then replaces “zy” with another variable, such as “w,” and then stops. The process stops when no byte pair occurs more than once.

2.8 Byte pair encoding for subword vocabularies

How can a tokenizer keep vocabulary size manageable while still preserving useful pieces of words for a language model? Modern LLMs such as GPT-2 and GPT-3 usually use byte pair encoding, so it is important to understand it. Byte pair encoding, or BPE, is a subword tokenization algorithm. It is a subword-based method because the tokens are subwords rather than full words or full characters. Byte pair encoding can be slightly modified to convert an entire sentence into subwords. In that role, it keeps the most common words as single tokens and breaks rare words into two or more subword tokens. A simple example is that a frequent word like “boy” can stay as one token, while a rarer related word like “boys” can be split into “boy” and “s”. This preserves a shared root token across related words, and the same idea helps with pairs such as “tokens” and “tokenizing”.

Consider a small data corpus of words. In this corpus, the words are “old”, “older”, “finest”, and “lowest”. With word-based tokenization, these would simply be four tokens, while character-level tokenization would make each character its own token. Before applying BPE for subword tokenization, each word is augmented with an end-of-word marker so the tokenizer knows where the word ends. In the example, “old”, “older”, “finest”, and “lowest” are all written with the added end token. The first step is then to split the words into characters and make a frequency table using the word frequencies in the corpus. Here, “old” appears 7 times, “older” 3 times, “finest” 9 times, and “lowest” 4 times. Because every word token includes the end-of-word marker, that marker appears once for every word token in the corpus, which is 23 times here. At this character level, the example has 12 tokens, and pure character-level tokenization would stop at that point.

The modified BPE procedure repeatedly finds the most frequent pairing in the dataset and merges that pair, then performs the same iteration again. In the first iteration of this corpus, the most frequent pair is “e” and “s”. That pair occurs 13 times in total: 9 times in “finest” and 4 times in “lowest”. The tokenizer merges them into the subword token “es” and updates the token table. When “es” is added, its count is 13, 13 is subtracted from the count of “e”, and the count of “s” drops to zero in this example. That also shows that in this corpus, “s” never appears without “e”. Once “es” becomes a token, the pair “es” and “t” is merged into the token “EST,” which appears 13 times, while “es” drops to zero occurrences. This identifies “EST” as a common root across “finest” and “lowest”, something the word-based and character-based tokenizers do not capture.

The next merge uses the end-of-word marker. After “EST” is formed, the pair “EST” and the end-of-word marker appears 13 times, more than any other pair in these words.

Merging “EST” with the end-of-word marker encodes that “EST” is an ending sequence in this dataset. That distinction lets the tokenizer treat a word like “highest” differently from “estimate”, where “EST” is not at the end. Once this final merged token is created, the intermediate tokens “es” and “EST” are no longer needed. Another frequent sequence is “o” with “l”, which occurs 10 times because it appears in “old” and “older”. That pair is merged into a new token, and then that token is merged with “d” to form another token that also appears 10 times. The token “old” captures a root shared by both “old” and “older”, preserving structure that was not captured here by word-level or character-level tokenization. By contrast, a frequent sequence that appears only inside one word, such as the beginning of “finest”, is not a useful merge just because it repeats within that single word.

After the merges are constructed, tokens whose frequency has become zero can be removed to obtain the final token table. Those final tokens serve as the vocabulary for later training steps such as vector embedding. BPE stops merging according to a stopping criterion, such as reaching a target token count or a chosen number of iterations. Compared with word-level encoding, this can reduce the number of tokens because related words like “boy” and “boys” do not both need separate full-word tokens. This is why BPE is presented as the best of both worlds: it keeps vocabulary size efficient while retaining root-word meaning.

A second advantage is handling text that is not already in the vocabulary. Byte pair encoding helps solve the out-of-vocabulary problem because its vocabulary includes subwords and sometimes characters. A word-based tokenizer can fail on an unknown word, but a BPE tokenizer can still produce token IDs for it. The underlying algorithm breaks words that are not in the predefined vocabulary into smaller subword units or even individual characters. Because of that, an unfamiliar word can be represented as a sequence of subword tokens or characters. To encode such text, the tokenizer scans from left to right, breaks it into characters or subwords based on what exists in the vocabulary, and assigns each piece a token ID. That is why BPE does not need a special unknown token to deal with unseen words. The highlighted advantages are that it keeps the vocabulary from becoming too large, retains root-word meaning, and automatically deals with unknown words. Character-level encoding is not generally used in large-scale LLMs.

2.9 Using the GPT-2 tokenizer with tiktoken

How can a Python program use the same kind of byte pair tokenizer as GPT-2 without reimplementing the algorithm from scratch? Implementing byte pair encoding from scratch is relatively complicated, so a practical approach is to use the Python open-source library “tiktoken”. “tiktoken” is essentially a byte pair encoder used for OpenAI models, and it is the tokenizer used in OpenAI models. To use it, the first step is to install the “tiktoken” library. After installation, the GPT-2 byte pair tokenizer is instantiated

with a call such as “tiktoken.get_encoding(”gpt2”)“, storing the result in a tokenizer object. That tokenizer object is similar to the earlier Python tokenizer classes because it has”encode” and “decode” methods, but here the tokenizer is created in one line through the library.

With that tokenizer in hand, the next step is to test encoding and decoding on actual text. An example text is encoded with the tokenizer while allowing the special end-of-text token, and token ID 50256 denotes the end-of-text token. In the GPT-2 tokenization scheme, token ID “50256” is the end-of-text token. The GPT-2 and GPT-3 byte pair encoding scheme uses a vocabulary of around 50,000 tokens, and a word-level English vocabulary would require roughly 170,000 to 200,000 words. Byte pair encoding reduces the vocabulary size while still giving strong performance. When the token IDs for the example are passed to the tokenizer’s decode method, it reconstructs exactly the same sentence that was originally given.

> **Note.** The course’s reference code differs here: The reference gives GPT-2’s exact BPE vocabulary as 50,257; the lecture additionally contrasts this with an estimated 170,000–200,000 English word-level vocabulary.

Two properties of the GPT-2 byte pair vocabulary are worth noting. In the GPT-2 byte pair tokenizer, the end-of-text token is the last token in the vocabulary, so it has the largest token ID, “50256”. The byte pair tokenizer used for GPT-2, GPT-3, and the original model used in ChatGPT has a total vocabulary size of “50257”. A byte pair encoding tokenizer can encode and decode unknown words without any special unknown-word token because it breaks text down to characters and subwords. For an unknown word, the tokenizer would ideally show an error, but when encoding there is no error.

This becomes even clearer on deliberately made-up words. A final example uses the nonsense sentence “a k w i r w i e r,” described as completely random words that do not make sense. When these random words are passed to the “tiktoken” GPT-2 byte pair encoder, it does not raise an error and still encodes them into tokens. Decoding the resulting integers gives back exactly the same sentence that was originally encoded. The tokenizer created with “get_encoding” can then be tested with both “encode” and “decode”.

Listing 02.09.1: This code creates a GPT-2 tokenizer by calling tiktoken.get_encoding(“gpt2”). Source: rasbt/LLMs-from-scratch, `ch02/01_main-chapter-code/ch02.ipynb`, Apache-2.0, commit cdbd33e.

```
tokenizer = tiktoken.get_encoding("gpt2")

text = (
    "Hello, do you like tea? <|endoftext|> In the sunlit terraces"
     "of someunknownPlace."
```

```
)

integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})

print(integers)

strings = tokenizer.decode(integers)

print(strings)
```

Output (executed):

```
[15496, 11, 466, 345, 588, 8887, 30, 220, 50256, 554, 262, 4252, 18250, 8812, 2114, 1659
Hello, do you like tea? <|endoftext|> In the sunlit terracesof someunknownPlace.
```

- *Checked by running the code:* `tokenizer.encode('<|endoftext|>', allowed_special={'<|endoftext|>'})[0]` = 50256 (the text says 50256)
- *Checked by running the code:* `tokenizer.n_vocab - 1` = 50256 (the text says 50256)
- *Checked by running the code:* `tokenizer.n_vocab` = 50257 (the text says 50257)

2.10 Why token embeddings are needed

Why are token embeddings needed after tokenization has already turned text into token IDs? Token embeddings are also called vector embeddings, and they are sometimes called word embeddings, but "word embeddings" is not entirely accurate because tokens can be words, subwords, or even characters. A word-based tokenizer makes each token one word, but GPT uses byte pair encoding as a subword tokenizer, so even parts of words or characters can be individual tokens. Computers require numerical representations of words, because they cannot understand words directly as inputs to a machine learning model or a large language model. Token embeddings are the step that converts token IDs into vectors.

The problem is not numbers by themselves, but what those numbers fail to represent. Assigning random numbers or token IDs to words does not capture semantic relationships between related words such as "cat" and "kitten". One-hot encoding has the same limitation: it does not capture semantic relationships between words such as "dog" and "puppy". Words carry meaning, so inputs to a large language model should exploit that meaning rather than discard it.

A more useful numerical representation is to encode each token as a vector whose dimensions correspond to features. For example, a vector can use features such as "has a

tail", "is eatable", "has four legs", "makes sound", and "is a pet", with high values when a feature applies. With this kind of representation, "dog" and "cat" are close to each other because similar dimensions have high or low values in both vectors. "Apple" and "banana" are also close to each other, while words with different feature values can be far apart.

This is the central idea behind embeddings. Representing words as vectors can capture semantic meaning, allowing similar words to be grouped together and dissimilar words to be farther apart. These vectors are called vector embeddings or token embeddings because each token is converted into a vector embedding. Representing words as vectors so that semantic relationships are preserved is called vector embedding, and it is also called token embedding. If token embeddings are trained properly, those vectors can carry meaning. The key takeaway is that vectors can capture semantic meaning. Because these embeddings retain the meaning of words, feeding them into a large language model makes a huge difference compared with feeding one-hot encodings or random token IDs.

Semantic meaning is not the whole story, however. The vector embeddings discussed so far do not encode the position of a word in a sentence. A word's position also matters for understanding sentence context. That is why it is advantageous to encode positional information along with semantic meaning. The main purpose of positional encoding is to inject positional information that token embeddings alone do not provide.

2.11 What embeddings capture

How can a model use the meaning already present in words instead of treating them as unrelated IDs? A useful analogy comes from vision: convolutional neural networks work well because they exploit spatial relations between pixels in an image, such as parts that are close to each other, rather than only flattening pixel values into one input vector. Text has a similar kind of structure to exploit: words carry meanings, and some words are closer in meaning to other words. Examples such as "cat" and "kitten" or "dog" and "puppy" show the kind of semantic closeness that should be represented. Token embeddings, also called vector embeddings, represent words or tokens as vectors in a way that preserves semantic relationships.

The central challenge is where these vectors come from. To obtain vector or token embeddings, a neural network has to be trained to create them from text. The training signal comes from sentences in textual documents, because they reveal which words are closer or more similar and therefore should have similar vectors. Creating the right embeddings is not easy. This is computationally expensive: GPT-2 is described as having a vocabulary of 50,000 words, and creating a vector embedding for each word is why training GPT takes a huge amount of time. In GPT, every word is converted into a vector in a high-dimensional space, and a neural network is trained to generate those vectors. Those embeddings are not taken from a separate pre-trained dataset in large

language models, but are trained along with the model itself, including the next-word prediction task.

Pre-trained embeddings make the idea easier to inspect directly. Many large companies, including Google, already provide pre-trained token embeddings. One example is "Word2Vec Google News 300", which consists of pre-trained vectors learned from a Google News dataset of about 100 billion words. In this name, "300" is the embedding dimension: each word is mapped into a 300-dimensional vector space. A five-dimensional space is described as not really enough to capture all meaning, which motivates using higher-dimensional embeddings such as 300 dimensions. A pre-trained Word2Vec model can take a word as input and convert it into a 300-dimensional vector. In code, the model is assigned to a dictionary called "word_vectors", where each word is associated with its embedding vector. Looking up the vector for "computer" and printing its shape shows a 300-dimensional vector, illustrating that every word is encoded into a 300-dimensional vector.

Once vectors are available, their structure can be tested. If embeddings really encode meaning, then vector arithmetic should preserve semantic relationships, and the example "king" plus "woman" minus "man" should produce something similar to "queen". To test this, the vectors for "king" and "woman" are added, the vector for "man" is subtracted, and then the most similar words to the resulting vector are printed. The top result is "queen", with a reported score of around 71%, which is presented as evidence that the vectors encode semantic information. The explanation given is that these vectors capture semantic properties such as masculinity and femininity, and that related pairs such as "king" and "queen" or "man" and "woman" are closer to each other. When embeddings are trained well, as in the Word2Vec Google News model, they can encode meaning in the vectors.

Semantic relationships also appear in similarity and distance measurements. Related pairs such as "woman" and "man" and "king" and "queen" are expected to have similar embeddings, and pairs such as "uncle" and "aunt", "boy" and "girl", and "nephew" and "niece" are described as related. To check relatedness in the embedding space, words are converted into vectors and their similarity is tested, described here as finding the distance between vectors. The code computes similarity scores between word vectors for pairs such as "woman" and "man". The reported similarity scores are high for the related pairs and low for "paper" and "water", because the related vectors are closer to each other while the unrelated ones are far apart. Embeddings can also be used to find words similar to a given word; for "tower", the shown similar words include "scraper", "tower", and "spire". Another check computes the norm of the difference between two word vectors with "np.linalg.norm" to measure how far apart their embeddings are. The magnitude of that difference is presented as an indication of how close or far the meanings of the corresponding words are. Words with similar meanings therefore have embeddings that are closer together in space, so the magnitude of the difference between their vectors is smaller than for unrelated words.

2.12 Embedding matrices and lookup layers

How does a model turn token IDs into the vectors it will actually train on, and what exactly is stored in the embedding layer? To create token embeddings for a large language model, start with the vocabulary, assign token IDs to the tokens, and convert each token ID into an embedding vector. A token embedding converts each token ID into an embedding vector, and the matrix that stores these vectors is also called the embedding layer weight matrix. To construct this matrix, two dimensions matter: the vocabulary size and the vector dimension. Together, these two parameters determine the size of the embedding matrix: the number of rows equals the vocabulary size, and the number of columns equals the vector dimension.

GPT-2 provides a concrete example. For GPT-2, the vocabulary size is stated as 50,257 and the embedding dimension as 768 for the smallest model. Its vocabulary consists of 50,257 subword tokens made through byte pair encoding, with corresponding token IDs. For each token ID, there is a corresponding vector of dimension 768. That means the embedding layer weight matrix has 50,257 rows and 768 columns. This matrix is called the embedding layer weight matrix and is extremely important.

When training GPT-2, you know that the embedding layer weight matrix must have 50,257 rows and 768 columns, but you do not know the actual weight values in advance. The first step is to initialize all embedding layer weights randomly. This random initialization is the starting point for learning, and the embedding layer weights are then optimized during LLM training. This optimization process is how token embeddings are created. Training data provides information about which words are closer to each other and which are farther apart, and that information is used to modify the embeddings. Backpropagation is used to optimize the weights of the embedding layer weight matrix. A large language model is a giant neural network, and training the embedding layer weight matrix is one part of that network's training. During GPT training, the model learns the embeddings while also training to predict the next word.

A small example makes the matrix shape and the lookup behavior easier to see. In a simplified example, the vocabulary size is 6 and the embedding dimension is 3, so each token is converted into a three-dimensional vector. In PyTorch, an embedding layer is created with "torch.nn.Embedding" by specifying the vocabulary size and the vector dimension. PyTorch describes the embedding as a simple lookup table that stores embeddings of a fixed dictionary and size. Using "torch.nn.Embedding" initializes the embedding matrix with random values. With six token IDs and three-dimensional vectors, the embedding layer weight matrix has six rows and three columns, and each row corresponds to one token ID's vector. Accessing "embedding_layer.weight" returns all the weights of the embedding layer, and printing them shows a randomly initialized matrix with the expected shape. These initial weights are small random values that are later optimized during LLM training, partly so the embeddings can capture meaning. At a broad level, LLM optimization includes optimizing the embedding layer weights and a second set of weights needed later to predict the next word.

Once the matrix exists, using it is just row lookup. Each row of the embedding weight matrix is the vector embedding for one token or token ID. To get the vector for a specific token ID, look up the corresponding row in the embedding weight matrix using zero indexing. For example, the vector returned for token ID 3 is identical to the fourth row of the matrix because the IDs are zero-indexed. An embedding layer is essentially a lookup operation that retrieves rows from the embedding layer's weight matrix using a token ID. A useful way to think about it is as a lookup table: go to the row number associated with the token ID and retrieve that vector. The same matrix is therefore both a matrix of embedding weights and a lookup table for the exact vector representation of a token ID. If someone asks what an embedding layer is, a precise answer is that it is a simple lookup operation that retrieves the vector for a particular token ID.

The same lookup works for one ID or for many IDs at once. When input IDs are passed into the embedding layer, it goes through each ID, looks up the corresponding row in the embedding matrix, and returns those rows as the output embeddings. You can pass either a single token ID or multiple token IDs to the embedding layer and get the corresponding vectors in one operation. In another small example, if the vocabulary size is 4 and the embedding dimension is 5, the embedding matrix has 4 rows and 5 columns, and each token ID gets a five-dimensional vector. If only certain IDs are needed, passing those IDs to the embedding layer extracts only the corresponding rows from the embedding matrix.

The practical GPT-style code example uses the same idea at a larger scale. A practical embedding layer is created using the same embedding function with GPT-2's vocabulary size of 50,257 and a vector dimension of 256, giving an embedding matrix with 50,257 rows and 256 columns. Here each token ID is represented as a 256-dimensional vector. The input token IDs from the data loader are passed into this token embedding layer to obtain the corresponding embedding vectors. If the input ID matrix has shape 8 by 4 and each token ID is converted into a 256-dimensional vector, the embedding output has shape 8 by 4 by 256. The token vocabulary is described as subwords made through byte pair encoding.

Listing 02.12.1: Creates a token embedding layer with a vocabulary size of 50257 and an embedding dimension of 256 using torch.nn.Embedding. Source: rasbt/LLMs-from-scratch, `ch02/01_main-chapter-code/ch02.ipynb`, Apache-2.0, commit cdbd33e.

```
vocab_size = 50257
output_dim = 256

token_embedding_layer = torch.nn.Embedding(vocab_size, output_dim)
```

- *Checked by running the code:* `vocab_size` = 50257 (the text says 50257)
- *Checked by running the code:* `vocab_size` = 50257 (the text says 50257)
- *Checked by running the code:* `token_embedding_layer.weight.shape[0]` = 50257 (the text says 50257)

- *Checked by running the code:* `token_embedding_layer.weight.shape[0]` = 50257 (the text says 50257)
- *Checked by running the code:* `token_embedding_layer.weight.shape[0]` = 50257 (the text says 50257)
- *Checked by running the code:* `token_embedding_layer.weight.shape[1]` = 256 (the text says 256)
- *Checked by running the code:* `output_dim` = 256 (the text says 256)

Listing 02.12.2: Passes input token IDs to the embedding layer and prints the resulting embedding vectors. Source: rasbt/LLMs-from-scratch, `ch02/01_main-chapter-code/ch02.ipynb`, Apache-2.0, commit cdbd33e.

```
print(embedding_layer(input_ids))
```

Output (executed):

```
tensor([[ 1.2753, -0.2010, -0.1606],
        [-0.4015,  0.9666, -1.1481],
        [-2.8400, -0.7849, -1.4096],
        [ 0.9178,  1.5810,  1.3010]], grad_fn=<EmbeddingBackward0>)
```

2.13 Embedding layers versus one-hot plus linear layers

How can a token embedding lookup be viewed as an ordinary neural-network operation, and why does that matter when turning token IDs into vectors? An embedding layer is the same operation as a neural network linear layer. To reproduce an embedding lookup with a linear layer, start from token IDs such as 2, 3, and 1, convert them into one-hot representations, and feed those vectors into a neural network. The network uses as many neurons as the embedding dimension, so in the example with five-dimensional embeddings it has five neurons. For this linear layer, the output is given by X multiplied by W transpose.

This formulation also makes the output shape easy to interpret. When there are three input tokens and five embedding dimensions, X multiplied by W transpose produces a matrix with three rows and five columns. There are three rows because there are three inputs, and each row corresponds to the vector embedding for one particular input. There are five columns because each input is mapped to a vector embedding with five dimensions. This linear-layer construction produces the same output as the embedding layer.

Even so, the equivalent formulation is not the one normally used for embeddings. The embedding layer is preferred because it is much more computationally efficient than using a linear layer on one-hot inputs. A linear layer is less efficient here because one-hot

encoding introduces many zeros, which leads to many unnecessary multiplications with zero and other unnecessary computations. Those unnecessary computations scale up when the vocabulary is very large, such as GPT-2's vocabulary size. So although a linear layer can be used to create the embedding matrix, the embedding layer is the preferred implementation.

2.14 Positional embeddings and input embeddings

How do token IDs become the actual vectors a large language model receives, and why is position part of that transformation? After tokenization and token IDs, the next preprocessing steps are token embeddings and positional embeddings. Token embeddings convert token IDs into vectors, and these token IDs are encoded into a higher-dimensional vector space. Positional embeddings encode information about the position. They are needed in addition to token embeddings because token embeddings alone do not include positional information, and that additional positional information is helpful for large language models. To create input embeddings, positional embeddings are added to token embeddings, and those input embeddings are the final input to the large language model. In short, input embeddings are the sum of token embeddings and positional embeddings.

Before constructing those vectors in code, it helps to distinguish the main kinds of positional information. The terms positional embedding and positional encoding are used interchangeably here. There are two types of positional encoding: absolute positional encoding and relative positional encoding. In absolute positional encoding, each position in the input sequence has a unique positional embedding vector that is added to the token embedding. For example, with the tokens "cat", "sat", "on", and "the", each token can start from a similar token embedding, but the first, second, third, and fourth positions receive different positional embedding vectors. Adding those positional vectors to the token embeddings produces the final input embeddings. Because position is encoded, the final input embeddings can differ even when the token embeddings are similar. For the same reason, the input embeddings for "cat sat on the mat" differ from those for "the mat the cat sat on" when position is taken into account. The positional embedding vectors must have the same dimension as the token embeddings because the two vectors are added together.

Relative positional encoding emphasizes a different notion of position. Relative positional encoding focuses on the relative positions or distances between tokens rather than their exact positions. Absolute positional encoding is well suited when the fixed order of tokens is crucial, such as sequence generation. GPT uses absolute positional encoding, and absolute positional encoding is more commonly used than relative positional encoding. Relative positional encoding is suitable for tasks like language modeling or long sequences where the same phrase can appear in different parts of the sequence.

With those definitions in place, the practical construction is straightforward. For a batch size of eight, a context length of four, and an embedding dimension of 256, the token embedding layer produces token embeddings with shape 8 by 4 by 256. The token embeddings for one batch are then combined with positional embeddings, and that sum becomes the input to the large language model. The next step is to create a separate embedding layer for positional embeddings. When creating the positional embedding matrix, the relevant number of positions is determined by the context length. Here the positional embedding dimension is 256, matching the token embedding dimension, because the positional and token vectors must be added. With a context length of four, only four positional vectors are needed, one for each possible token position. Therefore, the positional embedding matrix has shape 4 by 256. In code, the positional embedding layer is created with four rows and 256 columns, and the four positional vectors are obtained from it.

The final step is the addition itself. The same four positional vectors are added to each input sequence in the batch. After creating the positional embeddings, they are added to the token embeddings. When adding token embeddings of shape 8 by 4 by 256 and positional embeddings of shape 4 by 256 in Python, broadcasting duplicates the positional embeddings across the batch dimension to make the addition possible. The input embedding matrix is constructed by adding the positional embedding vectors to each row of the token embedding matrix to produce the corresponding rows of the input embeddings. The positional embedding matrix is initialized randomly, similar to the token embedding matrix. Like the values in the token embedding layer, the values in the positional embedding layer must be optimized during training.

Listing 02.14.1: This code adds the positional embeddings to the token embeddings to form input embeddings and prints the resulting tensor shape. Source: rasbt/LLMs-from-scratch, `ch02/01_main-chapter-code/ch02.ipynb`, Apache-2.0, commit cdbd33e.

```
input_embeddings = token_embeddings + pos_embeddings
print(input_embeddings.shape)

# uncomment & execute the following line to see how the embeddings look like
# print(input_embeddings)
```

Output (executed):

```
torch.Size([8, 4, 256])
```

- *Checked by running the code:* `input_embeddings.shape[0]` = 8 (the text says 8)
- *Checked by running the code:* `input_embeddings.shape[1]` = 4 (the text says 4)
- *Checked by running the code:* `input_embeddings.shape[2]` = 256 (the text says 256)

2.15 Creating next-token training examples

How does raw tokenized text become the concrete input and target sequences a language model trains on, and why does that matter before looking at embeddings? Creating input-target pairs is one part of data preprocessing. Large language models are trained to predict the next word. For example, if the input is “llm”, the next word is “learn”; if the input is “llms learn”, the next word is “to”; if the input is “llms learn to”, the next word is “predict”; and if the input is “the llms learn to predict”, the next word is “one”. To represent this numerically, the preprocessing first chooses a context size. Context size is the maximum length of the input that the large language model sees at one time, meaning the number of tokens it processes to predict the next word. The example here uses a context size of four, but large models usually use a much larger number of tokens, often more than 150 or 200 and sometimes around 500.

Once the context size is fixed, the text sample is broken into input-output pairs based on that size. With the sample sequence “one word at a time”, the model should predict “word” from “one”, then “at” from “one word”, then “a” from “one word at”, and then “time” from “one word at a”. One input-output pair therefore contains multiple prediction tasks. In the four-token example, each pair contains four prediction tasks. Before introducing token embeddings or positional embeddings, it is important to understand how data is loaded, how inputs and outputs are given to the model, and why context size and stride matter.

A PyTorch data loader can be used to create these input-target pairs with a sliding window approach. The code reads the raw text, encodes it with the byte pair tokenizer from tiktoken, and prints that the encoded text has length 5145 tokens. The same input-output pairs can be viewed either as token ID sequences or in textual format showing the corresponding outputs. For example, one ID-based pair is described as “290” to “4920”, then “290” and “4920” to “2241”, and the textual version is shown with inputs such as “and” paired with “established” and “and established” paired with “himself”. To build the data loader, first encode the entire text into token IDs and then chunk the text into overlapping sequences.

Stride is how many positions the input window moves to create the next input sequence. When the stride is four, consecutive inputs do not overlap. In that case, examples are described with a first input such as “in the heart of” and a second input such as “the city stood the”. When the stride is one, consecutive inputs overlap, as in moving from “in the heart of” to “the heart of the”. The method is called a sliding window approach because the input window slides by the stride amount to create the next input sequence. The final tensors created by this process are input IDs and target IDs.

In this setup, each row of the input tensor is one input to the large language model, and the corresponding row of the target tensor is the matching output. Corresponding rows therefore form input-output pairs: the first row of tensor “X” matches the first row of tensor “Y”, the second row matches the second row, and so on. The target tensor is the

input tensor shifted by one word because the model is trained to predict the next word. To construct the tensors, the code loops over the token IDs using the context length and stride, appends each input chunk to the input ID tensor, and appends the corresponding target chunk to the target ID tensor. An input chunk is four token IDs at one time, and a target chunk is that same chunk shifted by one position. As the code runs, each iteration appends one row to the input tensor and the corresponding row to the target tensor, and looping over the whole text produces all training inputs and outputs.

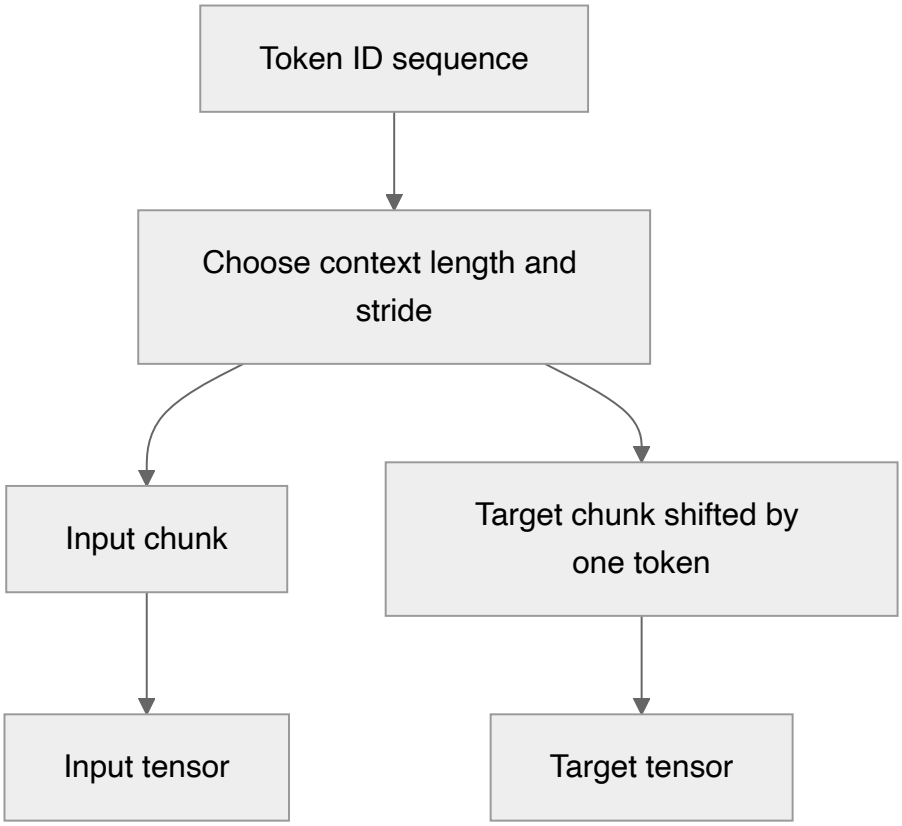


Figure 2.3: A sliding window over token IDs creates input chunks and target chunks, where each target chunk is the input chunk shifted by one token.

After the input-output pairs are created, the dataset is passed to a data loader, which processes the dataset and generates input-output batches from the input and output tensors. Batch size is how many input batches are processed at one time before updating the parameters, which is the same notion of batch size used in other machine learning frameworks. A batch size of one means one input-output pair is processed before a parameter update, while a batch size of four means four input-output pairs are processed before updating the parameters. The number of workers specifies how many threads the code is split across on the CPU for parallel processing. With a batch size of eight, the

model parameters are updated only after the entire batch is processed. In the shown batch example, the input has eight inputs and the output has eight rows, where each row corresponds to an input-output pair. The input data is arranged in batches of eight with a context length of four, so each batch contains eight sequences of four tokens each. Each input-output pair in that example contains four words in the input and four next-word prediction targets in the output, so each pair has four prediction tasks.

Key takeaways

- Text has to be preprocessed before a large language model can use it, and the overall pipeline goes from tokenization to token IDs to token embeddings to positional embeddings and finally to input embeddings.
- Tokenization turns text into tokens, and a simple word-based tokenizer treats each word and some punctuation marks as tokens, builds a sorted vocabulary, and assigns each token a numerical token ID.
- Word-based tokenization can fail on out-of-vocabulary words, treats related words as separate tokens, and needs a very large vocabulary.
- Character-based tokenization avoids out-of-vocabulary words and keeps the vocabulary small, but it loses the meaning associated with words and makes tokenized sequences much longer, while subword tokenization keeps frequently used words whole and splits rare words into smaller meaningful subwords.
- Modern LLMs commonly use byte pair encoding, a subword method that helps keep vocabulary size manageable, preserves useful root information, and can still encode unseen words without a special unknown token.
- Token embeddings convert token IDs into vectors that can preserve semantic meaning, positional embeddings add word-order information, and adding them together produces the input embeddings that are fed to the model.
- For next-word training, preprocessing turns tokenized text into input-target pairs with a chosen context size and stride, where each target sequence is the input sequence shifted by one token, and batches group many such pairs together.

3 Attention and Transformer Building Blocks

Based on L14: Lecture 14: Simplified Attention Mechanism - Coded from scratch in Python | No trainable weights; L16: Lecture 16: Causal Self Attention Mechanism | Coded from scratch in Python; L18: Lecture 18: Multi Head Attention Part 2 - Entire mathematics explained; L20: Lecture 20: Layer Normalization in the LLM Architecture; L22: Shortcut connections in the LLM Architecture.

The previous chapter carried raw text through tokenization, token IDs, and embeddings; here the focus shifts to what embeddings still lack and how transformer blocks supply it. A token embedding is a vector representation in a higher-dimensional space, and these vectors capture semantic meaning, with related words lying closer together, but an embedding by itself does not say how a word relates to the other words in its sentence. That missing relational information matters because next-word prediction needs context, and attention is the mechanism that turns each embedding into a context vector enriched with both token meaning and sentence-level relationships.

We begin by reconnecting tokens, embeddings, and context, then build intuition for simplified self-attention, attention scores, normalization, softmax, and context-vector computation before vectorizing the whole process over a sequence. From there, the chapter explains why trainable "Q", "K", and "V" projections are needed, then moves into causal masking, dropout, batching, and efficient multi-head attention implementations. It closes with the transformer blocks wrapped around attention itself, namely layer normalization and shortcut connections, while leaving fuller linear-algebra and matrix-shape intuition only partly spelled out and carrying the assembled pieces forward into the next chapter on putting a GPT model together.

At a glance

- An embedding vector is a token's vector representation in a higher-dimensional space, and embedding vectors capture semantic meaning so that semantically related words are closer together.
- An input embedding carries the meaning of a word itself, but it does not carry information about how that word relates to the other words in the sentence.
- Attention converts input embeddings into context vectors, and each context vector keeps token meaning while also encoding how that token relates to the other words in the sentence.

- For a given query token, attention scores are computed against every input token using the dot product, and higher dot products mean the vectors are more aligned and therefore receive higher attention.
- Attention scores are normalized into attention weights that sum to one so they can be interpreted as percentages of attention paid to different tokens.

3.1 From tokens to embeddings and context

How does a sentence become the vectors that attention can work on, and why is more than a plain embedding needed for next-word prediction? A large language model data pipeline first converts words into tokens, then tokens into token IDs, and then token IDs into vector embeddings. For the demonstration, the sentence "your journey starts with one step" has six tokens, and each token is assigned a three-dimensional vector embedding. GPT uses byte pair encoding, described here as a subword tokenizer. Because of that, one token is not generally equal to one word, but for simplicity the terms "token" and "word" are used interchangeably here. A vector embedding is the vector representation of a token in a higher-dimensional space. In large language models such as GPT, that space can be much larger, with dimensions described here as 500, 700, or even more than 1000, but the running example uses three dimensions for simplicity and illustration.

These embeddings already carry useful meaning, but they are still not enough. Embedding vectors capture the semantic meaning of words, so semantically related words are closer to each other in the vector space. Examples given for this closeness include "cat" and "puppy", "cat" and "kitten", and "dog" and "puppy". An input embedding encodes the meaning of a word itself, but it does not contain information about how that word relates to the other words in the sentence. For example, the embedding for "journey" does not say how "journey" relates to words such as "your", "starts", "with", "one", and "step", or how much relative importance those words have with respect to "journey". That relational information is crucial for predicting the next word, because the model needs context and needs to know how much attention should be paid to the different words in the sequence.

Attention is introduced to add exactly this missing context. The goal of an attention mechanism is to take the input embeddings for tokens and transform them into context vectors, also called context embeddings. A context vector is an enriched embedding vector: it contains the semantic meaning of the word and also information about how that word relates to other words in the sentence and how much attention should be paid to them. This contextual information helps next-word prediction and makes it better and more reliable. Attention produces one context vector for each input embedding vector. That same goal is shared by simplified attention and by the multi-head attention used in modern large language models: convert embedding vectors into context vectors that are then used as inputs to the model.



Figure 3.1: Attention transforms token embeddings into one context vector per token, enriching each vector with relational information used by the model.

To make the notation concrete, the input embeddings are written as “x”. “x1” denotes the vector representation of the first token, “x2” the vector representation of the second token, and “x3” the vector representation of the third token. For the word “journey”, which is the second word in the sentence, the input embedding is “x2”, and its context vector is denoted by “z2”. In the Python implementation, the inputs are stored as a tensor containing six three-dimensional embedding vectors, one for each token in the sentence. In that tensor, each row represents a token and each column represents a vector dimension, so with three dimensions there are three columns.

Listing 03.01.1: Passes the token ID tensor through the embedding layer and prints the resulting token embedding vectors. Source: rasbt/LLMs-from-scratch, `ch02/01_main-chapter-code/ch02.ipynb`, Apache-2.0, commit cdbd33e.

```
print(embedding_layer(input_ids))
```

Output (executed):

```
tensor([[ 1.2753, -0.2010, -0.1606],
        [-0.4015,  0.9666, -1.1481],
        [-2.8400, -0.7849, -1.4096],
        [ 0.9178,  1.5810,  1.3010]], grad_fn=<EmbeddingBackward0>)
```

3.2 Simplified self-attention intuition

How does a model decide which input tokens matter when it builds a context vector for one token? The aim is to create a context vector from each input embedding vector, and for the second word, “journey”, the input embedding is “x2” and its context vector is denoted by “z2”. A context vector is needed for all input vectors, not just for “journey”. To compute a context vector from an embedding vector, it is necessary to know how much importance should be given to each input token, and this is captured by attention weights. Attention weights specify how much attention should be given to each input token when computing the context vector for an embedding vector. Based on these attention weights, the context vector is derived and denoted by z.

To make the method concrete, start with one query token and then generalize. For demonstration, the computation starts with “journey”, and the same analysis is then

applied to the other words in the sentence. The token currently being focused on is called the query. When the token is “journey”, that query is “x2”, and its corresponding context vector is “z2”. This “z2” embedding contains information about “x2” and about all the other input elements, so it contains information not only about “journey” but also about the rest of the input. The first step in turning the embedding vector for the query into a context vector is to compute intermediate values “w”, which are also called attention scores. These attention scores quantify how much importance or attention should be paid to each input word for the query “journey”, and there is one attention score between the query and every input vector. Finding these attention scores is the first step toward the context vector because they tell us how much importance to give each token in the input.

If embedding vectors are aligned or have closer angles, that suggests similarity in meaning. Because of that, more importance should be paid to the vector for “starts” when the query is “journey”, since “starts” appears more aligned with “journey”. By contrast, the vector for “one” is almost perpendicular to the vector for “journey”, which indicates less similarity in meaning and therefore less importance than “starts” for the query “journey”. The mathematical operation used to quantify this alignment is the dot product. The dot product of two vectors equals the product of their magnitudes multiplied by the cosine of the angle between them. If two vectors are aligned, the angle is zero, the cosine is one, and the dot product is maximal. If two vectors are perpendicular, the angle is 90 degrees and the cosine is zero. A higher dot product means the vectors are more aligned, and a lower dot product means they are less aligned, so the dot product encodes how aligned or far apart vectors are.

This makes the attention mechanism much less mysterious. Attention scores can be computed by taking the dot product between the query vector and each input vector. This use of the dot product is a key idea in understanding attention because the operation quantifies alignment. In self-attention, higher dot product means higher similarity and a higher attention score between two tokens. Saying that two tokens attend more to each other means their vectors are more aligned, and saying they attend less means their vectors are less aligned. Visually, “journey” and “starts” attend more to each other because their vectors are aligned, while “one” and “journey” attend less because their vectors form a 90 degree angle.

With that intuition in place, the code mirrors the idea directly. The query is set to “inputs[1]” because Python uses zero-based indexing, and the second token, “journey”, is therefore at index 1. The code initializes an empty tensor for attention scores, loops over the input vectors, computes the dot product of each input vector with the query vector, and stores those dot products in the attention scores tensor. For the query “journey”, the largest attention scores are for “journey”, “starts”, and “step”, which matches the intuition that these vectors are closely aligned with “journey”. The word “one” has the lowest attention score for the query “journey” because its vector is almost at a 90 degree angle to the “journey” vector, so they are not aligned.

Listing 03.02.1: Selects the second input vector as the query, creates an empty tensor for attention scores, fills it with dot products between each input vector and the query, and prints the resulting scores. Source: rasbt/LLMs-from-scratch, `ch03/01_main-chapter-code/ch03.ipynb`, Apache-2.0, commit cdbd33e.

```
query = inputs[1]  # 2nd input token is the query

attn_scores_2 = torch.empty(inputs.shape[0])
for i, x_i in enumerate(inputs):
    attn_scores_2[i] = torch.dot(x_i, query) # dot product (transpose not necessary here
print(attn_scores_2)
```

Output (executed):

```
tensor([0.9544, 1.4950, 1.4754, 0.8434, 0.7070, 1.0865])
```

- *Checked by running the code:* `1` = 1 (the text says 1)

3.3 Normalization and softmax for attention weights

How do raw attention scores become quantities that are easy to read and safe to optimize? Normalization is needed for interpretability, because it lets us express attention in terms of percentages. To express attention in percentages, the values must sum to one, and unnormalized attention scores do not necessarily do that. A second reason for normalizing the attention weights is training stability: values between zero and one generally help during backpropagation and gradient descent. The simplest normalization is to sum the attention scores and divide each element by that total. That procedure makes the normalized values sum to one. In code, a separate tensor called attention weights is created by dividing the attention scores by their total sum, and the resulting values sum to one. Attention scores and attention weights mean the same thing intuitively, but attention weights differ in that their elements sum to one.

Simple normalization is useful as a first step, but machine learning practice usually prefers a different normalization function. In machine learning, it is more common and advisable to use the softmax function than plain summation-based normalization. If these values are normalized by dividing each one by their sum, the denominator is 406, so the largest value becomes 400 divided by 406 while the others remain small but nonzero fractions. When one value is extremely large, the desired behavior is for the small values to be close to zero and the extreme value to be close to one. Summation-based normalization does not achieve that behavior well enough. Values that are not exactly zero can confuse the optimizer during backpropagation and gradient descent, because some weight may still be assigned to values that should be neglected.

Softmax takes the exponent of each element and divides it by the sum of the exponents of all elements. Its outputs sum to one. Softmax also handles extreme cases better, because after exponentiation a very large value becomes dominant, making its normalized result very close to one and the smaller values very close to zero. That is why softmax is especially preferable for extreme values that may appear in large-scale models such as LLMs. In code, the same idea appears in a naive softmax implementation. A naive softmax implementation computes the exponent of each attention score and divides by the sum of those exponents. In that implementation, the sum uses dimension equal to zero because it sums all the entries in a row. These naive softmax attention weights differ from the values produced by simple summation-based normalization, but they still sum to one. Softmax attention weights are always positive because they are produced through the exponential operation. That positivity matters for interpretability, because attention can then be read as positive percentages assigned to tokens.

In practice, numerical stability is the key point here. PyTorch implements softmax by subtracting the maximum input value from each value before applying the exponential and normalization. Subtracting the maximum does not change the mathematical result, because the common exponential factor cancels between numerator and denominator. It is done to avoid numerical instability such as overflow when the inputs are very large or very small. So although the naive softmax formula is theoretically fine, in Python it is better to subtract the maximum value to prevent overflow problems. The code implements softmax both with PyTorch and with a manual implementation. The PyTorch version is computed with a single call to "torch.softmax" on the attention score tensor. That call uses the numerically stable form that subtracts the maximum value before exponentiation. A naive softmax implementation may suffer from numerical instability, including overflow for very large values and underflow for very small values, so in practice the PyTorch implementation is preferable. When no very large or very small values are present, the naive softmax and the PyTorch softmax produce the same attention weight tensor. PyTorch softmax is still preferred because later computations may involve very large parameters, so it is better to use numerically stable implementations.

Once the values are normalized, attention becomes readable. Attention weights make attention interpretable because they can be read as percentages assigned to words. For example, weights around 0.13, 0.23, 0.23, 0.12, 0.10, and 0.15 can be read as about 13% attention to "your", about 23% to "journey", about 23% to "starts", 12% to "with", 10% to "one", and 15% to "step". Because attention weights sum to one, it becomes easy to say that some words receive higher attention and others lower attention in percentage terms.

Listing 03.03.1: This code normalizes attention scores by dividing the tensor by its total sum and prints the resulting attention weights and their sum. Source: rasbt/LLMs-from-scratch, `ch03/01_main-chapter-code/ch03.ipynb`, Apache-2.0, commit cdbd33e.

```
attn_weights_2_tmp = attn_scores_2 / attn_scores_2.sum()
```

```
print("Attention weights:", attn_weights_2_tmp)
print("Sum:", attn_weights_2_tmp.sum())
```

Output (executed):

```
Attention weights: tensor([0.1455, 0.2278, 0.2249, 0.1285, 0.1077, 0.1656])
Sum: tensor(1.0000)
```

3.4 Computing context vectors

How do the normalized attention weights turn into the actual representation the model will use for a token? After computing the attention weights, the next and final step is to compute the context vector. Up to this point, the process is to take an input query, compute attention scores by dot products with the embedding vectors, and normalize those scores into attention weights that sum to one. With those weights in hand, the remaining task is straightforward. To compute a context vector, multiply each input embedding vector by its corresponding attention weight and then sum the resulting vectors. Multiplying each embedding by its attention weight can be understood as scaling down the vector in the vector representation. Each input embedding vector is scaled by its attention weight, so lower-importance vectors are scaled down more.

For the token "journey," the context vector is denoted as "z2." To compute the context vector for "journey," multiply every input token embedding by the corresponding attention weight and sum all the resulting vectors. The resulting summed vector is the context vector for the token being considered, here "journey." That vector has contributions from all the input vectors, so it is an enriched embedding vector. For example, a token with attention weight 0.2 contributes about 20% to the context vector, while a token with attention weight 0.1 contributes only about 10%. In modern large language models such as GPT, context vectors carry the same meaning: they are enriched embedding vectors. The attention mechanism is what makes it possible to obtain context vectors rather than being limited to the original embedding vectors.

The Python implementation follows the same idea directly. It uses the query for "journey," initializes a context vector tensor for token two, loops through all inputs, scales each input by the corresponding attention weight, and adds the scaled vectors together. Concretely, the first input embedding is multiplied by the first attention weight, the second input embedding by the second attention weight, and the same pattern continues through the last input vector. Once those scaled vectors are added together, the result is the final context vector for "journey."

Listing 03.04.1: This code uses the second input token as the query, initializes a zero context vector, then loops over all input embeddings and adds each embedding scaled by

its corresponding attention weight. Source: rasbt/LLMs-from-scratch, `ch03/01_main-chapter-code/ch03.ipynb`, Apache-2.0, commit cdbd33e.

```
query = inputs[1] # 2nd input token is the query

context_vec_2 = torch.zeros(query.shape)
for i,x_i in enumerate(inputs):
    context_vec_2 += attn_weights_2[i]*x_i

print(context_vec_2)
```

Output (executed):

```
tensor([0.4419, 0.6515, 0.5683])
```

3.5 Vectorizing self-attention over all tokens

How do the calculations for one query token become context vectors for the whole sequence? After computing the context vector for "journey," the task is still incomplete, because a similar context vector is needed for all the other tokens in the six-word input "your," "journey," "starts," "with," "one," and "step." The same computation used for "journey" extends to every other input: for each token, compute attention scores, then attention weights, and then the context vector. The matrix used to organize these results is called the attention weight matrix. In that matrix, each row represents the attention weights for one particular word. For the query "journey," its row contains the six attention weights between "journey" and each word in the sequence: "your," "journey," "starts," "with," "one," and "step." In this example, every query token needs six attention weights, one for each of the six input words. To handle all queries, repeat the same process for each token: if a token is the query, compute its attention weights with all other words and then form its context vector with the same summation operation used for "journey."

The first step is to compute attention scores for every query against every input. For a given query, an attention score is obtained by taking the dot product of that query with all the input vectors. One way to compute all attention scores is to loop through the inputs twice and compute dot products. With nested loops, one input is fixed as the query in the outer loop, the inner loop runs through all inputs to compute six dot products, and then the outer loop moves to the next query token. The result is a 6x6 attention score matrix in which each element is an attention score between a pair of input tokens. Each row represents the dot products between that query and all the other input vectors. The second row matches the attention scores computed earlier for the second query, because it represents the dot products between that second query and all

input vectors. Using two for-loops for these dot products is not computationally efficient, so matrix multiplication needs to be understood; linear algebra is a core foundation of machine learning concepts.

A more compact computation gives the same score matrix. If the input embeddings are arranged into an input matrix, multiplying that matrix by its transpose produces the same attention score matrix as the nested-loop dot products. This works because matrix multiplication computes dot products between the rows of the first matrix and the columns of the second matrix. That matrix multiplication is much more efficient than using two for-loops. Once the attention score matrix is available, normalize it by applying softmax to the attention scores matrix. Softmax is applied to each row, and the entries in each row of the resulting attention weight matrix sum to one. In functions like "torch.softmax," the "dim" parameter specifies the dimension of the input tensor along which the function is computed. Setting "dim = -1" applies the normalization along the last dimension of the attention score tensor. For a two-dimensional attention score tensor, the last dimension is the columns, so "dim = -1" normalizes across the columns. A simple check is to sum the entries of the second row and print the sums of all rows; if every row sums to one, the softmax has been applied correctly.

The final step can also be vectorized. Instead of looping over tokens to form one context vector at a time, the context vectors are computed in a matrix manner using an elegant matrix operation. The simplified operation is to multiply the attention weight matrix by the input matrix. That product gives the context vectors for all six tokens. With attention weights of shape 6x6 and inputs of shape 6x3, the multiplication is valid and produces a 6x3 context matrix. Each row of the final 6x3 matrix is the context vector for the corresponding token: the first row for the first token, the second row for the second token, and so on through the sixth row. A row of the output can be viewed as a weighted sum of the rows of the input matrix using the attention weights from the corresponding row. This is the same scaling-and-summing operation used earlier to compute a context vector manually. In code, an "all_context_vectors" tensor is created by multiplying the attention weight matrix with the input matrix. The second row of "all_context_vectors" matches the previously computed context vector for "journey," confirming that the matrix multiplication gives the correct result.

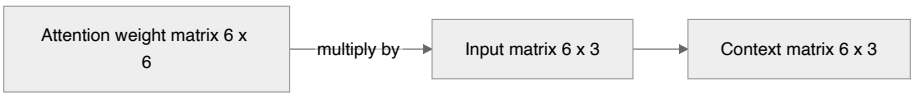


Figure 3.2: The full set of context vectors can be computed in one matrix multiplication by multiplying the attention weight matrix by the input matrix.

Listing 03.05.1: This code fills a 6×6 attention score matrix by looping over all input pairs and storing the dot product for each query-token and token pair. Source: rasbt/LLMs-from-scratch, `ch03/01_main-chapter-code/ch03.ipynb`, Apache-2.0, commit cdbd33e.

```
attn_scores = torch.empty(6, 6)

for i, x_i in enumerate(inputs):
    for j, x_j in enumerate(inputs):
        attn_scores[i, j] = torch.dot(x_i, x_j)

print(attn_scores)
```

Output (executed):

```
tensor([[0.9995, 0.9544, 0.9422, 0.4753, 0.4576, 0.6310],
        [0.9544, 1.4950, 1.4754, 0.8434, 0.7070, 1.0865],
        [0.9422, 1.4754, 1.4570, 0.8296, 0.7154, 1.0605],
        [0.4753, 0.8434, 0.8296, 0.4937, 0.3474, 0.6565],
        [0.4576, 0.7070, 0.7154, 0.3474, 0.6654, 0.2935],
        [0.6310, 1.0865, 1.0605, 0.6565, 0.2935, 0.9450]])
```

Listing 03.05.2: This code computes all context vectors at once by multiplying the attention weight matrix with the input matrix and printing the result. Source: rasbt/LLMs-from-scratch, `ch03/01_main-chapter-code/ch03.ipynb`, Apache-2.0, commit cdbd33e.

```
all_context_vecs = attn_weights @ inputs
print(all_context_vecs)
```

Output (executed):

```
tensor([[0.4421, 0.5931, 0.5790],
        [0.4419, 0.6515, 0.5683],
        [0.4431, 0.6496, 0.5671],
        [0.4304, 0.6298, 0.5510],
        [0.4671, 0.5910, 0.5266],
        [0.4177, 0.6503, 0.5645]])
```

- *Checked by running the code:* `all_context_vecs.shape[1]` $= 3$ (the text says 3)
- *Checked by running the code:* `all_context_vecs.shape[0]` $= 6$ (the text says 6)

3.6 Why trainable Q, K, V are needed

Why are trainable query, key, and value weights needed if simple self-attention already produces context vectors? In the simplified mechanism here, trainable weights are not used, although trainable weights are added later in LLMs because they help the

model understand context text much better and learn in a better way. That simplified mechanism still matters because the same underlying intuition and mechanism carry over to key, query, and value attention. The main change is in the final multiplication: instead of multiplying the attention weight matrix by the input matrix, this formulation multiplies by the values. Once this simpler version is clear, key, query, and value are easier to understand because the intuition remains the same.

The limitation of the current approach is that attention weights come only from dot products. In this setup, attention is driven by semantic alignment between vectors. But semantic similarity alone is not enough, because two vectors may fail to align semantically and still be important in the current sentence context. To capture what is happening in the current sentence, the model needs information beyond meaning alone, and without trainable weights it will not capture that context effectively. The model captures the context somewhat, but it gives attention only to words that are similar in meaning to the query.

A simple example makes the problem concrete. Take the sentence "the cat sat on the mat because it is warm" and use "warm" as the query. If trainable weights are not used, the model takes the dot product between the query "warm" and each word embedding. In that case, "warm" is most similar to itself, may be somewhat related to "mat", and words such as "the", "cat", and "sat" can receive low similarity scores because they are not semantically related to "warm". So without trainable weights, attention mainly follows semantic similarity to the query. Yet in this sentence, "mat" can be the most important word for the query "warm" because the mat is warm, even though "mat" and "warm" are not closely related in meaning. Trainable weights let the model learn that "warm" should pay more attention to "mat" even if "mat" is not semantically similar to "warm" in the embedding space. That is why trainable weights are needed to capture contextual relationships and long-range dependencies that would be lost if attention relied only on semantic similarity.

Then, the inputs are first transformed into queries, keys, and values. The input matrix is multiplied by the query, key, and value weight matrices to produce the queries matrix, keys matrix, and values matrix. These query, key, and value weight matrices are trainable parameters learned from input data, although the focus here is on the forward pass rather than training or backpropagation. After that, the attention scores are computed by multiplying the queries matrix by the transpose of the keys matrix. Each row of that attention score matrix shows how much attention one particular query pays to all the keys in the sentence. The scores are converted into attention weights by dividing by the square root of the key dimension and then applying softmax. Attention weights correspond to the attention scores, but each row of the attention weight matrix sums to one, giving the rows a probabilistic meaning. The final context vectors are obtained by multiplying the attention weight matrix by the values matrix. Each row of the resulting context vector matrix is the context vector for the corresponding token. A token's context vector is richer than its original embedding because it also contains information about how much attention should be paid to the other words. These context vectors are the inputs used

for language model training, and the aim of attention is to produce one context vector for each input embedding vector so that next-word prediction becomes more reliable.

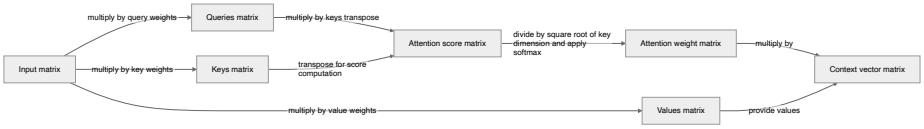


Figure 3.3: Trainable query, key, and value projections produce attention scores, convert them into attention weights, and combine values into context vectors.

Listing 03.06.1: This 'SelfAttention_v1' module defines trainable query, key, and value weight matrices, projects the input into queries, keys, and values, computes scaled softmax attention weights from 'queries @ keys.T', and returns context vectors as 'attn_weights @ values'. Source: rasbt/LLMs-from-scratch, `ch03/01_main-chapter-code/ch03.ipynb`, Apache-2.0, commit cdbd33e.

```
import torch.nn as nn

class SelfAttention_v1(nn.Module):

    def __init__(self, d_in, d_out):
        super().__init__()
        self.W_query = nn.Parameter(torch.rand(d_in, d_out))
        self.W_key   = nn.Parameter(torch.rand(d_in, d_out))
        self.W_value = nn.Parameter(torch.rand(d_in, d_out))

    def forward(self, x):
        keys = x @ self.W_key
        queries = x @ self.W_query
        values = x @ self.W_value

        attn_scores = queries @ keys.T # omega
        attn_weights = torch.softmax(
            attn_scores / keys.shape[-1]**0.5, dim=-1
        )

        context_vec = attn_weights @ values
        return context_vec

torch.manual_seed(123)
sa_v1 = SelfAttention_v1(d_in, d_out)
print(sa_v1(inputs))
```

Output (executed):

```
tensor([[0.2996, 0.8053],
        [0.3061, 0.8210],
        [0.3058, 0.8203],
        [0.2948, 0.7939],
        [0.2927, 0.7891],
        [0.2990, 0.8040]], grad_fn=<MmBackward0>)
```

3.7 Causal attention and masking

How can a model use self-attention without letting future tokens influence the current token? Causal attention, also called masked attention, is a special form of self-attention. It restricts the model to consider only the previous and current inputs in a sequence when processing a token, in contrast to ordinary self-attention, which allows access to the entire input sequence at once. When computing attention for a query token, the model only factors in tokens that occur at or before that token in the sequence. In GPT-like language models, this is done by masking out the future tokens that come after the current token. The purpose of causal attention is to prevent future tokens from influencing the current token. The context size matters here because it specifies how many words the LLM can look at before predicting the next word. In the running example, the context length is six because there are six tokens, which matches the six rows of the attention score or attention weight matrix.

A simple way to implement this starts from the usual self-attention pipeline. The self-attention class initializes trainable query, key, and value weight matrices randomly, multiplies the inputs by them to get queries, keys, and values, computes attention scores as queries times keys transpose, scales by the square root of the key dimension, applies softmax to get attention weights, and multiplies the attention weights by the values to get the context vector matrix. To begin causal attention from that code, first compute the queries and keys, then the attention scores as queries multiplied by keys transpose, divide by the square root of the key dimension, and apply softmax to get attention weights before masking. At that point, the causal part begins by generating a mask whose entries above the diagonal are zero. Causal attention masks out the attention weights above the diagonal of the attention matrix, and the region above the diagonal is the causal attention mask. One strategy is to compute the usual attention scores and attention weights, set the elements above the diagonal to zero, and then renormalize the remaining attention weights.

The mask is built from a triangular matrix. An upper triangular matrix has all elements below the diagonal equal to zero, and a lower triangular matrix has all elements above the diagonal equal to zero. “torch.triu” produces an upper triangular matrix, while “torch.tril” produces a lower triangular matrix with all elements above the diagonal set to zero. To

construct the causal mask with zeros above the diagonal, use a lower triangular matrix, so "torch.tril" is the right choice. The code builds this mask by applying "torch.tril" to a square matrix of ones with shape context length by context length, which sets all entries above the diagonal to zero. Multiplying the attention weights by this mask sets all elements above the diagonal in the attention weights to zero and produces a masked attention weight matrix. After masking, the result is not yet a valid attention weight matrix because each row no longer sums to one. To renormalize, compute the sum of each row and divide every element in that row by that row sum. After this renormalization, each row of the masked attention weight matrix sums to one. Then multiply the attention weights by the values matrix to obtain the context vector matrix.

That first method is easy to understand, but it has an important flaw. If masking is done after softmax, future tokens have already influenced the earlier attention values because the softmax denominator includes all scores. This creates a data leakage problem even if the future-token weights are later set to zero. A better approach is to apply an upper triangular infinity mask to the attention scores before softmax and then apply softmax once. In this method, the elements above the diagonal in the attention score matrix are replaced with negative infinity before softmax. The code constructs this mask as a matrix with ones above the diagonal and uses those ones as the positions to replace with negative infinity. The mask fill operation finds the positions where the mask is true or positive and replaces the corresponding attention-score entries with negative infinity. After masking, the masked matrix is divided by the square root of the key dimension and then softmax is applied. Setting future-score entries to negative infinity makes their exponentials zero, so softmax turns those positions into zeros and automatically produces rows that sum to one. Applying the negative-infinity mask before softmax avoids leakage because future-token influence is removed before normalization. Masking in transformers sets scores for future tokens to a very large negative value so their influence in softmax is effectively zero, and softmax then recalculates attention weights among the unmasked tokens. This ensures no information leakage from masked tokens. Once the attention weight matrix is obtained, it is multiplied by the value matrix to get the context vector.

Listing 03.07.1: This code builds an upper-triangular mask and uses masked_fill to replace the attention-score entries above the diagonal with negative infinity. Source: rasbt/LLMs-from-scratch, `ch03/01_main-chapter-code/ch03.ipynb`, Apache-2.0, commit cdbd33e.

```
mask = torch.triu(torch.ones(context_length, context_length), diagonal=1)
masked = attn_scores.masked_fill(mask.bool(), -torch.inf)
print(masked)
```

Output (executed):

```
tensor([[0.2899,   -inf,   -inf,   -inf,   -inf,   -inf],
        [0.4656, 0.1723,   -inf,   -inf,   -inf,   -inf],
```

```
        [0.4594, 0.1703, 0.1731,   -inf,   -inf,   -inf],
        [0.2642, 0.1024, 0.1036, 0.0186,   -inf,   -inf],
        [0.2183, 0.0874, 0.0882, 0.0177, 0.0786,   -inf],
        [0.3408, 0.1270, 0.1290, 0.0198, 0.1290, 0.0078]],
       grad_fn=<MaskedFillBackward0>)
```

3.8 Dropout, batching, and implementation details for causal attention

How does causal attention become robust enough for training and flexible enough to process more than one sequence at a time? Dropout is a deep learning technique in which a neural network has neurons in different layers randomly set to zero. A simple way to think about dropout is that it prevents a lazy-neuron or codependency problem: if one neuron relies on others to do most of the work, switching those others off forces it to participate. By randomly turning neurons off, dropout helps all neurons participate, prevents overfitting, and improves generalization performance, including on test data. In transformer architectures, including GPT, dropout in the attention mechanism is usually applied either after calculating attention scores or after applying attention weights to the value vectors. A more common choice is to apply dropout after calculating the attention weights or attention scores.

Dropout is applied by first creating a dropout mask that specifies which neurons are randomly turned off. With dropout probability 0.5, on average 50% of the attention weights in each row are turned off. This operates row by row by randomly zeroing out some elements with the chosen probability. To illustrate this in Python, PyTorch dropout with rate 0.5 is first applied to a 6 by 6 tensor of ones before it is applied to the attention weight matrix. When dropout with rate 0.5 is applied, the weights that are not switched off are rescaled by a factor of two. Because dropout is probabilistic, an exact number of weights need not be switched off in every row, but over many rows the average matches the dropout rate. Applied to an attention weight matrix at rate 50%, dropout randomly sets about half of the elements to zero and scales the remaining elements up by a factor of two. In PyTorch dropout, outputs are scaled by a factor of 1 divided by 1 minus p. That scaling maintains the overall balance of the attention weights so that the average influence of the attention mechanism remains consistent during training and inference.

The same dropout layer can then be applied directly to the final attention weight matrix by creating a dropout class instance with input argument 0.5 and applying it to that matrix. After dropout is applied, some attention weights are randomly set to zero and the remaining weights are scaled up. The causal attention class follows the earlier self-attention structure but adds causal masking and dropout. Its computation is to multiply the inputs by the query, key, and value weight matrices to obtain queries, keys, and values; multiply queries by keys transpose to get attention scores; mask out elements

above the diagonal; scale by the square root of the dimension; apply dropout; apply softmax to obtain attention weights; and multiply those attention weights by the values to get the context vector matrix.

The class also needs to handle multiple sentences at once as batches. A simple batch is formed with two inputs so that the model receives a tensor containing two sequences, each of shape 6 by 3. This batched input is a 3D tensor of two input texts with six tokens each, where each token has a three-dimensional embedding vector. With batching, the input shape becomes batch size by number of tokens by embedding dimension, unlike the earlier self-attention version without batches. The batched computation still multiplies the input by the key, query, and value weight matrices to obtain keys, queries, and values, and then multiplies queries by the transpose of keys. In the transpose operation with dimensions 1 and 2, the focus is on the token dimensions inside each batch. Using "keys.transpose(1, 2)" makes the matrix multiplication possible within each batch. This transpose preserves the batch rows while transposing the matrix inside each batch. When queries are multiplied by the transposed keys, the batches are processed sequentially and the resulting attention score matrices are stacked together.

Once batching is in place, the masking and normalization steps are the same idea as before. Here, the upper triangular mask has ones above the diagonal. In causal attention, positions marked by ones in that upper triangular mask are replaced with negative infinity in the attention scores matrix. After masking, the scores are divided by the square root of the key dimension and softmax is applied to obtain the attention weight matrix. After softmax, the rows of the attention weight matrix sum to one with the causal mask applied. Then dropout is applied to the attention weights, the context vector is computed as attention weights multiplied by values, and that context vector is returned.

> **Note.** The course's reference code differs here: The reference uses a lower-triangular mask via 'torch.tril'—1s on/below the diagonal and 0s above—to achieve the same causal masking effect.

The causal attention class is instantiated with input dimension D_in = 3, output dimension D_out = 2, context length 6, and dropout rate 0.0. Passing the stacked batch into this class produces a 3D output tensor of shape $2 \times 6 \times 2$, because each of the two inputs yields a 6×2 context vector matrix. Printing the context vectors shows that the first matrix corresponds to the first input and the second matrix corresponds to the second input, stacked on top of each other. That confirms that the implemented causal attention class can handle batches.

A few implementation details make this class easier to use correctly. A buffer is useful for fixed tensors such as the causal mask that do not need to be trained. Using "register_buffer" for fixed matrices like the mask causes them to move automatically with the model to the appropriate device, such as CPU or GPU. Defining fixed, untrained tensors with "register_buffer" is more convenient because it avoids manually ensuring that they are on the same device as the model parameters and helps prevent device mismatch errors. Slicing the mask by the number of tokens ensures that if a batch has fewer

tokens than the supported context size, the mask is created only up to the actual number of tokens. Taken together, this class applies a causal attention mask so that elements above the diagonal are set to zero, uses softmax so each row of attention weights sums to one, and includes dropout for generalizability and to prevent overfitting. Stacking multiple causal attention heads together leads to multi-head attention, which is what GPT implements. To understand multi-head attention, it is necessary first to understand causal attention, and causal attention in turn depends on understanding keys, queries, and values. Understanding dimensions, batches, three-dimensional tensors, matrices, and linear algebra fundamentals is therefore important for understanding what is going on here.

3.9 Efficient multi-head attention with weight splits

How can multi-head attention be implemented without paying for a separate set of initial matrix multiplications for every head? A straightforward implementation creates multiple weight matrices for the query, key, and value, one set per attention head, multiplies the input with those matrices, and obtains separate queries, keys, and values for each head. In that earlier approach, each attention head produced its own context vector, and the context vectors from different heads were concatenated into a larger matrix. The difficulty is efficiency: the number of initial multiplications grows with the number of heads. With 96 attention heads, this means 96 multiplications to get the queries, 96 to get the keys, and 96 to get the values. A more efficient method performs one multiplication for queries, one for keys, and one for values, then splits the resulting query, key, and value tensors into multiple heads. After that split, the rest of the attention procedure stays the same: multiply queries by transposed keys to get attention scores, turn those into attention weights, and multiply the weights by the values to get context vectors. This implementation is called multi-head attention with weight splits. It is described as more computationally efficient than the earlier multi-head attention implementation.

> **Note.** The course's reference code differs here: Earlier, this uses the intuitive per-head implementation; the stand-alone MultiHeadAttention later uses one query, key, and value matrix and then splits into heads.

Turning to the implementation, earlier code used separate classes for a multi-head attention wrapper and for causal attention, while the newer implementation combines both into a single multi-head attention class. In the old picture, the input "x" was multiplied by separate matrices such as "wq1" and "wq2" to obtain separate query matrices such as "q1" and "q2". In the weight-splitting version, the input is multiplied by one larger query matrix "WQ", and the resulting queries are then split into components for different heads. The larger query, key, and value matrices are chosen so that their output dimension already includes the number of heads. The relation is "D_out" equals head dimension multiplied by number of heads. In the illustrated small example, each head has dimension 2 and there are 2 heads, so the larger query matrix has "D_out"

equal to 4. In the combined class, multiple heads are implemented by reshaping the query, key, and value tensors to split them into heads.

To follow the implementation, start with the input to the forward method. The forward method takes an input tensor "X" and ultimately computes a context vector for every input token embedding. This input tensor has three dimensions: batch size, number of tokens, and input dimension. The input dimension "D_in" is the dimension of the embedding vector that represents each token. For the example input shape "1 × 3 × 6", the batch size is 1, there are 3 tokens, and each token is represented by a 6-dimensional vector. Two example tokens are "cat" and "sleeps." The output is one context vector per token, so its shape is number of tokens by "D_out". In the example, "D_out" is chosen to be 6, the same as "D_in", which is described as typical in GPT-based models. The number of attention heads in the example is 2. The head dimension is "D_out" divided by the number of heads, so here it is "6 / 2 = 3". Since each head has dimension 3 and there are 2 heads, the total "D_out" is 6.

> **Note.** The course's reference code differs here: Here, concatenating 2 heads of size 3 gives 6; elsewhere, a different example sets d_out=2, and some MultiHeadAttention implementations keep the final output dimension fixed.

The next step is to create the trainable projections that produce keys, queries, and values. The model initializes trainable weight matrices "W_k", "W_q", and "W_v". Their first dimension must equal "D_in" so that they are compatible with the input in matrix multiplication. Therefore the key, query, and value matrices have shape "D_in × D_out". In the worked example, "D_in = 6" and "D_out = 6", so "W_q", "W_k", and "W_v" are "6 × 6" matrices. In code, these trainable matrices are initialized in the constructor as neural network linear layers with dimensions "D_in" by "D_out". A linear layer is used because it is optimized for initializing weights and is better for backpropagation later. After initialization, the input is multiplied by these three weight matrices to produce keys, queries, and values. With input shape "1 × 3 × 6" and weight matrices of shape "6 × 6", the resulting keys, queries, and values each have shape "1 × 3 × 6". In that shape, 1 is the batch size, 3 is the number of tokens, and D_out is the output dimension. Each row corresponds to one token, and each token is represented by a 6-dimensional vector. In code, passing the input through the linear layers applies the trainable query, key, and value matrices and returns tensors of shape batch size by number of tokens by "D_out".

At this point, the tensors still do not explicitly represent individual heads. A fourth dimension is introduced because the existing three dimensions represent batch size, number of tokens, and "D_out", but not the number of heads or the head dimension. Because head dimension equals "D_out" divided by number of heads, "D_out" also equals head dimension multiplied by number of heads. The last dimension of the keys, queries, and values is reshaped so that "D_out" is split into number of heads and head dimension. With "D_out = 6" and 2 heads, the head dimension is 3, so each "1 × 3 × 6" tensor is reshaped to "1 × 3 × 2 × 3". Ignoring the batch dimension, a tensor of

shape "1 × 3 × 2 × 3" can be read as 3 tokens, with 2 heads inside each token, and a 3-dimensional vector for each head. The keys and values are reshaped in the same way, so they also have shape "1 × 3 × 2 × 3". In code, this split is done with view on the keys, replacing the last dimension by number of heads and head dimension and turning three-dimensional tensors into four-dimensional tensors.

After reshaping, one more reordering is needed before attention scores can be computed cleanly for all heads. Before transposition, tensors with shape "1 × 3 × 2 × 3" are grouped by tokens rather than by heads. To compute attention scores, they need to be grouped by heads instead. That regrouping swaps the number-of-tokens and number-of-heads dimensions, changing the shape from batch, tokens, heads, head dimension to batch, heads, tokens, head dimension, which here is "1 × 2 × 3 × 3". In code, this is done with "transpose(1, 2)", because Python uses zero-based indexing and dimensions 1 and 2 correspond to tokens and heads. After transposition to "1 × 2 × 3 × 3", the tensors are grouped by head, and within each head the token rows each carry a 3-dimensional head vector. Grouping by heads makes it possible to compute a separate attention score matrix for each head before the results are concatenated. The transpose is needed because grouping by tokens is not suitable for computing attention scores for each head in parallel.

Once the queries, keys, and values have the right shape, the attention computation itself follows the familiar pattern. Attention scores are computed by multiplying the queries by the keys transposed along the last two dimensions. Transposing the key tensor turns a row in the key matrix into a column so that the multiplication is compatible. With tensors grouped by head, the multiplication automatically pairs head 1 of the queries with head 1 of the keys and head 2 with head 2. After the key transpose, the query and key tensors have compatible shapes, and each head is multiplied only with its corresponding head. This multiplication produces the attention score matrix. The attention score tensor has dimensions batch size, number of heads, number of tokens, and number of tokens. The last two dimensions are token-by-token because a number-of-tokens-by-head-dimension matrix multiplied by a head-dimension-by-number-of-tokens matrix yields a number-of-tokens-by-number-of-tokens matrix, while the first two dimensions stay the same. Within each head, a row of the attention score matrix gives the attention scores between one query token and all tokens in the sequence. An attention score exists between each token and every other token, so each head's attention matrix has a row and a column for every token. In code, the attention score is the scaled product between queries and keys. Each head therefore has its own token-by-token attention score matrix.

To convert scores into attention weights, causal attention first restricts which positions can contribute. In causal attention, a token attends only to earlier tokens, not to later tokens. To enforce that rule, all elements above the diagonal are replaced with negative infinity. Those masked positions become zero after softmax. Before softmax, the scores are divided by the square root of the head dimension. This scaling prevents the query-key dot-product variance from growing with dimensionality, keeps pre-softmax values smaller, and is useful for backpropagation and stable gradients. In this example, the

head dimension is 3 because "D_out" is 6 and the number of heads is 2, so the scores are divided by the square root of 3. After softmax, each row sums to one. The attention weights have the same dimensions as the attention scores: batch size, number of heads, number of tokens, and number of tokens. So the full transformation from scores to weights is: apply the causal mask with negative infinity above the diagonal, divide by the square root of head dimension, and then apply softmax. Dropout can also be applied after obtaining attention weights, although it is omitted here for simplicity. In code, an upper triangular mask is created and applied to the attention scores so that all elements above the diagonal become negative infinity. The mask is built only for the context length, because the model does not look at more tokens than the context length at a time. If a batch contains fewer tokens than the context size, the masking code stops the mask at the actual number of tokens. In code, the scores are divided by the square root of the last dimension of the keys tensor, which is the head dimension, before softmax is applied. Softmax is applied along the last dimension so that the values in each row sum to one. Each element of the attention weight matrix is the attention weight between a particular query token and a particular key token within a head.

> **Note.** The course's reference code differs here: This example omits dropout for simplicity; the reference implementation applies dropout after attention weights, commonly with a 50% rate during training.

Finally, the context vector matrix is computed by multiplying the attention weights by the value matrix. The value matrix was computed earlier and is used only in this final step. For this multiplication, the attention weights have dimensions batch size, number of heads, number of tokens, number of tokens, and the values have dimensions batch size, number of heads, number of tokens, head dimension. The multiplication is valid because the number of columns in the attention weights matches the number of rows in the values. Attention weights differ from attention scores because each row of the weights sums to one, while that is not true for the scores. Multiplying the attention weights by the values produces a context vector output with shape batch size, number of heads, number of tokens, head dimension. In that output, each head contains one row per token, and each row is the context vector for that token with dimensionality equal to the head dimension.

The remaining job is to merge the separate head outputs back into the full output dimension. The final context matrix is supposed to preserve the "D_out" dimension, so the number of heads and the head dimension must be merged back together. To prepare for that merge, the number-of-heads and number-of-tokens dimensions are transposed so the shape changes from batch size, number of heads, number of tokens, head dimension to batch size, number of tokens, number of heads, head dimension. After this transpose, the grouping is by token, and within each token there is one context vector per head. The reason for this transpose is to bring number of heads and head dimension next to each other so they can be merged into "D_out" more easily. The outputs from multiple heads are then combined by flattening each token output into a single row. In the final context matrix, each row is the context vector for one token, and its length is

"D_out" because the head outputs have been merged together. The final context vector matrix therefore has shape batch size, number of tokens, "D_out". In code, the context vector is reshaped into batch size, number of tokens, "D_out". Calling "contiguous" before reshaping ensures the tensor is in the same memory block so the reshape works properly. An optional output projection can then be applied as a learnable linear layer, although it is not strictly necessary. This full process produces enriched context vector representations for the tokens.

> **Note.** The course's reference code differs here: Here, heads are merged to recover a D_out-sized output; in the reference, one wrapper returns d_out*num_heads, while MultiHeadAttention can keep output dimension at d_out.

A small run of the implementation makes the shapes concrete. In the demonstration, there are 3 tokens, embedding dimension 6, "D_out" equal to 6, context length equal to 6, dropout rate 0, and 2 heads. Dropout in this attention implementation randomly blocks out some attention weights, which is described as useful for generalization. For the first batch, the resulting context vector matrix has 3 rows and 6 columns, matching the earlier example in shape even if the values differ because of different random initializations. For larger text, the same pipeline applies: take sentences, break them into tokens, convert them into token IDs, map them to input embeddings, and compute context vectors for each token. The multi-head attention class can also handle multiple batches at once, producing one context vector matrix per batch.

3.10 Layer normalization in transformers

How do transformer blocks keep training stable when so many layers and parameters are stacked together? A transformer block contains layer normalization, masked multi-head attention, dropout, shortcut connections, another layer normalization, a feed-forward neural network, another dropout layer, and another shortcut connection. Layer normalization appears at multiple places inside the transformer block and also outside the transformer block in the GPT architecture. In the architecture here, normalization appears before the attention head, after the attention head, and once again after coming out of the transformer block, before output processing. That repeated use is why it makes sense to define layer normalization as a separate class when coding GPT. In GPT and modern transformer architectures, layer normalization is applied before multi-head attention, before the neural network module inside the transformer block, and once more outside the transformer block before the final output layer. Overall, layer normalization appears three times in the architecture being discussed: before the multi-head attention, after the multi-head attention, and once before the output.

Why is normalization worth this much attention? Layer normalization is important because it improves the stability and efficiency of neural network training. Training

deep neural networks with many layers can be challenging because they can suffer from vanishing gradients or exploding gradients, which leads to unstable training dynamics. If a layer's output is too large or too small, gradient magnitudes can also become too large or too small. If a gradient becomes too large during backpropagation, multiplying gradients together can make the gradient at early layers explode. If a gradient becomes very small during backpropagation, the gradients at earlier layers also become very small, the parameters are not updated, and learning stagnates. Both very small gradients and very large gradients lead to unstable training dynamics.

There is a second problem normalization is meant to reduce. Internal covariate shift is the problem that, as training proceeds, the input distribution seen by a layer changes from one iteration to another. Changing input distributions at each layer make training difficult, make weight updates harder, and delay convergence to an optimal solution. The two main reasons for using layer normalization are to stabilize training by preventing vanishing or exploding gradients and to reduce internal covariate shift so convergence is faster. The main idea of layer normalization is to take the outputs of a specific layer and adjust them so that they have mean zero and variance one. Layer normalization centers a layer's activations to mean zero and normalizes their variance to one, which helps stabilize training and reduce internal covariate shift.

A small numeric example makes the normalization step concrete. For four layer outputs "X1", "X2", "X3", and "X4", the mean is computed as ("X1" + "X2" + "X3" + "X4") / 4, which in the example equals 2.15. The variance is computed as one fourth of the sum of the squared differences between each output and the mean. To normalize the outputs, subtract the mean from each output and divide by the square root of the variance, which is the standard deviation. After this normalization, the normalized values have mean zero. After this normalization, the normalized values have variance one. The same procedure can be summarized simply as subtracting the mean from each layer output and dividing by the square root of the variance. After layer normalization, the normalized outputs have mean zero and variance one.

The code example starts with a simple network before moving to the GPT module. To illustrate layer normalization in code, the implementation is first applied to a simple neural network and then turned into a separate layer normalization class for integration into the GPT architecture. In the example, the input tensor has shape 2 by 5 because there are two batches and each batch has five inputs. The example network layer takes five inputs and produces six outputs, so the output tensor has shape 2 by 6, with one row of six outputs for each batch. This example neural network consists of a linear layer followed by an activation layer. The ReLU activation function outputs x when x is positive and zero when x is negative. In layer normalization, each output in an input is replaced by that output minus the mean, divided by the square root of the variance.

> **Note.** The course's reference code differs here: Here the lecture uses a toy batch of token IDs with shape 2×5; the reference code later shows embedded inputs with shape (2, 6, 3).

In code, the mean is computed with dimension equal to minus one so that the mean is taken across the columns for each batch. Using keepdim equal to true preserves the mean as a 2 by 1 matrix instead of reducing it to a vector, which keeps dimensions aligned for later calculations. The variance is also computed across the columns for both batches with keepdim equal to true. If keepdim equal to true is not used, subtracting the mean from each individual element later can cause problems. The main normalization step in code is to subtract the mean tensor from the output tensor and divide by the square root of the variance. After normalization in the example, the mean of each batch is approximately zero and the variance of each batch is one. Very small nonzero mean values after normalization are due to numerical errors in Python and are very close to zero.

The layer normalization class takes the output of a layer as input and applies normalization to it. In the GPT setting, layer normalization is applied by taking the mean and variance across the embedding dimension for each row, where each row corresponds to a token embedding. For GPT-2 in this example, the embedding size is 768. The embedding dimension must be passed when creating the layer normalization class because the trainable scale and shift parameters have sizes determined by the embedding dimension. Normalization does not change dimensions, so the output of layer normalization has the same dimensions as its input. In the forward method, the model takes the mean across columns, takes the variance across columns, subtracts the mean, and divides by the square root of the variance. Epsilon is a small constant added to the variance to prevent division by zero during normalization. After normalization, the output is multiplied by the scale and by the shift parameters. The scale and shift are trainable parameters with the same dimension as the input, and the model adjusts them during training if that improves performance on the training task. These scale and shift parameters act like adjustable knobs that fine-tune the normalization. Because the scale and shift vectors have the embedding dimension, they can be applied to the normalized output by element-wise multiplication and element-wise addition. The epsilon parameter is set to a small value such as 10^-5. The main step in layer normalization is to take each row of the input, subtract its mean, and divide by the square root of its variance. After layer normalization, the mean of each resulting row is zero and the standard deviation is one.

One implementation detail concerns how variance is computed. Setting unbiased to true in the variance calculation applies Bessel's correction, which divides by n minus 1 instead of n. Using n minus 1 instead of n in the sample variance formula gives an unbiased variance estimate. In large language models, using n instead of n minus 1 makes little practical difference because the embedding dimension is large. For this implementation, setting unbiased to false works well and is chosen for compatibility with GPT-2 normalization layers and TensorFlow's default behavior used in the original GPT-2 model.

The class can then be tested directly on the batch example. An instance of the layer normalization class is created by passing in the embedding dimension, which is set to

five for the batch example. The batch example is passed through the layer normalization layer, which computes the mean and variance across the dimensions, subtracts the mean, and divides by the square root of the variance. In the printed batch example, layer normalization is applied separately to each batch item by subtracting that item's mean from its elements and dividing its elements by the square root of that item's variance. The output of the layer normalization code has mean zero and variance one for both batch items, showing that the code works as expected.

It is also important not to confuse layer normalization with batch normalization. Layer normalization and batch normalization are very different from each other. In layer normalization, normalization is usually done along the feature dimension, which is the columns, and it does not depend on the batch size. In layer normalization, each layer's output is normalized using its mean and standard deviation regardless of batch size. In batch normalization, normalization is performed for an entire batch, so it depends on the batch size. Batch normalization is less flexible because batch size is often determined by available hardware. Layer normalization is more flexible and provides more flexibility and stability for distributed training. When resources or hardware capabilities force small batch sizes, or when batch-size-independent normalization is desired, layer normalization offers much greater flexibility.

Listing 03.10.1: This code defines a LayerNorm module whose forward pass normalizes each input row using its mean and variance and then applies learnable scale and shift parameters. Source: rasbt/LLMs-from-scratch, `ch04/01_main-chapter-code/ch04.ipynb`, Apache-2.0, commit cdbd33e.

```
class LayerNorm(nn.Module):
    def __init__(self, emb_dim):
        super().__init__()
        self.eps = 1e-5
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim))

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        norm_x = (x - mean) / torch.sqrt(var + self.eps)
        return self.scale * norm_x + self.shift
```

3.11 Shortcut connections and gradient flow

Why do transformer blocks include those extra arrows and plus signs, and why do they matter for training stability? Shortcut connections are an important building block of

the GPT architecture alongside layer normalization, the GELU activation, and the feed-forward neural network. In a Transformer block, the plus symbols and their associated arrows represent shortcut connections. Shortcut connections are also known as skip connections or residual connections. They were first proposed in computer vision to solve the vanishing gradient problem.

The vanishing gradient problem is that gradients become progressively smaller as they are propagated backward through a neural network. During backpropagation, gradient flow goes from the output of the network to the input, so later-layer gradients are used to compute earlier-layer gradients. Because backpropagation involves multiplications, if a later gradient becomes small then earlier gradients become even smaller, which is why gradients can vanish. When gradients become very small, weight updates are not made, learning becomes stagnant, convergence is delayed, and the language model does not learn well. The weight update rule is that the new weight equals the old weight minus the step size times the partial derivative of the loss with respect to the weight. If the gradient of the loss with respect to a weight approaches zero, the weight is not updated, so the neural network is not learning. This leads to stagnation, and shortcut connections solve that problem.

A shortcut connection creates an alternative path for the gradient to flow by skipping one or more layers. It is implemented by adding the output of one layer to the output of a later layer. If the earlier output is y_l and the later layer computes f(y_l), then with a shortcut connection the later output becomes f(y_l) + y_l. To prevent vanishing gradients, the partial derivative of the loss with respect to the first-layer output should stay as large as possible so that the gradients with respect to the first-layer weights do not go to zero. Using the chain rule, the partial derivative of the loss with respect to y_l equals the partial derivative of the loss with respect to y_(l+1) multiplied by the partial derivative of y_(l+1) with respect to y_l. Because y_(l+1) = f(y_l) + y_l, the derivative of y_(l+1) with respect to y_l includes the derivative of f(y_l) plus 1. That plus-one term keeps the gradient from vanishing even if the derivative through the main path becomes small. Because the shortcut connection keeps gradients flowing, the loss gradients with respect to earlier-layer weights do not ultimately vanish, so we avoid stagnation in the weight updates. Shortcut connections therefore play a crucial role during backpropagation by preserving gradient flow through the network. They also smooth the loss-function landscape and make gradient flow smoother.

A small code example makes this effect concrete by stacking multiple layers, optionally adding shortcut connections, and then comparing gradient-flow magnitudes with and without shortcuts. The argument “layer_sizes” specifies how many neurons are in each layer. If “layer_sizes” is “3, 3, 3, 3, 1”, the network has five layers with three neurons and one layer with one neuron. The argument “use_shortcut” determines whether shortcut connections are used: “false” disables them and “true” enables them. PyTorch’s “nn.Sequential” module connects different neural network layers together. The network is constructed by chaining layers so that one entry of “layer_sizes” gives the input dimension, the next entry gives the output dimension, and each layer uses the same

activation function. For a layer, the input dimension is determined by the previous layer's output size and the output dimension is determined by the next layer's size. The code uses an activation function for all of these layers.

In the forward method, first consider the case without shortcut connections. Without shortcut connections, the output of each layer is fed to the next layer until the final output is produced. With shortcut connections, each step adds the current layer output to the running value from previous steps, so the final result accumulates all earlier shortcut additions. The shortcut connection itself is implemented by a single line that updates "x" to "x" plus the layer output. The implemented deep neural network has five layers, each consisting of a linear layer and an activation, and the forward pass optionally adds shortcut connections when the "use_shortcut" attribute is true.

To inspect gradient flow, the example initializes a neural network without shortcut connections, uses layer sizes 3 3 3 3 3 1, takes a three-dimensional sample input with values 1, 0, and minus 1, and sets use shortcut to false. In layers with input dimension 3 and output dimension 3, each neuron has three weights, so the weight matrix is 3 by 3, while the final layer has a 3 by 1 weight matrix. To summarize gradient flow per layer, the backward pass computes gradients for all weights in a layer and then takes the mean of those values as that layer's mean gradient. The loss is defined from the model output y and the ground truth; for example, when the ground truth is zero, the loss is based on y - 0. In backpropagation, the partial derivatives of the loss are computed starting from the output layer and then moving backward through earlier layers until the first layer. The code sets the target to zero, computes the model output for x, uses a squared loss, calls loss.backward(), and prints a single gradient value per layer as the mean absolute gradient of that layer's 3×3 gradient values. In PyTorch, the backward method conveniently computes loss gradients during the backward pass.

The mean gradient becomes much smaller in earlier layers, illustrating the vanishing gradient problem. Adding shortcut connections creates alternative paths for gradients to flow between layers. With shortcut connections, the earlier-layer gradients remain much larger and do not become negligible, showing that shortcut connections solve the vanishing gradient problem in this example. With shortcut connections, the gradient values stabilize toward the first layer instead of shrinking to a vanishingly small value. Shortcut connections are an important way to overcome the vanishing gradient problem in deep neural networks. They are a core building block of large language models and help training by ensuring consistent gradient flow across layers in GPT models. They help stabilize training.

Returning to the Transformer block, the reason for those arrows is now clear. The arrows between layers are shortcut connections, and they provide an alternative path for gradients to flow. A shortcut connection can be added between layers of the Transformer block to help solve the vanishing gradient problem. Because backpropagation passes through the entire Transformer block from output to input, the architecture must ensure that gradients do not vanish so that learning remains stable.

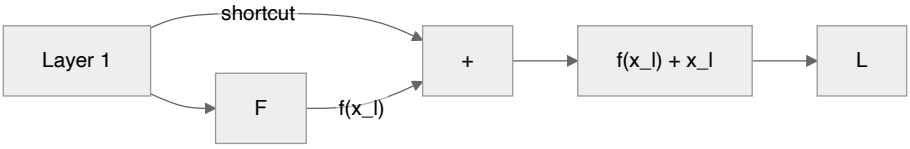


Figure 3.4: A residual-style block where the input from Layer 1 goes through a function F and is also added directly via a shortcut, producing f(x_l)+x_l; below it, derivative expressions indicate the gradient includes an added 1 term. Redrawn from the board in L22 30:56.

Listing 03.11.1: Sets layer sizes and a three-dimensional sample input, initializes the network without shortcut connections, and calls the gradient-printing helper on that model. Source: rasbt/LLMs-from-scratch, `ch04/01_main-chapter-code/ch04.ipynb`, Apache-2.0, commit cdbd33e.

```
layer_sizes = [3, 3, 3, 3, 3, 1]

sample_input = torch.tensor([[1., 0., -1.]])

torch.manual_seed(123)
model_without_shortcut = ExampleDeepNeuralNetwork(
    layer_sizes, use_shortcut=False
)
print_gradients(model_without_shortcut, sample_input)
```

Output (executed):

```
layers.0.0.weight has gradient mean of 0.00020173587836325169
layers.1.0.weight has gradient mean of 0.0001201116101583466
layers.2.0.weight has gradient mean of 0.0007152041071094573
layers.3.0.weight has gradient mean of 0.001398873515427112 6
layers.4.0.weight has gradient mean of 0.005049645435065031
```

- *Checked by running the code:* `layer_sizes[-1]` = 1 (the text says 1)
- *Checked by running the code:* `sample_input.shape[1]` = 3 (the text says 3)
- *Checked by running the code:* `sample_input[0,0].item()` = 1 (the text says 1)
- *Checked by running the code:* `sample_input[0,1].item()` = 0 (the text says 0)
- *Checked by running the code:* `model_without_shortcut.layers[0][0].in_features` = 3 (the text says 3)
- *Checked by running the code:* `model_without_shortcut.layers[0][0].out_features` = 3 (the text says 3)

- *Checked by running the code:* `model_without_shortcut.layers[-1][0].in_features` = 3 (the text says 3)
- *Checked by running the code:* `model_without_shortcut.layers[-1][0].out_features` = 1 (the text says 1)

Key takeaways

- A language model pipeline turns words into tokens, tokens into token IDs, and token IDs into vector embeddings, and GPT uses byte pair encoding so tokens are not generally the same as words even though this chapter uses the terms interchangeably for simplicity.
- Embedding vectors represent tokens in a high-dimensional space where semantically related words are closer together, but an input embedding by itself does not say how a word relates to other words in the sentence or how important those other words are for prediction.
- Attention transforms input embeddings into context vectors, which keep a token's meaning while also adding information about its relationships to other tokens and how much attention they should receive, making next-word prediction better and more reliable.
- To build a context vector for a query token such as "journey," attention uses dot products with all input vectors to get attention scores, larger scores mean stronger alignment and higher attention, and the scores are normalized into attention weights with softmax so they are positive and sum to one.
- The context vector is computed by scaling each input embedding by its attention weight and summing them, and for all tokens at once this becomes matrix multiplication of the attention weight matrix by the input matrix.
- Modern transformers add trainable query, key, and value weights, use causal masking so a token can attend only to itself and earlier tokens, apply dropout in the attention mechanism, and place the attention module inside a transformer block with layer normalization and shortcut connections.

4 Assembling and Pretraining a GPT Model

Based on L24: Coding the 124 million parameter GPT-2 model; L26: Measuring the LLM loss function; L28: Coding the entire LLM Pre-training Loop; L30: Top-k sampling in Large Language Models.

In the previous chapter, the transformer block brought together layer normalization, feed-forward layers, activation, and shortcut connections; here, those pieces are assembled into the full GPT pipeline that turns tokenized text into model outputs. This matters because the aim is not just to study isolated parts, but to assemble a fully working version of the original 124 million parameter GPT-2 model and to see how text moves through tokenization, dropout, transformer blocks, another layer normalization, and the output head. It also marks the shift from architecture to training, because next-token predictions need training to make sense rather than remain random.

We begin with what the model is and how the full GPT pipeline is assembled, then review configuration, tokenization, vocabulary, inputs, batches, embeddings, dimensions, dropout, layer normalization, and transformer-block internals. From there, the chapter stacks blocks into the final model, implements the code, counts parameters, defines next-token targets, introduces loss and perplexity, prepares data, and walks through the basic training loop with monitoring and evaluation mode. It closes by turning logits into text with greedy, temperature, and top-k decoding, and by noting a small narrative gap in how decoding is positioned within the chapter.

At a glance

- The chapter's central build target is a fully working version of the original 124 million parameter GPT-2 model.
- In the GPT architecture, text is tokenized, turned into embeddings, passed through dropout and transformer blocks, and then processed by a final layer normalization and linear output head to get logits.
- Masked multi-head attention is the key part of a transformer block, and the transformer block is the key part of the overall GPT model.
- GPT-2 uses a vocabulary of 50,257 tokens, and the model input is a tensor whose rows are batches and whose columns are token positions in the sequence.
- Token and positional embeddings use 768-dimensional vectors in this GPT-2 example, and both are initialized randomly. The token embedding parameters are learned during training.

4.1 What the model is and how the full GPT pipeline is assembled

How does a collection of separate language-model components become a complete GPT model that can take text as input and produce next-token predictions? The goal here is to assemble a fully working version of the original 124 million parameter GPT-2 model. The full architecture is built by bringing together components that were implemented separately and coding the entire GPT model as one system. In this assembled model, text is tokenized, the input embeddings pass through dropout, those embeddings are sent into transformer blocks, and the result is postprocessed by another layer normalization and another linear output layer to produce the GPT model output. Masked multi-head attention is the key component inside a transformer block, and the transformer block is the key component of the entire GPT model.

> **Note.** The course's reference code differs here: For instruction finetuning, the reference uses GPT-2 Medium (355M) instead of 124M because the smallest model is too limited for good qualitative results.

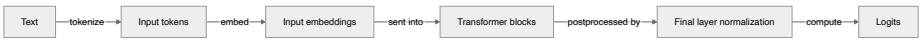


Figure 4.1: The assembled GPT model maps tokenized input through embeddings, transformer blocks, final normalization, and an output head to produce logits.

This overall flow can also be described directly in terms of the model stack. The implementation stacks token embeddings, positional embeddings, a dropout layer, transformer blocks, another layer normalization, and an output head, and then directly gets the output. The forward method takes a batch of input tokens, computes their embeddings, applies positional embeddings, passes the sequence through the transformer block, normalizes the final output, and computes logits representing unnormalized probabilities for the next tokens. The model itself is an instance of the GPT model class. It performs the operations in the GPT architecture schematic and gives the LLM output at the end.

What is this model trying to learn once the pipeline is assembled? The GPT model is trained for next-word prediction. For example, given the input text "every effort moves you", the model is trained to predict the next word. At this stage, the GPT architecture that has been built takes input text and can predict next tokens or next words. At this stage, the model can predict next tokens, but the predictions can be completely random. Training is the second stage of building the large language model, and it is needed so that the next token it predicts makes sense.

4.2 Configuration, tokenizer, and prerequisite vocabulary concepts

How does raw text get tied to the fixed numbers a GPT model is built around, and why do those numbers matter before any code runs? The configuration used here includes vocabulary size, context length, embedding dimension, number of attention heads, number of Transformer blocks, dropout rate, and whether query, key, and value use a bias term. The vocabulary size is the number of tokens in the vocabulary. GPT-2 has a vocabulary of size 50,257. The tokenizer used here is tiktoken, and it uses byte pair encoding to create a vocabulary of tokens. The constructed model uses the same vocabulary size, 50,257. For simplicity, one token is treated as one word in the running example. That simplification is not how GPT-2 actually tokenizes text, because GPT-2 uses byte pair encoding as a subword tokenizer. As a result, one word is not necessarily one token, since tokens can be small subwords or even individual characters.

The same setup also fixes how much prior text the model can use at once. The context size is the maximum number of words that can be used to predict the next word. In the small example built around the four words "every effort moves you", the context size is four, so only four positions matter. GPT-2 used a context length of 1024. Here the model uses a context length of 256 so the code is easier to train locally, can run on a learner's machine in less time, and requires fewer computational resources. The same code can be used with a longer context length if there is enough compute power and memory.

The GPT-2 configuration shown in code sets vocabulary size to 50257, context length to 1024, embedding dimension to 768, number of attention heads to 12, number of transformer blocks to 12, dropout rate to 0.1, and query-key-value bias to false. The configuration used for the model here keeps vocabulary size 50257, embedding dimension 768, 12 attention heads, 12 transformer layers, dropout 0.1, and query-key-value bias false, while reducing context length to 256. The embedding dimension is the higher-dimensional space into which token embeddings are projected to capture semantic meaning. Here that embedding dimension is 768, because the inputs are projected into a 768-dimensional space. The number of attention heads is the number of self-attention mechanism blocks within one transformer. The number of layers specifies how many transformer blocks the model has. In GPT-2, 12 transformer blocks are used, and each transformer's multi-head attention module uses 12 attention heads. The query-key-value bias setting means no bias term is used when initializing the query, key, and value weight matrices. Here that setting is false because the bias term is not needed for those query, key, and value weight matrices.

> **Note.** The course's reference code differs here: Later code shortens context_length to 256 for efficiency; the original GPT-2 124M configuration uses 1024.

Listing 04.02.1: This code defines the GPT-2 124M configuration dictionary with vocabulary size 50257, context length 1024, embedding dimension 768, 12 attention heads, 12 layers, dropout 0.1, and qkv bias disabled. Source: rasbt/LLMs-from-scratch, `ch04/01_main-chapter-code/ch04.ipynb`, Apache-2.0, commit cdbd33e.

```
GPT_CONFIG_124M = {
    "vocab_size": 50257,    # Vocabulary size
    "context_length": 1024, # Context length
    "emb_dim": 768,         # Embedding dimension
    "n_heads": 12,          # Number of attention heads
    "n_layers": 12,         # Number of layers
    "drop_rate": 0.1,       # Dropout rate
    "qkv_bias": False       # Query-Key-Value bias
}
```

- *Checked by running the code:* `GPT_CONFIG_124M['vocab_size']` = 50257 (the text says 50257)
- *Checked by running the code:* `GPT_CONFIG_124M['vocab_size']` = 50257 (the text says 50257)
- *Checked by running the code:* `GPT_CONFIG_124M['context_length']` = 1024 (the text says 1024)
- *Checked by running the code:* `GPT_CONFIG_124M['vocab_size']` = 50257 (the text says 50257)
- *Checked by running the code:* `GPT_CONFIG_124M['context_length']` = 1024 (the text says 1024)
- *Checked by running the code:* `GPT_CONFIG_124M['emb_dim']` = 768 (the text says 768)
- *Checked by running the code:* `GPT_CONFIG_124M['n_heads']` = 12 (the text says 12)
- *Checked by running the code:* `GPT_CONFIG_124M['n_layers']` = 12 (the text says 12)
- *Checked by running the code:* `GPT_CONFIG_124M['drop_rate']` = 0.1 (the text says 0.1)
- *Checked by running the code:* `GPT_CONFIG_124M['vocab_size']` = 50257 (the text says 50257)
- *Checked by running the code:* `GPT_CONFIG_124M['emb_dim']` = 768 (the text says 768)
- *Checked by running the code:* `GPT_CONFIG_124M['n_heads']` = 12 (the text says 12)
- *Checked by running the code:* `GPT_CONFIG_124M['n_layers']` = 12 (the text says 12)
- *Checked by running the code:* `GPT_CONFIG_124M['drop_rate']` = 0.1 (the text says 0.1)

- *Checked by running the code:* `GPT_CONFIG_124M['emb_dim']` = 768 (the text says 768)
- *Checked by running the code:* `GPT_CONFIG_124M['n_layers']` = 12 (the text says 12)
- *Checked by running the code:* `GPT_CONFIG_124M['n_heads']` = 12 (the text says 12)

4.3 Inputs, batches, embeddings, and dimensions

How does a GPT model organize its inputs so that text can move through the network with consistent tensor shapes? The input to the large language model is a tensor whose rows correspond to batches. In the GPT forward method, the input tensor is arranged so that the number of rows is the number of batches and the number of columns is the sequence length, or the number of tokens being considered. Inputs ideally come in batches, and in the coding example there are two batches with four token IDs in each batch. For simplicity, it is helpful to focus on the first batch and apply the same reasoning to the second batch afterward.

The first step is to map each input token to a token ID in the vocabulary. In the running example, the first batch contains four tokens, shown as “every”, “effort”, “moves”, and “you”, and those four tokens are converted into four token IDs. From there, the representation changes form. A token embedding vector is used to capture semantic meaning between words. Computers cannot directly understand words, and token IDs alone are not enough because the model needs representations that capture meaning between words. Words with similar meanings, such as “dog” and “puppy” or “cat” and “kitten”, should have embedding vectors that are closer to each other.

The next question is what size these vectors should have and where their values come from. In this GPT-2 example, the embedding size is 768, which is the size used in the smallest GPT-2 model. Each token in the sequence is encoded as a 768-dimensional vector. For each token, the token embedding can be visualized as a 768-dimensional vector, and positional embeddings can be visualized in the same way. Token embedding vectors are not known in advance, so they are initialized randomly in the 768-dimensional space and learned during GPT model training along with the other embedding parameters.

Meaning alone is not enough, because the model also has to know where each token appears in the sequence. Besides word meaning, the model also needs to capture the position of each word in the sentence. A positional embedding is an embedding vector that represents a token’s position in the sequence. In the example sequence, there are four positions, and position one, position two, position three, and position four are each represented by a 768-dimensional positional embedding vector. Like token embeddings, positional embedding values are initially random because their best values are not known beforehand, and these values are trained during the training procedure. The positional embedding vectors have size 768, the same size as the token embeddings.

With both kinds of embeddings available, the model can combine token identity with token position. To form the input embedding for each token, the model adds that token's token embedding to the positional embedding for its position. Because token embeddings and positional embeddings have the same dimensions, each input embedding also has dimension 768. The relationship is written as input embedding equals token embedding plus positional embedding. In the GPT model, token IDs are converted to token embeddings, positional embeddings are generated for each position in the sequence, and the two are added to form the input embeddings.

A clear picture of these shapes matters because the code is not the hard part; tracking dimensions is. The code for this GPT model is described as pretty simple and straightforward, but students often get confused because many tensors and many dimensions come into the picture. For that reason, a visual flow map is used to follow a specific input sequence through the GPT model step by step and to represent input and output dimensions visually. The purpose of that flow map is to make dimension changes easier to understand across token embedding, positional embedding, input embedding, dropout, the Transformer block, layer normalization, and the output head, which are hard to visualize directly from code.

Listing 04.03.1: This code adds positional embeddings to token embeddings to form input embeddings and prints their shape. Source: rasbt/LLMs-from-scratch, `ch02/01_main-chapter-code/ch02.ipynb`, Apache-2.0, commit cdbd33e.

```
input_embeddings = token_embeddings + pos_embeddings
print(input_embeddings.shape)

# uncomment & execute the following line to see how the embeddings look like
# print(input_embeddings)
```

Output (executed):

```
torch.Size([8, 4, 256])
```

4.4 Dropout, layer normalization, and transformer block internals

How do the embeddings prepared earlier become the internal computation of a Transformer block, and why are dropout and normalization needed before and inside it? Before the Transformer block begins, the sequence goes through token embedding, positional embedding, input embedding by adding them together, and then dropout. These embedding steps happen outside the Transformer block, and after this dropout the model enters the block. Dropout randomly turns some elements of each input embedding to zero

according to the dropout rate. For each token, the input embedding is a 768-dimensional vector, and dropout is applied to every token. If the dropout rate is 50%, then around 384 of the 768 elements in each embedding are turned off on average, not exactly half every time, because the process is probabilistic. Dropout is implemented to prevent overfitting and improve generalization. One way to understand it is that dropout prevents some neurons from being lazy: when neurons are turned off randomly, neurons that were depending on others have to learn something themselves, which improves generalization.

Once the block starts, the first operation is layer normalization. Layer normalization normalizes the values within each token embedding so that their mean is 0 and their variance is 1. It is performed separately for every single token to improve stability during training. Layer normalization subtracts the mean from every value and divides by the square root of the variance. In code, the layer normalization class does this for each token's 768-dimensional representation and also includes trainable scale and shift parameters. From there, attention does the key work. Before attention is applied, each token vector contains meaning about itself, but it does not yet capture how much attention it should give to other words when predicting the next word. Masked multi-head attention converts embedding vectors into context vectors. A context vector keeps the token's semantic meaning while incorporating information from the other tokens in the sentence through attention. Attention is the most important step in the GPT model because it determines which words are important to look at and how different words relate to each other when predicting the next word. After multi-head attention, the output size is still the same, so each token still has a 768-dimensional representation, now interpreted as a context vector.

When embeddings enter the Transformer block, dimensionality is preserved, so each token is still represented by a 768-dimensional vector. After multi-head attention there is another dropout layer that randomly turns some context vector values to zero. A shortcut connection adds the output of a layer back to the input that entered the Transformer block. Shortcut connections provide another route for gradient flow and help prevent the vanishing gradient problem, which makes training proceed more smoothly. After this shortcut connection, another round of layer normalization is applied without changing the dimensions. In that second layer normalization, each token's values are normalized by subtracting the mean and dividing by the square root of the variance. The feed-forward neural network is an expansion-contraction network. It processes each token sequentially. Each token enters this network with embedding size 768, is projected into a hidden layer of size 4 times 768, and is then contracted back to the original dimension. The output of the feed-forward neural network has the same dimension as its input. This expansion and contraction allow a richer exploration of parameters and help the model predict the next word better. Because the middle layer has 4 times the embedding dimension, it contains many parameters and provides a richer exploration space. After the feed-forward network there is another dropout layer that randomly sets certain values in every token to zero, and typically one dropout rate is specified at the start and used everywhere dropout layers are implemented.

> **Note.** The course's reference code differs here: The reference applies dropout to attention weights; here the lecture describes dropout after multi-head attention on context vector values, another common placement.

One more shortcut connection completes the block. The second shortcut connection adds the output after this dropout layer to the input of that sublayer. The output from the shortcut connection is the Transformer block output. A Transformer block preserves the token embedding dimension, so its output has the same dimensions as its input, with each token still represented by a 768-dimensional vector. The GPT model preserves dimensionality throughout these steps, which makes it easier to add multiple modules together because the dimensions do not change. The Transformer block is the heart of the entire GPT architecture. Its internal workflow can be read as eight steps in order: layer normalization, masked multi-head attention, dropout, a shortcut connection, another layer normalization, a feed-forward neural network, dropout, and a final shortcut connection to produce the block output. In the coded summary of the block, the steps are described as input embeddings going through dropout, layer normalization, masked multi-head attention, dropout, a shortcut connection, another layer normalization, a feed-forward neural network, dropout, and another shortcut connection. The feed-forward block has an input and output of the embedding dimension, expands to a hidden layer four times the embedding dimension, then contracts back to the embedding dimension, and uses a GELU activation function. GELU differs from ReLU because for negative inputs it is not zero and it is differentiable at zero, which is why GELU is generally used in LLM training.

4.5 Stacking blocks, final normalization, output head, and logits

How does the model turn the transformer's token representations into the final tensor used for next-word prediction? The GPT model takes input tokens, converts them into token embeddings, adds positional embeddings, applies dropout, passes the result through transformer blocks, applies a final normalization layer, and returns logits. After the transformer block, there is another layer normalization step even though the transformer block itself already contains two normalization layers. In this layer normalization, the embedding values within each token are normalized so that the resulting values have mean zero and variance one. This normalization is applied for all four tokens, and it does not change the tensor dimensions. Each token still has an embedding size of 768 after this step.

The output head is the final neural network stage of the GPT model. Its input can be a representation with four tokens, where each token has 768 dimensions. The output head maps each 768-dimensional token representation to an output of size 50257. The output of this final neural network is called the logits matrix. With four input tokens, that logits matrix has four rows and a number of columns equal to the vocabulary size, not 768. For every token position, the model therefore produces a 50,257-dimensional

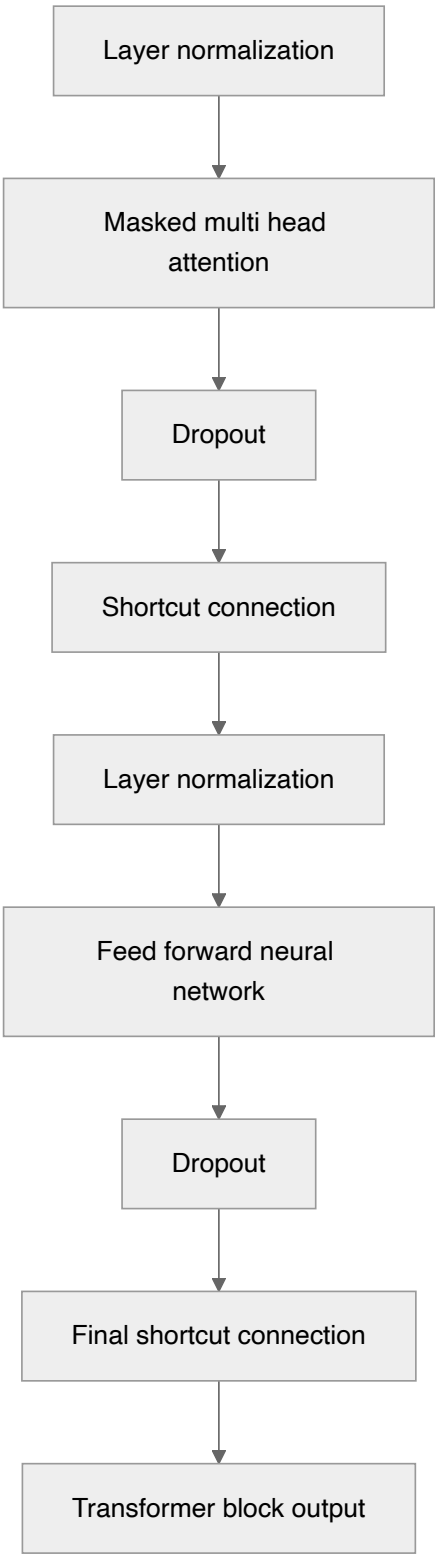


Figure 4.2: A Transformer block applies normalization, masked multi head attention, dropout, a shortcut connection, another normalization, a feed forward network, dropout, and a final shortcut while preserving embedding size.

logits vector because it is predicting the next word. This output head is the only step in this flow where the dimensions change, from four tokens by 768 to four tokens by 50257. For one batch with four tokens, the final logits tensor has shape 4 by 50257.

The rows of that tensor correspond to prediction tasks, not to a single prediction for the whole input sequence. For the four-token context “every effort moves you”, there are four prediction tasks, one for each token position, not just one prediction for the full input sequence. The four tasks are predicting “effort” from “every”, “moves” from “every effort”, “you” from “every effort moves”, and the output for “every effort moves you”. In the example sequence “every effort moves you forward”, the model should predict “effort” after “every”, “moves” after “every effort”, “you” after “every effort moves”, and “forward” after “every effort moves you”. The returned logits have one row per input token position and a number of columns equal to the vocabulary size.

Batching adds one more dimension in front of the token and vocabulary dimensions. When there are multiple batches, the output tensor includes the batch size as the first dimension. For batch size 2 and context size 4, the output tensor shape is 2 by 4 by 50257. The output tensor dimensions are ordered as batch size, then number of tokens or context size, then vocabulary size 50257. More generally, the GPT model outputs logits with shape batch size by number of tokens by vocabulary size. The logits or probability tensor is also described with shape batch size by number of tokens by vocabulary size. In one concrete example, the logits tensor has shape 2 by 3 by 50257, and the target tensor has shape 2 by 3. The probability tensor shape 2 by 3 by 50257 corresponds to two batches, three token positions, and a size of 50257 in the last dimension. The model’s immediate output is this final tensor, and the next step is to decode it to get the next word.

4.6 Coding the model and parameter accounting

How does a GPT model turn integer token IDs into the logits that will later drive prediction, and why does that simple-looking code already involve more than 100 million parameters? The model workflow starts with input IDs, converts them into token embeddings, adds positional embeddings, applies dropout, passes the result through the Transformer block, applies another layer normalization layer, applies an output head layer, and returns the logits. After the input embeddings are formed by combining token embeddings and positional embeddings, the remaining stages are dropout, the Transformer blocks, another layer of normalization, and the output head. An earlier version of the GPTModel class already had a forward method, but its Transformer block and layer normalization were only dummy implementations. Over the last three to four lectures, the layer normalization class, feed-forward neural network class, and Transformer class were coded. The Transformer block class itself was coded in the last step of that sequence.

The stacked Transformer portion is implemented by chaining 12 Transformer blocks with nn.Sequential. A simple test batch passed through the model produces an output whose last dimension is the vocabulary size, 50257. The full output tensor has shape 2 by 4 by 50257, corresponding to two batches, four tokens in each batch, and the vocabulary size. Even this compact implementation has more than 100 million parameters because it includes 12 Transformer blocks, eight steps in each block, expansion and contraction layers inside each block, and additional token and positional embedding parameters.

Using the numel method, short for number of elements, we can collect the total number of parameters in the model's parameter tensors. If you print out the total number of parameters, you will see that the number of parameters is equal to 163 million. The model's parameters come from token embeddings, positional embeddings, Transformer block parameters, and the final layer before the logits. The token embedding parameter count is vocabulary size multiplied by embedding dimension, and the positional embedding parameter count is context size multiplied by embedding dimension. Together, the token and positional embeddings contribute 38.4 million parameters, and those embedding parameters are also optimized during training. The query, key, and value are three trainable matrices, each with input and output embedding dimension 768, for 1.77 million parameters. The multi-head attention block also includes an output head, and the total number of parameters in that block is 2.36 million.

The feed-forward network uses an expansion-contraction pattern, projecting from the embedding dimension to a hidden layer of four times that size and then compressing back, and it contributes 4.72 million parameters. One Transformer block contributes 2.36 million plus 4.72 million parameters, and across 12 Transformer blocks that totals 85.2 million parameters. The final layer has embedding dimension multiplied by vocabulary size parameters, which is 38.4 million. Adding 38.4 million embedding parameters, 85.2 million Transformer parameters, and 38.4 million final-layer parameters gives about 162 million parameters in total. The model being trained has on the order of more than 100 million parameters.

The gap between 163 million parameters in this implementation and 124 million parameters in the smallest GPT-2 comes from weight tying. In the original GPT-2 architecture, weight tying means reusing the token embedding layer weights in the output layer. The token embedding layer has shape 50257 by 768, and the output layer has the same shape, which is why GPT-2 reuses those weights. If the output-layer parameters are excluded because of weight tying, the total becomes 124 million parameters, matching the GPT-2 model size. Weight tying reduces the overall memory footprint and the computational complexity of the model. This implementation keeps separate token embedding and output layers because separate layers give better training and model performance, and the same choice is described as true for modern LLMs as well.

For this model, 163 million parameters require around 600 megabytes of space. Assuming each parameter is a 32-bit float taking 4 bytes, the size is computed by multiplying the total number of parameters by 4, which gives about 620 MB. This illustrates the relatively large storage capacity required to accommodate even relatively small LLMs,

while extremely large models are described as impossible to run on local devices, which is why GPU access is needed.

Listing 04.06.1: This code sums the number of elements in all model parameter tensors and prints the total parameter count. Source: rasbt/LLMs-from-scratch, ch04/01_main-chapter-code/ch04.ipynb, Apache-2.0, commit cdbd33e.

```
total_params = sum(p.numel() for p in model.parameters())
print(f"Total number of parameters: {total_params:,}")
```

Output (executed):

```
Total number of parameters: 163,009,536
```

- *Checked by running the code:* `total_params` = 1.6301e+08 (the text says 1.63e+08)
- *Checked by running the code:* `total_params` = 1.6301e+08 (the text says 1.62e+08)
- *Checked by running the code:* `total_params` = 1.6301e+08 (the text says 1e+08)
- *Checked by running the code:* `total_params` = 1.6301e+08 (the text says 1.63e+08)
- *Checked by running the code:* `total_params` = 1.6301e+08 (the text says 1.63e+08)
- *Checked by running the code:* `(32 // 8)` = 4 (the text says 4)

Listing 04.06.2: This code multiplies the total parameter count by 4 bytes per parameter, converts the result to megabytes, and prints the model size. Source: rasbt/LLMs-from-scratch, ch04/01_main-chapter-code/ch04.ipynb, Apache-2.0, commit cdbd33e.

```
# Calculate the total size in bytes (assuming float32, 4 bytes per parameter)
total_size_bytes = total_params * 4

# Convert to megabytes
total_size_mb = total_size_bytes / (1024 * 1024)

print(f"Total size of the model: {total_size_mb:.2f} MB")
```

Output (executed):

```
Total size of the model: 621.83 MB
```

4.7 Next-token prediction tasks, targets, and basic generation from logits

How does a GPT model turn an input sequence into next-token predictions, and what counts as the correct answer for each position? Here, the word "target" means the true values. The loss function is determined by the predicted values and the target values and how close they are. For an input like "every effort", there are three prediction tasks. From the prefix "every", the correct next token is "effort". From the prefix "every effort", the correct next token is "moves". From the prefix "every effort moves", the correct next token is the token represented in the captions as "U". For the input tokens "I really like", there are also three prediction tasks. From the prefix "I", the correct next token is "really". From the prefix "I really", the correct next token is "like". From the prefix "I really like", the correct next token is "chocolate".

When the input has three tokens, there are three prediction tasks. The targets are the true prediction values for the given inputs. In code, the first batch targets are 3626, 610, and 345, and the second batch targets are 1107, 588, and 11311. These targets are the input shifted by one position. The reason given is that there are three input-output pairs.

The function "text to token IDs" converts input text into token IDs using the tiktoken tokenizer, and "token IDs to text" does the reverse conversion. For a simple illustration, it is enough to use a vocabulary with seven elements, even though the actual example vocabulary has 50257 elements. In that toy vocabulary, the input text "every effort moves" maps to the token IDs 2, 1, and 4. The input tensor is then passed into the GPT model, which returns an output tensor called the logit tensor. To generate outputs, the logits are passed through softmax because the logits are not yet normalized into probabilities between 0 and 1. In the batched case, the probabilities have shape batch size, number of tokens, and vocabulary size, which is described as 2, 3, 50257. Each row contains the probabilities of the possible next tokens. For each input prefix, the row gives the probabilities of the next token.

To get output token IDs, the inputs pass through the GPT architecture to produce logits, softmax is applied to obtain probabilities, and then the index with the highest probability is extracted from each row. Using torch.argmax with dim=-1 returns the index of the maximum value along the columns for each row. In the toy example, the three predicted output token IDs for "every effort moves" are 1, 4, and 5. These correspond to the tokens "effort", "moves", and "you". In the cited example, the range of the output token IDs for GPT-2 is stated as 0 to 50257. After obtaining output token IDs, they can be decoded back into text to inspect the model's prediction.

For the first batch, the target text is "effort moves you", but the decoded output is "armed H". This shows a large mismatch between outputs and targets. The model has not yet been optimized, the loss function has not yet been minimized, and the current output token IDs are not the actual target token IDs because training has not happened

yet. The goal is for the model's output token IDs to be as close as possible to the target token IDs. The loss function is defined between the outputs and the targets so that the target text and output text become as close as possible. An untrained model does not yet produce good text, which is why a loss function is needed to measure how good the text is.

4.8 Loss function: from target probabilities to cross-entropy and perplexity

How can next-token predictions be turned into a number that tells whether a language model is doing well, and why does that number matter for training? In traditional machine learning, regression generally uses least-square or mean-squared-error losses, while classification uses losses such as cross-entropy or hinge loss. A loss function is a quantitative metric that captures how good or bad the large language model's output is. If a loss function quantifies LLM performance, then training can aim to minimize that loss so the model does better and better. Once a loss function is constructed, gradient-descent-based backpropagation algorithms can be used to train the model. Here, text evaluation means defining a loss function that compares what the model generated with the good output that is actually wanted. That loss is determined by the predicted values, the target values, and how close they are. Because a randomly initialized LLM does not initially produce the outputs that are wanted, it must be trained.

Before training, those correct-index probabilities are not necessarily the highest, because the model is not trained yet. The probability tensor is indexed at the target positions, and those probabilities are collected together. With two batches and three token predictions in each batch, there are six target probabilities in total: "p11", "p12", and "p13" from the first batch, and "p21", "p22", and "p23" from the second. The training goal is to make the selected target probabilities as close to one as possible, so that the output indices with the maximum probabilities line up with the target indices. Conceptually, the workflow starts from the logits tensor, converts it to probabilities with softmax, identifies the target-index probabilities, and merges them together.

Because the problem is expressed in probabilities, logarithms and cross-entropy are the natural tools to use. Rather than working directly with the target probabilities, it is better mathematically and from an optimization perspective to take their logarithms, average them, and then negate the result. The negative log-likelihood is the negative of the mean of the log target probabilities. For these six collected probabilities, the negative log-likelihood is the negative of the sum of log "p11" through log "p23" divided by 6. Using the negative form makes the objective intuitive, because the task becomes minimizing a loss instead of maximizing it. This negative log-likelihood is also called cross-entropy loss. Cross-entropy loss essentially measures the difference between two probability distributions, and in this setting it can be viewed as categorical cross-entropy. Minimizing cross-entropy pushes the target probabilities toward one, so the loss should

be as low and as close to zero as possible. If the loss reaches zero, that indicates the mean is effectively at one and the target probabilities are at or near one, which is the training goal. When this loss is minimized, the predicted next-token indices align closely with the target indices, which indicates that the language model is performing well.

The same computation can be written explicitly in code before using a library helper. A first coding step is to find the target probabilities p11, p12, p13, p21, p22, and p23. For the first batch, the code looks at rows 0, 1, and 2 and picks the probability values at target indices 3626, 610, and 345. For the second batch, it again looks at rows 0, 1, and 2 and picks the probability values at target indices 1107, 588, and 11311. After selecting them, the six probabilities are concatenated, their logarithms are taken, the mean of those log values is computed, and then the negative is taken to obtain the negative log-likelihood loss. In deep learning, it is conventional to use negative log-likelihood rather than positive log-likelihood so that the loss is minimized instead of maximized.

In practice, the loss is usually computed in a simpler way. The logits are flattened from shape 2 by 3 by 50257 to shape 6 by 50257 by merging the batch and token dimensions. The target tensor is flattened from two rows and three columns into six values. After flattening, the target sequence is 3626, 610, 345, 107, 588, and 11311, and the example uses these indices to pick out probabilities such as p11, p12, and p23. The call "torch.nn.functional.cross_entropy(logits_flat, targets_flat)" computes the loss from flattened logits and flattened targets. This function first applies softmax to the logits to convert them into probabilities, then finds the probabilities at the target indices, takes the negative log, and averages them. A single call performs all the required steps and returns the final loss between the output and the target. Conceptually, inputs such as "every effort moves" and "I really like" pass through the GPT architecture to produce logits, the batches are merged into "logits_flat", the target-index probabilities are examined, and the cross-entropy loss is computed from them. In an ideal model, the probabilities at the target indices would be close to one, but in an unoptimized LLM they are typically small, so the goal is to bring them as close to one as possible by taking the negative log of these probabilities and averaging them as the cross-entropy loss. The function is described as finding the cross-entropy loss between the input logits and the target. The goal during training is to reduce this categorical cross-entropy loss as close to zero as possible.

Cross-entropy is not the only way to report model quality. Perplexity is another measure of loss, described as a minor modification of cross-entropy that measures how well the model's predicted probability distribution matches the actual distribution of words in the dataset. It is presented as a more interpretable way to understand model uncertainty in predicting the next token. A lower perplexity score means better predictions. Perplexity is calculated as the exponent of the loss. In the example, the loss is 10.79 and the perplexity is 48725. A perplexity of 48725 means the model is roughly as uncertain as if it had to choose the next token randomly from about 48725 tokens in the vocabulary. For the input "every effort moves", that means the current model is at a stage where predicting the next token is like choosing among 48725 possible tokens, which indicates high uncertainty and poor performance. If the perplexity were 2, the model would

effectively need to choose between only two tokens, which would indicate a very accurate model. A perplexity of 48725 also suggests that about 48000 tokens are effectively equally likely to be the next token, so the model is not good at all. Perplexity is usually more interpretable than a raw categorical cross-entropy value such as 10, because it can be related directly to the vocabulary size.

Listing 04.08.1: This code flattens the logits across batch and token dimensions and flattens the target tensor into a single sequence, then prints their shapes. Source: rasbt/LLMs-from-scratch, `ch05/01_main-chapter-code/ch05.ipynb`, Apache-2.0, commit cdbd33e.

```
logits_flat = logits.flatten(0, 1)
targets_flat = targets.flatten()

print("Flattened logits:", logits_flat.shape)
print("Flattened targets:", targets_flat.shape)
```

Output (executed):

```
Flattened logits: torch.Size([6, 50257])
Flattened targets: torch.Size([6])
```

- *Checked by running the code:* `logits.shape[0]` = 2 (the text says 2)
- *Checked by running the code:* `logits.shape[1]` = 3 (the text says 3)
- *Checked by running the code:* `logits.shape[2]` = 50257 (the text says 50257)
- *Checked by running the code:* `logits_flat.shape[0]` = 6 (the text says 6)
- *Checked by running the code:* `logits_flat.shape[1]` = 50257 (the text says 50257)
- *Checked by running the code:* `targets_flat[0]` = 3626 (the text says 3626)
- *Checked by running the code:* `targets_flat[1]` = 6100 (the text says 610)
- *Checked by running the code:* `targets_flat[2]` = 345 (the text says 345)
- *Checked by running the code:* `targets_flat[3]` = 1107 (the text says 107)
- *Checked by running the code:* `targets_flat[4]` = 588 (the text says 588)
- *Checked by running the code:* `targets_flat[5]` = 11311 (the text says 11311)

Listing 04.08.2: This code computes the cross-entropy loss from the flattened logits and flattened targets and prints the resulting loss value. Source: rasbt/LLMs-from-scratch, `ch05/01_main-chapter-code/ch05.ipynb`, Apache-2.0, commit cdbd33e.

```
loss = torch.nn.functional.cross_entropy(logits_flat, targets_flat)
print(loss)
```

Output (executed):

```
tensor(10.7940)
```

- *Checked by running the code:* `loss.item()` = 10.794 (the text says 10.79)
- *Checked by running the code:* `round(loss.item())` = 11 (the text says 10)

4.9 Preparing data for pre-training

How does raw text become the training examples needed to start pre-training a GPT model, and why does that preparation matter before the backward pass can even run? At this stage, pre-training means training the large language model by minimizing the loss function through backpropagation. The loss function is obtained by comparing the model output with the target value, and training and validation losses are then calculated. The first preprocessing step is to split this dataset into training and validation data with a training ratio of 0.9, using the initial 90% for training and the remaining 10% for validation.

That split is only the beginning, because autoregressive language modeling still needs targets even though it does not use labels in the usual sense. Since LLMs are autoregressive models, they do not have labels, so the training and validation data must be divided internally into input-target pairs. A data loader is the component that looks at the dataset and creates these input-target pairs. Here, the target is the true or actual value that the LLM is meant to approximate. Input-target pairs are formed by shifting each input row by one position to create the corresponding target row. To build the inputs, take spans whose length equals the context size, move through the dataset with a stride that is usually equal to the context size, and create the targets by shifting each input to the right. The one-token-per-word example is only for illustration, because the actual tokenizer is a byte pair encoder. After this split, there are many input batches and target batches. In the example batch, the input has two samples, the target has two samples, and each sample contains four tokens.

> **Note.** The course's reference code differs here: The reference code example uses a 2×6 input batch; this lecture example illustrates a smaller 2×4 input/target batch.

Once the data loader has produced batches, those batches can be sent through the model to obtain the values used in the loss. To get the LLM output, pass the input batches through the LLM architecture batch by batch. This architecture includes tokenization, embedding, positional embedding, a dropout layer, transformer blocks, another normalization layer, and an output head that produces a logit tensor. In the example, the logit tensor has shape 2 by 4 by 50257, corresponding to four tokens and 50257 columns for each of the two samples. That tensor is then flattened by merging the first and second samples in the batch into a single tensor of shape 8 by 50257. The raw logits are not probabilities, because the values associated with one token do not add up

to one, so a softmax is needed to convert them into probability vectors. After softmax, each value in a row is the probability that the corresponding token is the next token.

These probability rows can be read as next-token predictions for progressively longer prefixes. For an input like "I", a trained model should assign the highest probability to the token for "had". For the input "I had", a trained model should assign the highest probability to the token for "always". For the input ending with "thought," a trained model should assign the highest probability to the token for "Jack". To evaluate the output against the target, collect the target token IDs and, for each probability row, extract the probability assigned to the correct target token. The goal of training is to make these probabilities as close to one as possible so that the predicted next tokens match the target tokens. When the model is not yet trained, those target-token probabilities are not close to one, so training uses cross-entropy loss through the negative log-likelihood. The optimization objective is for the loss to decrease and get as close to zero as possible.

With the data, targets, logits, and loss in place, pre-training becomes a parameter-optimization problem. All parameters in the GPT architecture are optimized so that, for the given inputs, the loss function is minimized. It tries to make the LLM outputs as close as possible to the target tensor by doing backpropagation. Once the loss function has been written down, the pre-training loop is conceptually simple because it mainly requires doing the backward pass. Running that backward pass depends on first understanding how to calculate the loss, which in turn depends on understanding logits, LLM outputs, the GPT architecture, tokenization, positional embeddings, multi-head attention, dropout, and feed-forward networks.

4.10 Backpropagation, optimizer steps, and the training loop

How does pre-training turn a loss value into actual changes in a GPT model's parameters, and how is that process organized into a runnable loop? The pre-training loop uses multiple epochs, where one epoch is one pass through the entire training set. Each epoch contains multiple batches because the training set is divided into batches. One loop iterates through each batch in the training data set, and another iterates over the number of epochs. After all batches have been processed once, the same procedure is repeated again until the specified number of epochs is finished.

The code takes a batch from the training loader and divides it into an input batch and a target batch. One input batch and one target batch are processed at a time. The inputs are passed through the GPT model to get logits, softmax is applied, and then the cross-entropy loss between the output and the target is computed. For a batch, the cross-entropy loss is computed for the entire batch and then the backward pass is run to calculate the loss gradients. In Python, those gradients are computed with the method "loss.backward()". Frameworks such as TensorFlow or PyTorch can compute the entire backward pass in essentially one line of Python.

The backward pass is the most important step because it calculates the gradients of the loss. Those loss gradients make it possible to update the model weights and parameters. More concretely, once the loss is obtained, backpropagation finds the partial derivative of the loss with respect to all the parameters in the model. Backpropagation through the GPT workflow is possible because the whole operation is differentiable, and differentiability is maintained at every step. In the model under discussion, the parameter count is described as around 161 million, and a single call to "loss.backward()" computes gradients with respect to all of them. The scale is striking because this backward pass and optimization can be done on a laptop in about five to six minutes.

> **Note.** The course's reference code differs here: Here the lecture discusses a ~161M-parameter setup; the reference later loads the 355M medium model instead of the smaller 124M version.

After gradients are available, the parameters are updated from them. A parameter update follows the rule that the new value equals the old value minus the step size multiplied by the loss gradient. During backpropagation, a gradient is computed for each parameter, and then all model weights are updated using those gradients. The code illustrates simple gradient descent for the update rule, but the actual implementation uses Adam. Even with Adam, the underlying idea is still to compute gradients of the loss with respect to the weights and then update parameter values based on those gradients. Parameters are updated in each iteration with the hope that the loss value keeps decreasing and reaches some minimum. This is often pictured as moving through a loss landscape toward a minimum where the loss is minimized.

Putting these pieces together gives the main pre-training algorithm. In each iteration, the loop finds the loss, performs the backward pass to get loss gradients, and updates the parameters based on those gradients. This repeated process is the main algorithm for pre-training a large language model. Optimizer.step() updates the parameter values based on the obtained gradient values. The code also resets the loss gradients from the previous batch to zero before repeating the same process on the next batch. With a context size of 256 and two samples in a batch, the model processes 512 tokens in that input batch. The "tokens seen" value tracks how many input tokens have been processed so far, and it accumulates as the loop moves through batches.



Figure 4.3: In each training iteration, a batch is split into inputs and targets, the model computes loss, backpropagation computes gradients, parameters are updated, and gradients are reset before the next batch.

The optimizer used here is "AdamW". AdamW is a variation of Adam that uses weight decay. Adam is described as a go-to optimizer for modern machine learning algorithms such as classification and regression because it works very well, avoids local minima, and

leads to faster convergence. With "AdamW", the learning rate and the weight decay are specified, and these are hyperparameters that need to be tuned.

Around the core training steps, the loop also measures progress and runtime. Setting the number of epochs to 10 means repeating the entire training process 10 times. With that setting, a generated sample is printed after each epoch. An evaluation frequency and evaluation iteration of five means that training loss and validation loss are printed after every five batches. The code records a start time and an end time to measure how long training takes. The main step in the training code is "loss.backward()"; the remaining parts mainly print training and validation losses every five batches, print a text sample after each epoch, and track tokens seen. Because "loss.backward()" calculates the loss gradients and then the code calls "optimizer.step()," the training loop is only about 15 to 20 lines of code.

> **Note.** The course's reference code differs here: The reference's classifier-training variant reports accuracy after each epoch; the lecture here describes the earlier text-generation training loop that prints a sample each epoch.

4.11 Evaluation mode, monitoring progress, and overfitting

How can a training loop do more than update parameters and also show whether the model is actually improving or merely memorizing? After the parameters are updated, the loop prints the training and validation losses, inspects the LLM output, moves to the next batch, resets the previous batch's gradients to zero, and repeats this process for all batches and then for multiple epochs. An "evaluate_model" function is used during training to compute the training loss and validation loss and print them as training proceeds. This function uses the previously defined loss-calculation functions to compute loss for the training loader and the validation loader. The losses are not printed after every batch. They are shown only at a chosen evaluation frequency, such as every five batches when the global step is a multiple of five. When evaluating, the model is put in evaluation mode with dropout disabled, because evaluation only calculates loss through a forward pass.

Loss values help, but generated text makes progress easier to see. A generate_and_print_sample function is used after each epoch as a convenience function for tracking whether the model improves during training by showing generated text. It prints 50 newly predicted tokens so the output can be inspected for whether it makes sense. The function takes a starting text snippet, called start context, converts it into token IDs, and feeds it to the LLM to generate a text sample at the current point in training. The initial prompt is "every effort moves you," and an initial text is required so the function can print what the LLM predicts next. The loop also tracks tokens seen, meaning the number of tokens consumed from input batches up to that point. With a dataset of around 5,000 tokens and 10 epochs, the total tokens seen should be on the

order of 50,000. In the run shown, training completed in 6.6 minutes on a laptop while optimizing about 160 million parameters for 10 epochs.

The printed losses and samples reveal both learning and overfitting. The reported training loss starts at about 9.781 and is later reported as 391. The validation loss started around 9.933 and then stagnated around 6.4 to 6.2, which is presented as a classic sign of overfitting. After the first epoch, the generated output was mostly commas, showing that the model had not yet learned meaningful text generation. After the second epoch, the output was still incoherent, repeating tokens such as commas and "and." As training continued, the model began to use words from the training text in its generated samples. By later epochs, the generated text made sense but directly recycled text from the training data, which is another sign of overfitting. The model's language skills improved during training, progressing from repeated commas to grammatically correct text. Producing words that make sense instead of completely random output is evidence that training is helping the model learn.

A loss plot makes the same pattern easier to see. A plot shows that the blue training-loss line continuously decreases, while the validation loss decreases and then remains stagnant. The validation loss decreases during training but never becomes as small as the training loss, and it stagnates at 6.37 after the 10th epoch. The plot also tracks tokens seen, and as the number of tokens seen increases, the training loss decreases a lot while the validation loss stagnates. Both training and validation loss improve after the first epoch, but they begin to diverge after the second epoch. When the validation loss is much larger than the training loss and the two losses diverge, the model is overfitting to the training data. The model is memorizing the training data, as shown by generated text that exactly matches phrases in the dataset, and this memorization is a classic sign of overfitting.

> **Note.** The course's reference code differs here: The reference says slight overfitting appears around 1 epoch; here the plotted losses are described as diverging after the second epoch.

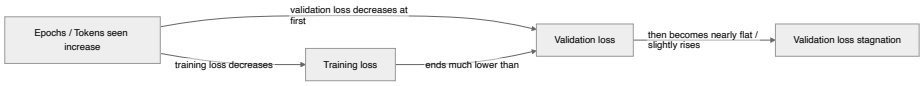


Figure 4.4: A line plot of loss versus epochs, with a top axis for tokens seen. The training loss steadily decreases, while the validation loss decreases early and then stays nearly flat with a slight rise. Redrawn from the board in L28 38:47.

Why is overfitting so pronounced here? This memorization is expected because the training dataset is very small and the model is trained for multiple epochs. Common practice is to train on a much larger dataset for only one epoch. The dataset used here has only 5,000 tokens and 20,000 characters, whereas such large language models are usually trained on datasets consisting of millions of tokens. Large datasets reduce

overfitting because they contain much more variability, whereas a small dataset lets the model get away with memorizing pieces of data. It is useful to vary hyperparameters such as learning rate, weight decay, and number of epochs to see how the code changes. Even with overfitting, the loop successfully minimizes the loss and shows that the large language model is learning.

4.12 Decoding logits into text: greedy, temperature, and top-k sampling

How should a GPT model turn its next-token scores into an actual token ID, and why is the choice of decoding rule so important for the text it produces? A simple decoding strategy selects the generated token with the largest probability score among all tokens in the vocabulary. To predict the next token in this strategy, the model output is examined for the index with the maximum probability score, and that token is chosen as the next token. Choosing the next token this way is deterministic. That determinism motivates a probabilistic alternative. Instead of choosing the maximum-probability token, the logit tensor can be passed through softmax to obtain probabilities, and then the next token can be sampled with a multinomial distribution. Multinomial sampling selects the next token proportionally to its probability score. As a result, the token with the largest probability is still favored, but other tokens can also sometimes be selected according to their probabilities. This probabilistic choice enables more creativity in the model output and helps control randomness.

Sampling becomes more useful once the logits are adjusted before softmax. Temperature scaling divides the logits by a temperature value and then applies softmax. When the temperature is increased, the next-token probabilities become more uniform, whereas a very low temperature produces a sharper distribution that favors one token more selectively. An animation is used to show this change from a sharp distribution at low temperature to a more uniform distribution at high temperature. Higher temperature values therefore lead to more uniformly distributed next-token probabilities and more diverse outputs. This can explore less likely but potentially more interesting and creative paths, but it can also lead to grammatically incorrect or completely nonsensical output. A temperature that is too low gives a sharper distribution, while a temperature that is too high can produce nonsensical output. The weakness of temperature scaling by itself is that all tokens remain possible next tokens, including completely random ones.

Top-k sampling keeps only the top k most likely tokens and excludes all other tokens from consideration. The filtering is done before softmax by keeping only the top k logits and replacing all other logits with negative infinity. When the non-top-k logits are replaced with negative infinity, softmax makes their probabilities effectively zero, so only the surviving top-k tokens can be selected. This removes tokens that do not make sense from the next-token candidates. In PyTorch, “torch.topk” applied to the next-token logits returns both the top k values and the indices where they occur. The code then uses

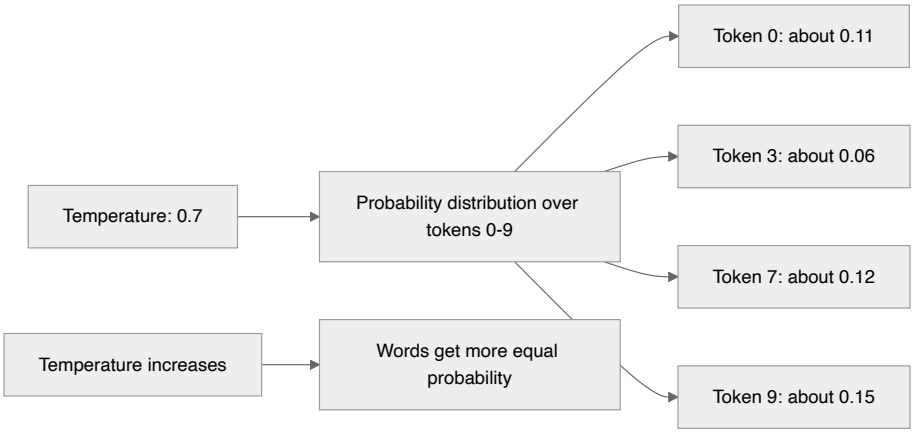


Figure 4.5: A bar chart titled "Temperature: 0.7" with tokens 0 through 9 on the x-axis and probabilities on the y-axis; the visible caption says each word gets equal probability as the temperature increases. Redrawn from the board in L30 4:28.

"torch.where" to replace logits below the smallest top-k logit with negative infinity, and after softmax only the top-k tokens retain nonzero probabilities that sum to one.

With that in place, the full decoding workflow is straightforward. The combined procedure is to obtain the logits from the model, keep only the top-k logits, replace the others with negative infinity, divide by temperature, apply softmax, and sample from the multinomial distribution. After top-k filtering and temperature scaling, multinomial sampling still favors the highest-probability surviving token most often, but the other surviving tokens can also be sampled according to their probabilities. This combination preserves some creativity through temperature scaling, blocks random tokens through top-k filtering, and makes token selection probabilistic rather than deterministic through multinomial sampling. Top-k sampling is used specifically because it addresses the temperature-scaling problem in which random low-probability tokens can still become the next token. Restricting sampling to only the top k most likely tokens improves the meaning of generated tokens and helps avoid nonsensical or random output.

The final generation function packages this decoding logic into the token-generation loop. A final function is written to generate the next token by combining temperature scaling and top-k sampling. This function takes the model, the maximum number of new tokens, the context size, the temperature value, the top-k value, and an end-of-sample ID as inputs. Inside the function, logits are obtained from the GPT output and then top-k filtering is applied, keeping only the top-k values in the logit tensor and replacing the rest with negative infinity. If the temperature is greater than zero, the logits are scaled by temperature, passed through softmax, and the next token ID is sampled from the resulting multinomial distribution. If the temperature is not specified, the next token

is chosen by taking the token ID with the maximum probability instead of sampling. "EOS" stands for end of sequence. If an end-of-sequence token is encountered, generation can stop early; otherwise each new token ID is appended to the current IDs until the maximum number of new tokens is reached. The flow that is shown is: start from the logit tensor, apply top-k sampling, scale the logits with temperature, sample the next token from a multinomial distribution, or use argmax otherwise, and keep appending tokens until the new-token limit is reached. The "generate" function now in use implements this token-generation process.

A concrete example shows how these choices change the produced text. The new "generate" function is called with the GPT model, the input token IDs for "every effort moves you", a maximum of 15 new tokens, top-k set to 25, and temperature set to 1.4. A temperature of 1.4 is presented as a good trade-off. The output produced by this new generation function differs from the earlier output produced by greedy decoding.

4.13 Interpreting decoding behavior and a gap in the course narrative

Why combine top-k sampling with temperature scaling when decoding a GPT model's next token? Top-k sampling is a decoding strategy for large language models, and it is used along with temperature scaling. Temperature scaling helps make the distribution sharper or more uniform when sampling from a multinomial distribution. Top-k sampling is paired with it because, after temperature scaling, random tokens can still have an opportunity to become the next token, whereas top-k sampling restricts sampling to the top-k most likely tokens. In the stated workflow, the model first produces a logits tensor, then top-k sampling is applied, tokens outside the top k are replaced with negative infinity, temperature scaling is applied to the logits, softmax is applied, and the next token is finally sampled from the multinomial distribution. That workflow is presented as a way to reduce or minimize overfitting.

The decoding behavior is then interpreted in terms of overfitting. An earlier deterministic prediction setup is described as causing an overfitting problem, and the newer decoding strategy is described as solving that problem. Under deterministic prediction, the next tokens are just taken from the available text, which is overfitting. Directly taking some next tokens from the source text is identified as a classic sign of overfitting. By contrast, the implemented decoding strategies are described as probabilistic in nature, and that probabilistic sampling is presented as the reason the GPT model will not memorize the passage when predicting the next words. The same idea is related to ChatGPT: it is said to give new output rather than memorizing what was written, because of decoding strategies such as top-k sampling and temperature scaling.

The example output is used as evidence for that interpretation. The generated continuation includes words such as "moves us stand to work on surprise," and these words

are said not to be present in the original text. A search of the original text for "moves you" and for "stand to work" is reported as finding that those phrases are not there. From this, the generated next tokens are described as completely new, with the model generating new tokens and new sentences instead of just memorizing the original text.

Key takeaways

- The chapter builds a complete working GPT-2 style model by combining the previously implemented parts into one system that tokenizes text, forms embeddings, runs transformer blocks, applies final normalization, and produces logits.
- GPT is trained for next-token prediction, but before training its predictions can look random, so training is needed to make the predicted next token make sense.
- The model uses a 50,257-token vocabulary and, in this implementation, keeps the original GPT-2 settings such as 768-dimensional embeddings, 12 attention heads, and 12 transformer layers while reducing the context length to 256 to make local training easier.
- Input tokens are arranged in batches, converted to token IDs, mapped to learned 768-dimensional token embeddings, combined with 768-dimensional positional embeddings, and added together to form the input embeddings.
- Inside the Transformer block, dropout and layer normalization are used around masked multi-head attention, shortcut connections, and a feed-forward expansion-contraction network, while the token dimension stays at 768 throughout the block.
- The final output head changes each token representation from size 768 to vocabulary size 50,257, so the model returns logits with shape batch size by number of tokens by vocabulary size.
- For pretraining, the data is split into training and validation sets, input-target pairs are made by shifting the input one position, cross-entropy loss is computed between logits and targets, and training proceeds over batches and epochs with loss.backward and optimizer.step.

5 Using Pretrained GPT-2 and Fine-Tuning for Classification

Based on L32: Loading pre-trained weights from OpenAI GPT-2; L34: Dataloaders in LLM Classification Finetuning | Python Coding | Hands on LLM project; L36: Coding a fine-tuned LLM spam classification model | From Scratch.

In the previous chapter, you built and pretrained a GPT-style model and used it for next-token generation; here the focus shifts from training a model from scratch to loading released GPT-2 weights into that same architecture and then adapting it for a supervised classification task. That shift matters because you load the publicly released OpenAI GPT-2 weights into the GPT model built earlier, and pretraining is not the end of working with a language model because those pretrained weights can be fine-tuned for specific applications such as email classification.

We begin with why these pretrained weights are useful, what assumptions they bring with them, and which extra tools are needed to work with OpenAI's TensorFlow-formatted release from a PyTorch-based codebase. From there, the chapter walks through the released GPT-2 files, the settings and tokenization metadata, the structure of the parameter dictionary, and how those weights map onto the model class you already have. The second half turns to classification: preparing the spam dataset, handling tokenization and padding, building dataloaders, attaching a classification head, checking accuracy and loss before training, running the fine-tuning loop, reviewing the results and overfitting checks, and finally using and saving the fine-tuned model.

At a glance

- The same GPT architecture and model class built earlier can be reused with publicly released GPT-2 weights.
- OpenAI saved the GPT-2 weights in TensorFlow, while the surrounding code uses PyTorch, so this setup installs TensorFlow and "tqdm", with TensorFlow version at least 2.15 and "tqdm" version at least 4.66.
- The main loading function in "gpt_download3.py" downloads seven GPT-2 files, locates the latest checkpoint, loads the checkpoint parameters into a dictionary named "params", and returns that along with a "settings" dictionary.
- Among the released files, "encoder.json" maps tokens to token IDs, the GPT-2 vocabulary has 50,257 tokens, and "hparams.json" stores settings such as vocabulary size, context length, embedding dimension, attention heads, and

transformer blocks.
- For the fine-tuning project, the model takes email text as input and predicts whether the message is spam or not spam, and fine-tuning means starting from pretrained GPT-2 weights and training the model again on task-specific data.

5.1 Why load GPT-2 weights and what prerequisites are assumed

Why bring in pretrained GPT-2 weights at all, instead of continuing with the model trained from scratch? The text generated by the earlier training loop was not very coherent and was not making much sense. That result is tied to the setup used there: the model is trained on one small book, which is described as a very small data set, and the later output is then examined. Even after temperature scaling and top-k sampling were introduced to reduce overfitting, the generated next sentence still did not really make too much sense. A later comparison makes the same point clearly: without GPT-2 weights, the model produced output that did not make any sense at all.

The model uses pretrained OpenAI weights loaded into the GPT model so that text generation is coherent. The exact GPT architecture built earlier is reused, and the same GPT model class is integrated with the GPT-2 OpenAI weights that were publicly released. For example, without GPT-2 weights, the output did not make sense, and after loading the weights, the generated text is coherent enough that the words make sense. There is one practical prerequisite gap to keep in mind when loading those released files. OpenAI originally saved the GPT-2 weights with TensorFlow, whereas the code used here is in PyTorch.

Work with a language model does not end at pre-training, because after obtaining a pretrained model you can use the pretrained weights for specific applications by fine-tuning them. Examples of such application-specific work include building a text classifier or an educational quiz app. The same GPT model built from scratch is kept for those application-oriented projects rather than switching to some other GPT model. Fine-tuning means adapting a pretrained model to a specific task by training the model on additional data. In the usage emphasized here, fine-tuning starts from pretrained GPT-2 weights and then trains the model again on a specific data set so that it performs well on that data set. Two kinds of fine-tuning are distinguished here: instruction fine-tuning and classification fine-tuning.

Now consider a hands-on project based on classification fine-tuning. The classification fine-tuning project uses a large language model for email classification. The specific task is to look at emails and classify them into one of two categories: spam or not spam. In that setup, the model takes the input text and the desired output is whether the email is spam or not spam.

5.2 Installing tools and downloading released GPT-2 files

How do the released GPT-2 files become something a PyTorch-based setup can actually use, and what needs to be installed first? Because the GPT-2 weights were originally saved via TensorFlow while the code used so far is in PyTorch, this setup installs TensorFlow and the "tqdm" library. TensorFlow should be version 2.15 or higher, and "tqdm" should be version 4.66 or higher. After importing both libraries, the code prints the installed TensorFlow version and the installed "tqdm" version on the system. The released GPT-2 package consists of seven files totaling about 500 megabytes. Before the weights can be integrated with the GPT architecture, a number of preprocessing steps are still required.

The released models also come in several sizes, which determines which set of files and settings you load. OpenAI made GPT-2 weights public for models with 124 million, 355 million, 774 million, and 1558 million parameters. The settings shown here correspond to the smallest GPT-2 model. For simplicity, the model discussed here is the 124 million parameter version rather than the larger 355 million, 774 million, or 1.5 billion parameter releases. This 124 million parameter model has 12 transformer blocks. As GPT-2 models become more complex, the number of transformer blocks increases. The same loading code can also be run for the larger GPT-2 architectures.

To handle those files, the loader is organized around one main function and a small set of helpers. The file "gpt_download3.py" contains three functions: the main function "download_and_load_gpt2" and the helper functions "download_file" and "load_gpt2_params_from_tf_checkpoint". The main function performs two jobs: it first downloads the seven GPT-2 files to the local computer, and then it calls the TensorFlow checkpoint-loading function. That second step stores the extracted parameters in a dictionary called "params". In the download step, the code specifies a URL, calls the download function, fetches the seven files, and saves them on the local machine. The "checkpoint" file defines the model checkpoint path, meaning the path where the current GPT-2 parameters and weights are stored. After downloading, the code uses TensorFlow's latest_checkpoint utility to search the model directory, find the checkpoint entry, and determine where the model parameters are stored. The purpose of "load_gpt2_params_from_tf_checkpoint" is to extract parameter values from the TensorFlow checkpoint and convert them into the "params" dictionary.

Once the loader exists in a Python file, using it from a notebook is straightforward. The download_and_load_gpt2 function is imported from the Python file and then run. Running it requires two arguments: the model size and the directory. The model size can also be set to 355 million, 774 million, or 1558 million, and experimentation with these options is encouraged. After the function runs, it returns both the "settings" dictionary and the "params" dictionary. This download-and-load stage is the most time-consuming part, taking about 5 to 10 minutes for roughly 500 megabytes in the example, and better internet connectivity helps the process complete smoothly.

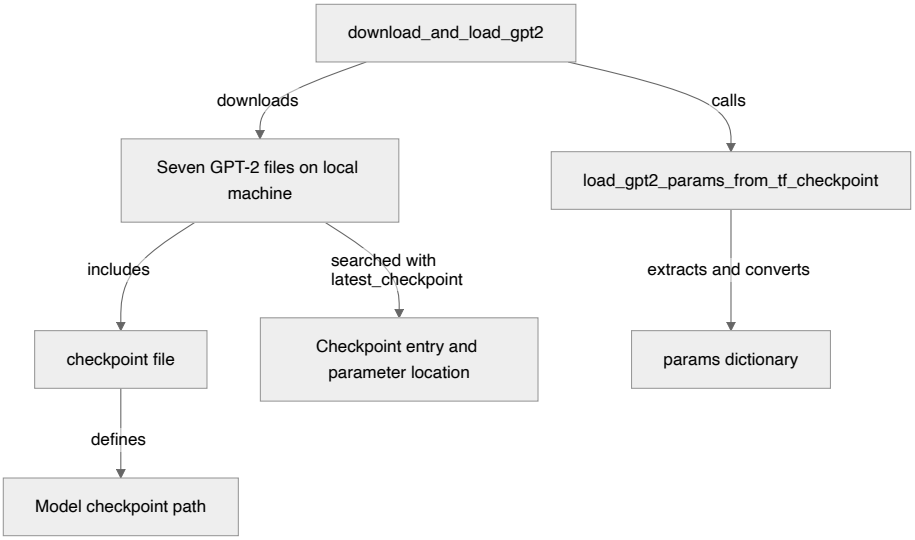


Figure 5.1: The main loading function downloads the released GPT-2 files, finds the checkpoint, and extracts TensorFlow parameters into the params dictionary.

Listing 05.02.1: This code imports the download_and_load_gpt2 function from the local gpt_download module. Source: rasbt/LLMs-from-scratch, `ch05/01_main-chapter-code/ch05.ipynb`, Apache-2.0, commit cdbd33e.

```
# Relative import from the gpt_download.py contained in this folder

from gpt_download import download_and_load_gpt2
# Alternatively:
# from llms_from_scratch.ch05 import download_and_load_gpt2
```

Not executed: error in cell 131.

5.3 Understanding GPT-2 release files, settings, and tokenization metadata

How do the released GPT-2 files tell you what tokens the model knows and what settings the model expects? The file "encoder.json" is the vocabulary: it maps tokens to token IDs. That vocabulary has 50,257 tokens. Its token IDs start at 0, and the end-of-text token is 50,256. The second file is BPE, a subword tokenization scheme. In this scheme, the encoder looks for pairs of tokens that occur most frequently, merges a pair, and then treats that merged pair as a token. BPE gives a list of merged tokens, with those of highest probability appearing at the top of the list. A special marker shown as "g-dot" is

an OpenAI convention that indicates one token has ended and a new token has begun. You can think of that marker as a space that marks the beginning of a new token.

The model settings file contains the model settings, including vocabulary size, context length, embedding dimension, number of attention heads, and number of transformer blocks. For the settings shown here, the vocabulary size is 50,257, the context length is 1,024, and the embedding dimension is 768. Each token ID is converted into a vector in a 768-dimensional space. The parameter "n_heads" is the number of attention heads in each transformer block, and "n_layer" is the number of transformer blocks. The values being shown are for the smallest GPT-2 model.

The code maintains a dictionary called "settings". This settings dictionary contains the information from hparams.json, including the vocabulary size, context length, embedding dimension, number of attention heads, and number of transformer blocks. If you inspect that dictionary, you see keys such as "n_vocab", "n_ctx", "n_embed", "n_head", and "n_layer". Those keys are the "hparams.json" information converted into a dictionary. Downloading the data from Kaggle is not enough, because it is also necessary to understand the rest of the code and the format in which the returned params dictionary is stored.

5.4 The parameter dictionary and how GPT-2 weights map onto the architecture

How do the released GPT-2 weights become a parameter dictionary that matches the model architecture, and why does that mapping matter before assigning weights into a scratch implementation? The loading code takes the TensorFlow checkpoint path together with the settings dictionary and uses them to load parameters into a special dictionary called "params". When this function finishes, it returns two things: the settings dictionary and the "params" dictionary. After execution, the parameter dictionary contains five keys: "wte", "WP", "blocks", the final normalization scale, and the final normalization shift. The printed keys are described as "blocks", "b", "g", "wpe", and "wte", which are the same five components in dictionary form. This correspondence is important because wherever the model has trainable parameters, the parameter dictionary includes matching keys.

The first two keys correspond to the token and positional embeddings used for GPT-2 inputs. Input token IDs are first converted into token embeddings using trainable parameters, then positional embeddings, which are also trainable parameters, are added to form the input embeddings passed to the Transformer block. The token embedding matrix has shape 50257 by 768 because the vocabulary size is 50257 and each token has a 768-dimensional vector. For the smallest GPT-2 model considered here, the positional embedding matrix has one embedding for each of 1024 positions, and each embedding also has dimension 768. Token embeddings and positional embeddings must have the

same vector dimension so they can be added, which is why both use 768 here. The WT key stores the token embedding matrix values, and the wp key stores the positional embedding matrix values. The dropout layer has no trainable weights, so it has no corresponding parameter-dictionary keys.

Within a Transformer block, trainable weights appear in the multi-head attention layer, the feed-forward neural network, and layer normalization. The multi-head attention layer contains trainable query, key, and value weight matrices. The feed-forward neural network has an input, a hidden layer with four times the input dimension, and a final output layer that projects back to the original size. In that feed-forward network, the first layer is the fully connected layer, and the second is an output projection layer called "P", and both have trainable weights. Layer normalization here includes a trainable scale parameter and a trainable shift parameter in addition to subtracting the mean and dividing by the square root of the variance. These trainable scale and shift parameters are said to make a big difference. Layer normalization 1, layer normalization 2, and the final normalization layer all belong to the trainable-parameter category. The top-level "blocks" key corresponds to all trainable weights and parameters inside the Transformer block, while top-level "g" and "b" correspond to the trainable parameters in the final layer normalization.

Inside "blocks", the released GPT-2 checkpoint uses a repeated nested structure. GPT-2 has 12 Transformer blocks, so the same parameter structure is repeated from "h0" through "h11". For attention, GPT-2 releases the query, key, and value matrices fused into one large matrix, which must later be split into separate query, key, and value trainable matrices. To access the fused query-key-value weights for the first block, follow the nested keys "blocks", "Transformer", "h0", and "ATN", then the fused query-key-value entry and its weights. The biases of the fused query, key, and value projections are handled similarly, and this is repeated for each Transformer block from h1 through h11. The feed-forward network inside each block has two layers, a fully connected layer and a projection layer, and both layers have weights and biases. For the first Transformer block, the fully connected feed-forward layer is accessed using the keys "h0" and "mlp", and its weights and biases can then be accessed. To access the projection layer in the feed-forward network, use its projection-layer key and then access its weights and biases. The attention mechanism also has an output projection layer with associated weights. To access the output projection layer weights, one goes through "transformer", then the block key ("h0" through "h11"), and then "attn" and "c_proj" for each of the 12 blocks.

Each Transformer block has layer normalization 1 and layer normalization 2, and both have trainable scale and shift parameters. To access the parameters of layer normalization 1 or layer normalization 2 inside a block, use that block's keys and then "g" for scale and "b" for shift. Within the "blocks" dictionary, the stored parameters cover attention weights, feed-forward network weights, output projection weights, and layer normalization weights, which together account for all trainable parameters inside a Transformer block. Outside the blocks, the final layer normalization has separate top-level keys, with "g"

for scale and “b” for shift. Using “params” with key “b” returns the final normalization shift parameters, and using “params” with key “g” returns the final normalization scale parameters. To access attention weights from the parameter dictionary in code, use “params”, then “blocks”, then the block number, then “atn”, then “c_atn”, and then “w”.

It begins by defining an empty “params” dictionary, creating the “blocks” key, and filling it with the attention layers, feed-forward network, and output projection parameters taken from the checkpoint and placed into the appropriate dictionary locations. The code then augments “params” with the other keys “wte”, “wpe”, “g”, and “b” from the checkpoint data. After the loading code runs, you can inspect the settings dictionary and the parameter dictionary keys instead of printing all parameter values. At this point, the GPT-2 architecture and its parameters have been loaded onto the laptop, and the parameters appear to be loaded correctly. The “wte” entry can then be inspected directly, and it is the token embedding matrix with shape 50257 by 768, matching the expected dimensions. Those token embedding values are optimized values, so each embedding dimension encodes some semantic meaning. Understanding the released weight format and how to convert it into the parameter dictionary matters for research work with large language models.

5.5 Updating the scratch GPT model and assigning pretrained weights

How does a scratch GPT model have to change before GPT-2 weights can fit into it correctly? The earlier “GPT config 124 million” matched GPT-2 almost exactly, except that it used a context size of 256 while the actual GPT-2 context size is 1024. To match GPT-2, the configuration is updated by setting the context length to 1024 and setting the query-key-value bias to true. Bias vectors are not commonly used in LLMs anymore because they do not improve modeling performance much and are not very necessary, but they are enabled here so the settings match the pretrained weights consistently. With that updated configuration, the next step is to create an instance of the GPT model class.

At this point, the GPT model class still initializes the token embedding matrices, the positional embedding matrices, and the transformer block weights to random values. Those random initializations need to be replaced by the downloaded GPT-2 parameters stored in the “params” dictionary. To do that, use the helper function “assign”. The helper function “assign” checks whether the shapes of its left and right values match. If the shapes match, “assign” returns the right value, effectively assigning the left side from the right side. If the shapes do not match, it raises an error, which indicates that the GPT-2 weights are not being loaded or assigned correctly. The main loading routine, “load_weights_into_gpt”, takes a GPT model instance and the “params” dictionary containing the GPT-2 weights, then assigns those weights into the GPT model.

Next, load the downloaded GPT-2 weights into each transformer block. To integrate the GPT-2 weights, the downloaded attention parameters are assigned to each transformer's attention object, and the downloaded feed-forward parameters are assigned to its feed-forward object. The downloaded attention weights are accessed through the "params" dictionary under "blocks", then the transformer attention subkeys. Because those weights are a fusion of queries, keys, and values, they are split along the columns to obtain separate query, key, and value weight matrices. After that split, the separate query, key, and value matrices are assigned to the attention object's query, key, and value weights. The attention bias terms are updated in the same way, except that the bias entries are used instead of the weight entries. The attention output projection weights and biases are accessed from the GPT-2 parameters and assigned to the attention object's output projection layer.

The feed-forward network has two layers: a fully connected layer and a projection layer. For the fully connected layer, the weights and biases are taken from the downloaded GPT-2 values and assigned to the feed-forward object. The same is done for the second feed-forward layer, the projection or output layer of the multilayer perceptron. The same pattern applies to the normalization layers. Each transformer block also contains two layer normalizations, and each one has both scale and shift parameters. For both layer normalizations in the transformer block, the downloaded scale and shift parameters are assigned to the corresponding parts of the GPT model. Outside the transformer block, the final normalization layer is loaded from the "g" and "b" entries, which are assigned to the model's final normalization scale and shift values.

One remaining question is the final output head. GPT-2 uses weight tying, meaning that the token embedding weights are reused for the final output head instead of defining new weights for that layer. Weight tying is not used too much these days, but it was used in the GPT-2 architecture. Using weight tying brings the total parameter count from about 162 million to 124 million; without weight tying, it is about 164 million. Once the GPT model instance and the "params" dictionary have been created, calling "load_weights_into_gpt" loads the downloaded GPT-2 parameters into the model instance. After that call, the GPT model built from scratch is fully ready and functional for testing.

5.6 Verifying generation and identifying what can be changed

How can you tell whether a scratch-built GPT is really working after pretrained weights are assigned, and what becomes possible once it does? A simple check is to call the "generate" function with the loaded model, using input token IDs for the prompt "every effort moves you", a maximum of 25 new tokens, temperature 1.5, and top-k 50. With the GPT-2 weights loaded, generation does not take much time because the model already has pretrained weights. The generated continuation is much more coherent than the earlier output produced without GPT-2 weights. The weights are treated as loaded

correctly because the model produces coherent text and at least the words make sense. Even a tiny mistake in this process would cause the model to fail.

Once the model generates sensible text, you can start probing how its sampling behavior changes. Increasing the temperature increases the entropy, and the output should be a bit random. For example, setting the temperature to 10 is used specifically to see the effect of a much higher temperature value. At that setting, the text is expected to become a bit random, and the resulting output is described as random. Having an in-house GPT implementation opens the door to experimentation with settings such as temperature, as well as to creativity, hyperparameter tuning, and research.

That freedom also extends to the model itself. To change the architecture, you go to the defined GPT model and add or subtract layers. One concrete change is to reduce the number of transformer blocks if you want to test a smaller language model instead of using 12 transformer blocks. Because the implementation works on a local computer and runs fast, it allows iteration and testing. The parameters named for experimentation include top-k, maximum new tokens, the number of attention heads, the number of transformer blocks, the optimizer size, and the optimizer step size. The optimizer used here is Adam with a learning rate of 5 into 10 to the minus 4 and weight decay 0.1, and those settings can also be changed to check their effect on the output.

Listing 05.06.1: This code seeds PyTorch, calls generate on the GPT model with the prompt "Every effort moves you" using 25 new tokens, top-k 50, and temperature 1.5, and prints the decoded output text. Source: rasbt/LLMs-from-scratch, `ch05/01_main-chapter-code/ch05.ipynb`, Apache-2.0, commit cdbd33e.

```
torch.manual_seed(123)

token_ids = generate(
    model=gpt,
    idx=text_to_token_ids("Every effort moves you", tokenizer).to(device),
    max_new_tokens=25,
    context_size=NEW_CONFIG["context_length"],
    top_k=50,
    temperature=1.5
)

print("Output text:\n", token_ids_to_text(token_ids, tokenizer))
```

Not executed: error in cell 145.

5.7 Preparing the spam-classification dataset

What does the spam-classification dataset look like before any model can use it, and why is that preparation necessary? The dataset is downloaded from a machine learning repository. It is an SMS spam collection dataset made of text messages that are labeled as either spam or not spam. Each entry consists of a text together with a label indicating whether it is ham, meaning no spam, or spam. In this dataset, non-spam is also called ham.

One description gives the dataset as around 747 spam emails and more than 3,000 non-spam emails. Another count states that there were 4,825 no-spam entries and 747 spam entries, and also summarizes this as around 4,800 no-spam messages and 747 spam messages. To balance the classes, 747 non-spam entries were randomly sampled so that spam and non-spam each had 747 examples. After balancing, the dataset contains 747 spam examples and 747 non-spam examples, for a total of 1,494 examples.

The labels are then encoded numerically. Ham is encoded as 0 and spam is encoded as 1. Accordingly, the target tensor contains zeros and ones, where zero means no spam and one means spam. With the labels encoded, the full dataset is split into training, validation, and testing data. The split uses 70% for training, 10% for validation, and 20% for testing, and these splits were converted into CSV files.

The dataloaders batch the dataset into inputs and targets. The same split proportions are also described as 70% training, 20% testing, and 10% validation. Because the balanced dataset has 1,494 examples, taking 70% for training gives 1,045 training samples. Because the balanced dataset has 1,494 examples, taking 70% for training gives 1,045 training samples, with batches corresponding to that split. Next, look at the token IDs, padding, and batch shapes.

5.8 Tokenization, padding, truncation, and dataset design for classification

How do raw email texts become the uniform input-target pairs that a classifier can batch and train on? The purpose of the data loader is to manage input-target pairs more effectively. In PyTorch, “Dataset” and “DataLoader” can be used with a custom dataset to give easy access to samples, run batches on the data, and even do parallel processing. The goal is to convert each email into tokens and produce an input tensor together with a target tensor in which each email is labeled 0 or 1. A key difficulty is that email texts have varying lengths, while batching requires every row in the batch to have the same number of columns. One possible way to equalize lengths is to find the shortest email and truncate every longer email to that length. That approach is not recommended because information from the longer texts would be lost.

An alternative approach is to use the longest email length and pad every shorter email with additional tokens until it reaches that length. The first preprocessing step is to tokenize each text message. The tokenizer used here is the byte pair encoder tokenizer that GPT models use. The "tiktoken" library is the tokenizer library used by OpenAI, and it converts a sentence into token IDs. Byte pair encoding is a subword tokenizer, so a word is not necessarily equal to one token. After tokenization, shorter token sequences are padded with token "50256" until all sequences have the same length. In the GPT-2 vocabulary, token "50256" is the end-of-text token. A vocabulary maps tokens to token IDs. It is helpful to think of the vocabulary like a dictionary: instead of words with meanings, it provides words with associated token IDs. The end-of-text token signifies that the end of a document or sentence has been reached, and it is used during GPT training to distinguish between separate document sources. Although any token could be used for padding, using the end-of-text token makes symbolic sense because it indicates that there is no additional text and avoids confusing the model with a random word.

With those pieces in place, the preprocessing workflow becomes straightforward. The overall workflow is to take the train, validation, and test CSV text, tokenize it into token IDs, find the longest email, and pad all messages with end-of-text token "50256" to that same length. Before a "DataLoader" can be instantiated, a PyTorch "Dataset" must be implemented to specify how the data is loaded and processed. The dataset defines how data is loaded and what the input-target pairs are, and only after that is defined is the data loader instantiated. The "SpamDataset" class identifies the longest sequence in the training set, converts each text message into token IDs, and pads the other sequences to match the longest sequence. To create an instance of "SpamDataset", you specify the CSV file and the tokenizer being used. The CSV file is read with pandas into a data object, and then each email or text message in that data is converted into token IDs. If no maximum length is specified, the maximum length is taken from the longest encoded text in the dataset. The "longest_encoded_length" function finds the lengths of all text messages and returns the maximum length among them. If the user specifies a maximum length, that user-defined value is used instead.

A user-defined maximum length introduces one more preprocessing step. If some sequences are longer than the maximum length, they must be truncated. If the maximum length is selected from the data itself, this problem will not arise. The pad token ID is "50256", and texts shorter than the maximum length are padded with this token ID. For each text message, pad token IDs are appended until the message reaches the maximum length, so all text messages have the same size. The dataset creates two tensors named "encoded" and "label". The "encoded" tensor is the tensor of token-ID encodings for the text messages. The "label" tensor contains the labels for the examples, and those labels are zeros or ones. The dataset's main function is "getitem", and calling it converts the dataset into the "encoded" tensor and the labeled tensor. The "encoded" tensor ensures that every sentence has been converted into token IDs and that all sentences have equal length so they can be batched together. For this classification setup, the input-target pairs differ from language-model training because the targets are zeros and ones rather than token IDs shifted by one position.

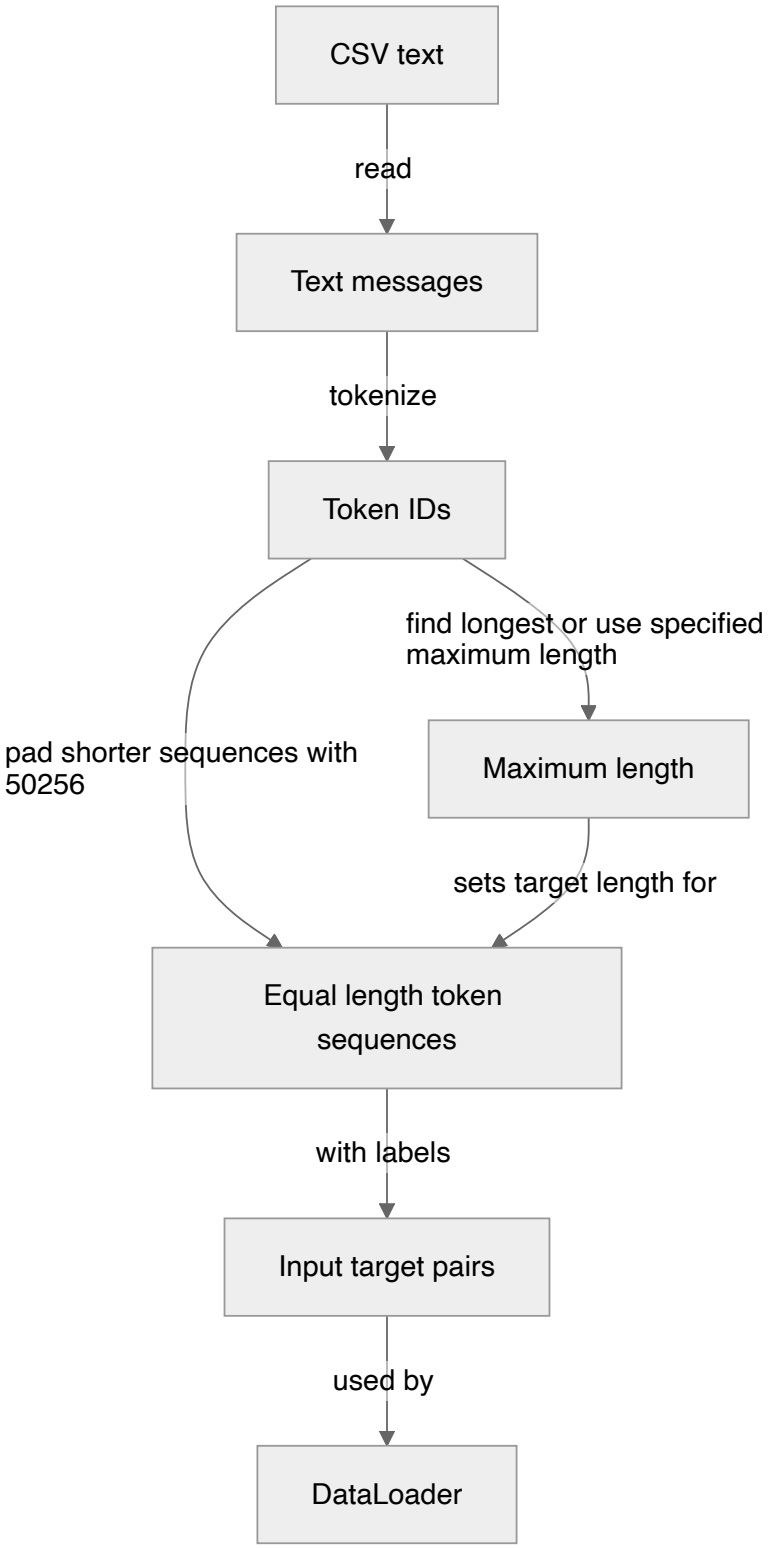


Figure 5.2: The dataset workflow reads CSV text, tokenizes messages, determines a maximum length, and pads shorter sequences with the end-of-text token so inputs can be batched with labels.

The same dataset design is then applied across the train, validation, and test splits. The "SpamDataset" class loads data from CSV files, tokenizes the text using the GPT-2 tokenizer from "tiktoken", and pads or truncates sequences to a uniform length defined either by the longest sequence or by a predefined maximum length. An instance of SpamDataset is created from the train.csv file, and if no maximum length is passed, that maximum length is computed from the dataset itself. In this example, the computed maximum length is "120". GPT-2 can handle sequences up to "1024" tokens because its context length is "1024", so the maximum length passed here can be at most "1024". The validation and test datasets are padded to match the length of the longest training sequence. Validation and test samples that exceed the length of the longest training examples are truncated. When creating the validation and test dataset instances, the maximum length is passed in as "120", the maximum length from the training dataset. When a maximum length has been defined for validation and test data, sequences in those sets that are longer than that maximum length must be truncated. There is no requirement that the specified maximum length equal the maximum length in the training dataset, and it can also be set to none. If the user-defined maximum length "120" is passed for validation or test data, printing the maximum length shows "120". The dataset serves as the input to the data loader, so the dataset is implemented first and then passed into a data loader. Data preprocessing, data cleaning, datasets, and data loaders are usually very important before model training begins.

Listing 05.08.1: Creates a SpamDataset from train.csv with the tokenizer and no explicit maximum length, then prints the dataset's computed max_length. Source: rasbt/LLMs-from-scratch, `ch06/01_main-chapter-code/ch06.ipynb`, Apache-2.0, commit cdbd33e.

```
train_dataset = SpamDataset(
    csv_file="train.csv",
    max_length=None,
    tokenizer=tokenizer
)

print(train_dataset.max_length)
```

Output (executed):

```
120
```

- *Checked by running the code:* `train_dataset.max_length` = 120 (the text says 120)

Listing 05.08.2: Creates validation and test SpamDataset instances using the same tokenizer and the training dataset's max_length for padding and truncation. Source: rasbt/LLMs-from-scratch, `ch06/01_main-chapter-code/ch06.ipynb`, Apache-2.0, commit cdbd33e.

```
val_dataset = SpamDataset(
    csv_file="validation.csv",
    max_length=train_dataset.max_length,
    tokenizer=tokenizer
)
test_dataset = SpamDataset(
    csv_file="test.csv",
    max_length=train_dataset.max_length,
    tokenizer=tokenizer
)
```

5.9 Building and checking the data loaders

How does a prepared classification dataset become batches that a model can consume reliably and inspect easily? Once the dataset is defined, it serves as the input to a data loader. When a dataset is passed to a data loader, the loader can be configured with a batch size and a number of workers for parallel processing. Here, the batch size is set to 8 and the number of workers is set to 0. Setting the number of workers to 0 is only for simplicity, so no parallel processing is used. Setting "drop_last" to True means the last batch is dropped if it is smaller than the others. The train, validation, and test loaders are then created by creating instances of the data loader and passing the corresponding datasets as input. After initialization, these loaders make it easy to extract batches such as the first, second, or eighth batch.

A quick check confirms that the loaders return batches with the intended structure. To test the training loader, iterate through it and print the input batch dimension and the label batch dimension. The input batch dimension is 8 by 120, and the label batch dimension is size 8. In the training dataloader, each input batch therefore contains eight samples and 120 token ID positions per sample. A label tensor of size 8 means there are 8 labels in the batch, and each label is either 0 or 1. The token ID 50256 is the end-of-text token, and shorter text messages were padded with it so that all messages have the same number of token IDs.

The batch counts follow directly from the dataset split and the batch size. The training dataset has 1045 samples. With batch size 8, the number of training batches is approximately 130, since 1045 divided by 8 is approximately 130. Printing the length of the training loader gives the number of training batches, which is 130 here. That count matches the earlier calculation. The validation loader has 19 batches and the test loader has 38 batches. Because 20% of the data is test and 10% is validation, the number of test batches is twice the number of validation batches. Overall, the dataloaders contain 130 training batches, 19 validation batches, and 38 test batches.

It is also useful to picture each loader in terms of its full batch structure. One batch from the training loader has shape 8 by 120, and with 130 such batches, the full training loader can be viewed as a three-dimensional tensor of shape 130 by 8 by 120. The test loader can be viewed as having dimensions 38 by 8 by 120. At this point, data preparation is complete, because the training, validation, and test loaders have been created and can provide input batches and label batches whenever they are needed.

Listing 05.09.1: This code iterates through the training loader and then prints the shapes of the last input batch and target batch. Source: rasbt/LLMs-from-scratch, `ch06/01_main-chapter-code/ch06.ipynb`, Apache-2.0, commit cdbd33e.

```
print("Train loader:")
for input_batch, target_batch in train_loader:
    pass

print("Input batch dimensions:", input_batch.shape)
print("Label batch dimensions", target_batch.shape)
```

Output (executed):

```
Train loader:
Input batch dimensions: torch.Size([8, 120])
Label batch dimensions torch.Size([8])
```

- *Checked by running the code:* `input_batch.shape[0]` = 8 (the text says 8)
- *Checked by running the code:* `input_batch.shape[1]` = 120 (the text says 120)
- *Checked by running the code:* `target_batch.shape[0]` = 8 (the text says 8)
- *Checked by running the code:* `input_batch.shape[1]` = 120 (the text says 120)
- *Checked by running the code:* `target_batch.shape[0]` = 8 (the text says 8)

5.10 Adapting GPT for classification and making predictions

How can a pretrained GPT model be turned into something that decides whether a message is spam or not, and how do its outputs become a class label? For this classification task, the original final output layer took 768 inputs, matching the embedding dimension, and produced 50,257 outputs, matching the vocabulary size. That layer is replaced with a classification head that still takes 768 inputs but produces 2 outputs, for spam or no spam. Attaching a classification head on top of the GPT architecture is an example of using an LLM for a classification task. Fine-tuning requires changing the GPT model architecture so that the model answers better and its accuracy improves.

Once the architecture is modified, a sequence input produces outputs at every token position. For example, if the input is “do you have time”, each token produces two

outputs corresponding to spam or no spam. With one sample containing four tokens, the output tensor has shape 1 x 4 x 2: one batch element, four tokens, and two outputs per token. For classification, only the output of the last token is used, because the last token contains information about the other tokens through its attention weights. For classification, the last output token produces two values.

The next step is to turn those two final values into a predicted label. The procedure is to take the last-token outputs, apply softmax to convert them into probabilities, and then choose the index with the highest value. For example, one message can produce probabilities 0.99 and 0.01 after softmax, and then the predicted class is index 0, which means no spam. Another message can produce probabilities 0.01 and 0.99 after softmax, and then the predicted class is index 1, which means spam. Applying softmax is unnecessary when only the predicted class is needed, because the index of the largest logit is the same as the index of the highest probability.

A concrete pair of logits makes the conversion clear. For a last-token output of -3.5983 and 3.9902, the code applies softmax to print the resulting probabilities and uses argmax on the last-token output to find the class label. In that example, one printed softmax value is 0.9995. Argmax returns class label 1, so the model predicts that the input text is spam.

Listing 05.10.1: This code applies softmax to the last-token outputs, takes the argmax of the resulting two-class probabilities, and prints the predicted class label. Source: rasbt/LLMs-from-scratch, `ch06/01_main-chapter-code/ch06.ipynb`, Apache-2.0, commit cdbd33e.

```
probas = torch.softmax(outputs[:, -1, :], dim=-1)
label = torch.argmax(probas)
print("Class label:", label.item())
```

Not executed: error in cell 76.

5.11 Accuracy and loss before training

How can a classifier be judged before any fine-tuning begins, and why is that worth checking now? Classification accuracy measures the percentage of correct predictions across a dataset. The output labels are the target labels, which are the true values, and accuracy is obtained by comparing predicted labels with these correct labels. To determine classification accuracy, the argmax-based prediction is applied to all examples in the dataset and then compared with the actual values. For a batch, each input is passed through the model, the two logits are taken, and argmax gives the predicted class label. At the loader level, the code passes all samples in a batch through the model, takes

the logits of the last output token, applies argmax for the whole batch, and compares the predicted labels with the target labels.

The bookkeeping is simple but essential. Whenever a prediction matches the target label, the count of correct predictions is increased by one, and every processed example increases the total example count by one. Accuracy is then computed as the number of correct predictions divided by the number of examples. The example given has 600 correct predictions, so the accuracy is 600 divided by the total number of examples. The accuracy function uses the full dataloader length as the number of batches when no batch count is specified, and otherwise uses the minimum of the requested number and the dataloader length. The code calls this accuracy function on the training, validation, and test loaders, and then prints the three accuracies.

Before backpropagation is added, strong results are not expected. Because the model has not yet been optimized and backpropagation has not yet been implemented, the training, validation, and test accuracies are not expected to be good. The initial reported performance is poor: training accuracy is 46%, validation accuracy is 45%, and test accuracy is 48%. That result is described as worse than a coin toss, since a random prediction would be right 50% of the time. To improve prediction accuracy, the model must be fine-tuned by defining a loss function from the true targets and predicted values, computing gradients of that loss with respect to the trainable weights, and updating those weights repeatedly until the loss is minimized. The update rule is weight new equals weight old minus the partial derivative of the loss with respect to that weight. A variation of simple gradient descent such as Adam or AdamW is used for optimization. A simplified picture of optimization is moving down a loss curve from a higher-loss starting point toward a global minimum. When the loss is minimized, the accuracy should also become higher automatically.

That raises the next question: what loss should be used for classification? For classification with targets Yi and predictions Pi, the loss function used here is categorical cross-entropy loss. Categorical cross-entropy is defined as the negative sum over class labels of Yi times log of Pi. For example, if the true label is not spam with one-hot encoding [1, 0] and the predicted values are [0.8, 0.2], the loss is computed by multiplying each true-value entry by the log of the corresponding predicted value and taking the negative sum. If the predicted value exactly equals the true value, the cross-entropy loss is zero. Negative Yi log Pi is presented as a good loss function for categorical predictions in classification tasks.

Its shape also explains what the model should learn to do. The graph of negative log of x is used to show that the probability of the correct class should be as close to one as possible, so that the loss becomes as close to zero as possible. Cross-entropy loss is differentiable, which makes it useful for backpropagation. Classification accuracy cannot be used directly as the loss, or inverted to form a loss, because accuracy is not differentiable. Cross-entropy loss is therefore used as a proxy to maximize accuracy, and it is the same loss used to pre-train the large language model. Given an input batch and a target batch, the model processes the input batch, the logits of the last output token

are selected, and the categorical cross-entropy loss is computed between those logits and the target batch. The code uses "torch.nn.functional.cross_entropy" to compute this loss, and the resulting loss is differentiable for backpropagation.

A "calculate loss batch" function computes the loss for one batch. A loader-level loss function handles multiple batches by choosing the number of batches in the same way as the accuracy code: it uses the dataloader length if no number is specified, otherwise the minimum of the requested number and the dataloader length. For each selected batch, it takes one input batch and one target batch, computes the categorical cross-entropy loss with "calculate loss batch", adds each batch loss into a running total, and divides by the number of batches to get the average loss per batch. Before backpropagation is implemented, these loss values are expected to be high, matching the previously observed poor accuracy. To inspect the initial losses, the code applies the loss-loader function to the training, validation, and test loaders with the number of batches set to five, and then prints the three losses. At this point, the workflow has already covered downloading and preprocessing the dataset, creating data loaders, initializing the model, loading pretrained weights, modifying the model for fine-tuning, and implementing evaluation utilities for loss and accuracy.

Listing 05.11.1: Defines a function that iterates over batches in a data loader, takes the last-token logits from the model output, uses argmax to get predicted labels, counts matches against targets, and returns accuracy over a chosen number of batches. Source: rasbt/LLMs-from-scratch, `ch06/01_main-chapter-code/ch06.ipynb`, Apache-2.0, commit cdbd33e.

```
def calc_accuracy_loader(data_loader, model, device, num_batches=None):
    model.eval()
    correct_predictions, num_examples = 0, 0

    if num_batches is None:
        num_batches = len(data_loader)
    else:
        num_batches = min(num_batches, len(data_loader))
    for i, (input_batch, target_batch) in enumerate(data_loader):
        if i < num_batches:
            input_batch, target_batch = input_batch.to(device), target_batch.to(device)

            with torch.no_grad():
                logits = model(input_batch)[:, -1, :]  # Logits of last output token
            predicted_labels = torch.argmax(logits, dim=-1)

            num_examples += predicted_labels.shape[0]
            correct_predictions += (predicted_labels == target_batch).sum().item()
        else:
```

```
            break
    return correct_predictions / num_examples
```

Listing 05.11.2: Computes average loss on the train, validation, and test loaders for five batches each with gradients disabled, then prints the three loss values. Source: rasbt/LLMs-from-scratch, `ch06/01_main-chapter-code/ch06.ipynb`, Apache-2.0, commit cdbd33e.

```
with torch.no_grad(): # Disable gradient tracking for efficiency because we are not trai
    train_loss = calc_loss_loader(train_loader, model, device, num_batches=5)
    val_loss = calc_loss_loader(val_loader, model, device, num_batches=5)
    test_loss = calc_loss_loader(test_loader, model, device, num_batches=5)

print(f"Training loss: {train_loss:.3f}")
print(f"Validation loss: {val_loss:.3f}")
print(f"Test loss: {test_loss:.3f}")
```

Not executed: error in cell 89.

5.12 Training loop for fine-tuning

How does the pretrained classifier actually get adapted to the spam dataset, and why have the losses and accuracies so far not yet reflected training on this data? Fine-tuning means implementing a training function that adjusts the parameters to minimize the training loss and also prints the validation loss and the test loss. At this point, the dataset has been downloaded and preprocessed, data loaders have been created, the model has been initialized, pretrained weights have been loaded, the model has been modified for fine-tuning, and evaluation utilities for loss and accuracy have been implemented. What has not happened yet is training on this dataset, so the parameters are not optimized for it. The next stage is to define the training loop, implement backpropagation, and use that training function to fine-tune the pretrained LLM to improve spam classification accuracy. Training is still required even when the data is supervised, such as spam and no-spam labels, so there is a training process in both pre-training and fine-tuning. The fine-tuning function is the same training function used earlier for pre-training, and it is described as very close to the earlier "train model simple" function. The main distinction is that this version tracks the number of text examples rather than the number of tokens.

The loop itself is organized around epochs and batches. One epoch means going through the entire dataset once. Within each epoch, the code has a second loop that goes within each batch. The first step in the code is to set the model to training mode. The second step is to reset the loss gradients from the previous batch before processing the current batch. The most important step is to compute the loss on the current batch, run a

backward pass to calculate the loss gradients, and then update the model weights with an optimizer step. After the weights are updated, the code prints the training loss and the validation loss, and this process repeats across multiple epochs. The backward pass is the most important step because it produces loss gradients with respect to the parameters, which is why the loss function must be differentiable. Once those gradients are available, the parameters can be updated repeatedly so that the loss is hopefully minimized.

The implementation also keeps track of progress through the data and decides when to evaluate. The code tracks the number of examples seen, where "input_batch.shape[0]" gives the number of samples in the batch. An example seen means one text message processed by the model. Whenever the model goes through a full batch, the global step is increased by one. The evaluation frequency specifies after how many processed batches the training loss and validation loss are printed. If the training data loader has 130 batches and the evaluation frequency is 50, the training and validation loss are printed about two times per epoch. The evaluation code can compute training and validation loss on only a specified number of batches, such as five or ten, to save time.

Instead of vanilla gradient descent, the optimization in practice uses a more complicated algorithm that keeps track of previous gradients and gradient squares so that optimization is done better and the model does not get stuck in local minima. AdamW is used here, and it is described as a modification of the Adam optimizer with weight decay. It is described as helping avoid local minima, converging in a smooth manner, and leading to faster convergence. The training code calls train classifier simple with the modified GPT model that has a classification head on top, together with the train loader, validation loader, optimizer, number of epochs, and evaluation frequency. In this call, the evaluation frequency is set so that training and validation loss are printed after every 50 batches, and the evaluation iterations are set to five batches to keep the calculations quick. Evaluating on only five batches is not the best evaluation method, but it still gives a useful sense of how the training loss and validation loss are progressing. On a MacBook Air 2020, the full training process took about 8.8 minutes, and on a typical i5, i7, or MacBook laptop it should take about 7 to 10 minutes. Because the optimizer choice and hyperparameters can be changed, including trying Adam and changing the learning rate and weight decay, this setup can be used to explore their effects on loss and accuracy.

> **Note.** The course's reference code differs here: Runtime varies by model and hardware: the reference reports about 5 minutes on an M3 MacBook Air for gpt2-small, while larger models and other laptops can take longer.

5.13 Results, overfitting checks, and what was trained

How can you tell whether fine-tuning actually worked, and whether the model learned the task without simply overfitting the training data? After training, the training loss reaches about 0.083, the validation loss reaches about 0.074, the training accuracy improves

to around 100%, and the validation accuracy reaches 97.5%. As the model sees more examples, both the training loss and the validation loss go down. Very low training loss and very low validation loss, together with no noticeable gap between them, indicate good training with little to no overfitting. If the validation loss is much higher than the training loss, that is a sign of overfitting. The accuracy plot follows the same pattern: as loss decreases, training accuracy rises to 1 and validation accuracy rises to about 97%.

Those plotted accuracies need one important qualification. Because the evaluation iteration was set to five, the plotted accuracy values are not representative of performance on the entire dataset. During training, the training and validation performance shown in those plots was based on only five batches for efficiency. To assess performance on the full dataset, the accuracy loader is run with the train loader, the model, and the device. The calculate_accuracy_loader function uses the model to make predictions, compares them with the actual values, reports accuracy, and does this over all batches in the training set rather than only five batches. The accuracy measure for the training, validation, and test sets is a much better representative than these plots because the plots use evaluation iterations set to five.

With that fuller evaluation, the picture changes slightly. On the full datasets, the training accuracy is 97%, the validation accuracy is 97%, and the test accuracy is 95%. A small gap where test accuracy is slightly lower than training accuracy suggests a small amount of overfitting. Validation accuracy is often somewhat higher than test accuracy because model development often involves fine-tuning parameters on the validation set. That validation-test gap could potentially be reduced by adjusting model settings such as increasing the dropout rate or the optimizer's weight decay parameter. This setup leaves room to experiment with dropout rate, learning rate, weight decay, and which parameters are unfrozen during fine-tuning. In the current setup, the trained parameters are the output classification head, the last transformer block, and the final normalization layer. One possible experiment is to train more transformer blocks instead of only the last one.

These results also highlight the trade-offs of fine-tuning. In this project, training reduced the loss and increased the accuracy. A disadvantage of fine-tuning is that it requires additional training time on a specific dataset. An advantage is that the model works very well on the target dataset, here the spam collection, and can act as a spam classifier. The same fine-tuning code can be applied to a wide range of classification tasks and datasets. Classification fine-tuning is one category of fine-tuning, and another major category is instruction fine-tuning. Instruction fine-tuning can be used to build a chatbot that answers or replies to specific instructions. This approach opens research opportunities both in changing and testing LLM architectures and in applying them to classification projects.

Listing 05.13.1: This code computes accuracy on the full training, validation, and test loaders with 'calc_accuracy_loader' and prints the three percentages. Source: rasbt/LLMs-from-scratch, `ch06/01_main-chapter-code/ch06.ipynb`, Apache-2.0, commit cdbd33e.

```
train_accuracy = calc_accuracy_loader(train_loader, model, device)
val_accuracy = calc_accuracy_loader(val_loader, model, device)
test_accuracy = calc_accuracy_loader(test_loader, model, device)

print(f"Training accuracy: {train_accuracy*100:.2f}%")
print(f"Validation accuracy: {val_accuracy*100:.2f}%")
print(f"Test accuracy: {test_accuracy*100:.2f}%")
```

Not executed: error in cell 105.

5.14 Inference on new messages and saving the fine-tuned model

How does a fine-tuned spam classifier get used on messages it has never seen before, and why is that the step that matters now? After fine-tuning on supervised data and plotting the training and validation loss, the remaining step is to use the model on new data. The real test is performance on data the model has not seen before, because the goal is to judge how well it works as a spam classifier on unseen messages. At this final stage, the fine-tuned GPT-based model is used to classify spam messages.

To make that possible, a function called "classify_review" takes any text and predicts whether it is spam or not. The first step is to convert the input text into token IDs. The tokenizer used for this is the tiktoken tokenizer, described as a byte pair encoder that takes a sentence and converts it into a bunch of tokens. The model also needs its supported context length, which in this case is 1024. That context length is read from the positional embedding weight shape: the number of rows equals the context length, and the number of columns equals the embedding dimension.

With token IDs and context length in hand, the next question is how long each input sequence should be. The working maximum length is chosen by comparing the maximum token-ID length found in the training set with the supported context length and taking the smaller of the two. If the training-set maximum length is greater than the context length, sequences are truncated to the context length. Otherwise, the maximum length is used. Ideally, this maximum length comes from the longest message in the training dataset, but if that exceeds the context length then the context length is used instead.

A new message must then be adjusted to that chosen length before it can be passed to the model. If a new message is shorter than the chosen maximum length, it is padded with end-of-text tokens until it reaches that length. The end-of-text token used for padding is 50256. Padding every input sequence to the same length is important because batching requires every sentence to have the same number of token IDs. After tokenization and padding, the input is converted into a tensor and a batch dimension is added before

inference. For inference on a new text message, the trained model is used only after making sure that the token IDs match the maximum length.

Next, consider how the model is used at inference time. During inference, the model produces logits for the last output token, and applying argmax yields the prediction of spam or not spam. For example, the message "you are a winner you have been specially selected to receive ,000 cash or $2,000 reward" is passed through "classify_review", and the model predicts spam. A second message, "hey just wanted to check if we are still on for dinner tonight let me know", is also passed through the model. The model predicts that this legitimate message is not spam, and these new text samples appear to show good performance.

The model can be saved so that it does not need to be trained again. "torch.save" saves the model parameters for later use. "torch.load" loads previously saved model parameters from a specified path, and the loaded parameters can be used for inference or for further fine-tuning. This saves time and effort. Fine-tuning is necessary here because without it the model made very bad predictions and could not correctly answer a prompt asking whether a text was spam with a yes-or-no response. The same classification fine-tuning code can also be tried on other datasets, such as a heart disease dataset.

Key takeaways

- Training a GPT model from scratch did not produce sensible text, but loading pretrained GPT-2 weights into the same architecture made generation coherent enough that the words made sense.
- Fine-tuning means starting from pretrained GPT-2 weights and training the same GPT model again on task-specific data, with classification fine-tuning and instruction fine-tuning presented as two main types.
- The classification project fine-tunes the model to classify messages as spam or not spam, using the SMS Spam Collection dataset, balancing the classes, encoding ham as 0 and spam as 1, and splitting the data into training, validation, and test sets.
- To prepare batches, the messages are tokenized with a byte-pair encoder tokenizer and padded with token 50256, the end-of-text token, so every encoded input has the same length for batching.
- For classification, the GPT output layer is replaced with a two-class head, and the prediction comes from the last token's two logits using argmax, with softmax needed only if you want probabilities.
- Before training, accuracy is poor, but fine-tuning with cross-entropy loss and backpropagation greatly improves results, reaching about 97% training accuracy, 97% validation accuracy, and 95% test accuracy on the full datasets.

6 Instruction Fine-Tuning and Evaluation

Based on L38: Data Batching in LLM instruction fine-tuning | Hands on project | Live Python coding; L40: Instruction fine-tuning: Loading pre-trained LLM weights; L42: Evaluating fine-tuned LLM using Ollama.

Pretrained LLMs are good at text completion but struggle with following specific instructions, which is why instruction fine-tuning is used to make them better at responding to instructions. If you are comfortable with PyTorch training loops, data loaders, and transfer-learning conventions from the previous chapter, you already have most of the practical setup; where this chapter adds new ideas, it will make the data shape and training target explicit before moving into evaluation.

We begin by clarifying why instruction fine-tuning is needed, what the instruction dataset contains, and how instruction, input, and output records are converted into Alpaca-style prompts. Next, we walk through tokenization, variable-length sequences, per-batch padding with the end-of-text token, shifted targets for next-token prediction, ignoring padding in the loss with "-100", and the complete batching recipe, including optional masking choices. From there, we load pretrained GPT-2 weights, map those parameters into the model, check the model before fine-tuning, and then close by evaluating a fine-tuned model with examples, response extraction, broader benchmarking ideas such as MMLU, automated judging with "Ollama" and "Llama 3", and practical directions for improving results.

At a glance

- Pretrained LLMs can complete text well, but they do not follow specific instructions well enough on their own, which is why instruction fine-tuning is needed.
- Instruction fine-tuning uses examples made of instructions and desired responses, and some examples also include an additional input field.
- The dataset used here has 1,100 instruction-input-output entries from various fields, and its purpose is to teach the model to follow instructions correctly.
- Before fine-tuning, each dataset entry is converted into an Alpaca-style prompt that combines a fixed introductory template with the instruction, the input when present, and the response from the dataset output.
- When batching tokenized prompts, shorter sequences are padded within each batch to the length of the longest sequence using token ID 50256, the end-of-text token, so all rows have the same number of token positions.

6.1 Why instruction fine-tuning is needed and what data it uses

Why is instruction fine-tuning needed when a model is already pre-trained? Pre-trained LLMs are good at text completion, but they struggle with following specific instructions. Examples of such instructions include fixing the grammar in a text and converting a text into passive voice. Because a pre-trained LLM does not follow instructions very well, it needs to be trained with a specific dataset. Fine-tuning here means training a pre-trained LLM again so that it becomes better at responding to instructions. More specifically, fine-tuning on instruction data means modifying the weights and parameters of the GPT model so that it can learn to understand instruction input-output pairs from a specific dataset.

That raises the next question: what does the instruction-tuning dataset look like? In instruction fine-tuning, the dataset consists of instructions and the desired responses. The dataset used here contains 1,100 entries of instructions, inputs, and outputs. These entries come from various fields, including English, geography, and general knowledge. The aim of using this dataset is to teach the LLM to correctly follow instructions. For example, one entry uses the instruction "translate the following sentence into French," the input "where is the nearest restaurant," and the output as the French translation. Another uses the instruction "rewrite the following sentence so that it is in active voice," the input "the cake was baked by Sarah," and the output "Sarah baked the cake." Some pairs have no separate input field, because the sentence or question is included directly in the instruction.

Once the data is collected, it must be divided for training and evaluation. The dataset was split into training, testing, and validation sets using 85% for training, 10% for testing, and 5% for validation. With 1,100 total pairs, this split gives 935 training examples, 55 validation examples, and 110 test examples. Although the full dataset contains 1,100 instruction-input-output pairs, with an 85% split only about 800 instruction-input-output pairs are used for training.

6.2 Formatting instruction data into Alpaca-style prompts

How do instruction, input, and output fields become training text a model can actually consume for fine-tuning? Formatting the dataset means converting entries with the three keys "instruction", "input", and "output" into the "alpaka" prompt style before feeding them to the large language model for fine-tuning. In this prompt style, the prompt starts with a fixed introductory instruction: "Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request." After that introduction, the prompt includes the dataset instruction, then the dataset input, and then the response taken from the dataset output. That is the specific formatting applied to instruction-input-response data to convert it into an Alpaca-style prompt.

The code begins by downloading the dataset and checking that it loaded correctly. It reports 1100 entries and verifies the contents by printing the 50th and 999th examples and confirming that they match the dataset. The conversion itself is handled by a function called "format_input". This function takes an entry containing instruction, input, and output fields and converts it into the Alpaca prompt. It builds the prompt from the instruction plus the input, and the response is handled separately as the output. For a single entry, that means taking the introductory first line, adding the instruction and the input, and then appending the desired response to obtain the full prompt.

Once each example is written in this prompt format, the next step is to turn it into token IDs. Using the "tick token" library, the process is applied to the first instruction-input-output example, then to the second, and then to all examples in the dataset: convert to the "alpaka" format and convert that text into a bunch of token IDs. The next step is to organize the data into training batches by coding an instruction dataset class. This class takes the dataset in this format and first converts it into the Alpaca-style format. It does that by using the "format_input" function to produce the first sentence, the instruction, and the input text, and then appending the response or output.

When an instance of this instruction dataset class is created, it first stores the dataset in "self.data". Then, for each entry, it applies "format_input" and appends the response so that the full text in the "alpaka" format is created. After that, it tokenizes the formatted data by defining an empty list and appending token IDs to that list. For every prompt corresponding to an input-output pair, each token in the prompt is converted into a token ID and appended to the list. The result returned by the instruction dataset class is, for every dataset entry, the "alpaka" style prompt converted into a bunch of token IDs.

Listing 06.02.1: Implements an InstructionDataset class that formats each data entry, appends a response section from the output field, and tokenizes the full text for storage. Source: rasbt/LLMs-from-scratch, `ch07/01_main-chapter-code/ch07.ipynb`, Apache-2.0, commit cdbd33e.

```
import torch
from torch.utils.data import Dataset


class InstructionDataset(Dataset):
    def __init__(self, data, tokenizer):
        self.data = data

        # Pre-tokenize texts
        self.encoded_texts = []
        for entry in data:
            instruction_plus_input = format_input(entry)
            response_text = f"\n\n### Response:\n{entry['output']}"
            full_text = instruction_plus_input + response_text
```

```
            self.encoded_texts.append(
                tokenizer.encode(full_text)
            )

    def __getitem__(self, index):
        return self.encoded_texts[index]

    def __len__(self):
        return len(self.data)
```

6.3 Tokenization and variable-length sequences

How do formatted instruction prompts become batchable numbers, and why is that necessary before fine-tuning can proceed? To create a batch, each dataset entry is converted into a numerical representation so that the first sample forms the first row, the second sample forms the second row, and the third sample forms the third row. If a batch contains three samples, the first sample becomes the first row, the second sample becomes the second row, and the third sample becomes the third row after this numerical conversion. In this numerical representation, the rows must have the same size. Here, size means the number of columns in each row. That requirement creates a problem immediately. Different instruction-output pairs produce different prompts, and their lengths may differ as well.

The first step in turning formatted data into a numerical representation is tokenization. A tokenizer is used to convert each prompt into token IDs. The tokenizer used here is the byte pair encoder used by OpenAI, accessed through the tiktoken library. For example, after an instruction-input-output record is formatted in Alpaca style, the entire prompt is converted into a bunch of token IDs. The same tokenization process is applied to every instruction-input-output example in the dataset. Once prompts are tokenized, their lengths still do not automatically match. Because prompts can have different lengths, the number of token IDs differs across samples. In the example, the first prompt looks longer than the second prompt, so the first prompt has more token IDs than the second. But if two samples are placed in one batch, the number of token IDs for each sample must be the same.

After the full text in Alpaca format is created, the formatted data is tokenized by first defining an empty list. The token IDs for each prompt are then appended to that empty list. For every prompt corresponding to an input-output pair, this produces a list of token IDs. The tokenizer passed to the instruction dataset class is the tiktoken library. For more detail on this library, see the separate discussion of the byte pair encoder.

6.4 Batch padding with the end-of-text token

How do variable-length token sequences become a batch that can be processed together? The way to do it is to pad token ID sequences so that all numerical representations have exactly the same length. In each batch, the longest sequence is found, that sequence is kept as it is, and the remaining sequences are padded until their lengths match the longest length. For example, with input lengths 5, 2, and 3, the first input stays unchanged, the second is padded with "50256" three times, and the third is padded with "50256" two times so all rows have length 5. After padding, all token ID representations in the batch have the same length, the number of columns matches across rows, and they can be processed together as a batch. This padding is done separately for each batch.

> **Note.** The course's reference code differs here: Here the lecture describes per-batch padding; the reference implementation instead pads to the longest training sequence for the whole dataset.

Why use "50256" for padding instead of some other ID? "50256" is the end-of-text token. GPT-2 has a vocabulary size of "50257", so token IDs run from 0 to "50256", and "50256" is the last token ID. That last token corresponds to end of text, which marks that one text sample has ended and a new text sample begins. That is why the end-of-text token ID 50256 is used. Using another token ID could associate the padding with an actual token and confuse training.

A custom collate function automates this batch padding step. The custom collate function takes the inputs in each batch, finds the input with the maximum length, and pads all other inputs with token ID "50256". The draft version of this function takes a batch, a padding token ID of "50256", and a device such as CPU. It finds the longest sequence in the batch, pads the other sequences to that length, converts the list of inputs into a tensor, and transfers it to the target device. The padded result is converted into an input tensor, and the custom collate draft one function returns that input tensor.

One extra step is added to make the target tokens easy to construct later. The function first finds the longest sequence length in the batch and adds one to that length. For every item in the batch, it first adds one "50256" token and then pads with additional "50256" tokens until the sequence reaches the maximum length. After padding, the function removes the extra added token. The reason for adding this extra "50256" token first is that it later helps create the target, because the target is formed by removing the first element of the input and adding "50256". So the code adds an extra "50256" to each input first, gets rid of it later, and pads the inputs with "50256" until every sequence matches the maximum length in the batch.

Listing 06.04.1 (excerpt): This function finds the longest sequence length in a batch, adds one end-of-text token with ID 50256 to each item, pads shorter items with 50256, removes the extra final token, stacks the results into a tensor, and moves it to the target

device. Source: rasbt/LLMs-from-scratch, ch07/01_main-chapter-code/ch07.ipynb, Apache-2.0, commit cdbd33e.

```
def custom_collate_draft_1(
    batch,
    pad_token_id=50256,
    device="cpu"
):
    # Find the longest sequence in the batch
    # and increase the max length by +1, which will add one extra
    # padding token below
    batch_max_length = max(len(item)+1 for item in batch)

    # Pad and prepare inputs
    inputs_lst = []

    for item in batch:
        new_item = item.copy()
        # Add an <|endoftext|> token
        new_item += [pad_token_id]
        # Pad sequences to batch_max_length
        padded = (
            new_item + [pad_token_id] *
            (batch_max_length - len(new_item))
        )
        # Via padded[:-1], we remove the extra padded token
        # that has been added via the +1 setting in batch_max_length
        # (the extra padding token will be relevant in later codes)
        inputs = torch.tensor(padded[:-1])
        inputs_lst.append(inputs)

    # Convert list of inputs to tensor and transfer to target device
    inputs_tensor = torch.stack(inputs_lst).to(device)
```

6.5 Shifted targets for next-token prediction

How do the target token IDs tell the model what to generate, and why are they built from the input itself rather than from the response alone? For training, batches must include target token IDs because they represent what the model should generate and are used to compute the loss. The key step is to create the target by shifting the input by one and adding an additional padding token 50256 so that the target and input have the same length. If the input token IDs are “0 1 2 3 4”, the target token IDs are formed by

dropping the first ID, keeping the remaining IDs, and adding the padding token 50256 so that the target has the same length as the input. If the input token IDs are "56 50256 50256 50256", the target is formed by removing the first token, keeping the remaining tokens, and padding with an extra "50256". This shifted-target construction is described as the most important step in the fine-tuning process, and it can be hard to grasp at first.

The target pair is shifted to the right by one because the model still performs the next-token prediction task. A shifted input-target sequence means that if the input is "0", the output should be "1"; if the input is "0 1", the target is "2"; if the input is "0 1 2", the target is "3"; and if the input is "0 1 2 3", the target is "4". When the input reaches the full sequence, the model is at the end of the sentence. Training on the shifted sequence teaches the model sequentially that if this much is the input, this should be the output. Although this setup includes some redundant training, it works through next token prediction so that once the model has seen the instruction and input, it predicts the response.

This is the nonintuitive part of instruction fine-tuning. The input is the full sequence and the target is the full sequence shifted by one, not just the response alone. The input sequence itself contains both the instruction-input part and the output part, because LLMs are trained with next token prediction. By learning to predict the next token, the LLM learns to follow the instructions, and at a certain stage the output is the response. Instruction fine-tuning is very similar to pre-training because both use targets shifted by one token. Even though this shifted-target setup is nonintuitive in instruction fine-tuning, it works because predicting the next token teaches the model to take the instruction and input and predict the response. It may seem nonintuitive that the true values include the instruction and input as well as the response, but in next-word prediction the LLM learns to predict the response from the instruction and input.

The batching code follows the same idea directly. A simple way to understand the batching code is that the inputs are padded to the same length and the targets are the inputs shifted to the right by one. In the code, the targets are formed by taking the padded input from the second element onward, which works because an extra "50256" token was already added. Custom collate draft two truncates the last token for the inputs so they have the correct length and shifts the input to the right by one to create the target tensor. In the draft two example, each target row is the corresponding input row shifted right by one position, with a "50256" token appended at the end.

One detail matters for the end of a sequence. One "50256" end-of-text token is kept because it teaches the LLM when to generate an end-of-text token to mark that the response is complete. When padding-related "50256" tokens in the targets are later replaced with "-100", the first "50256" should be retained. The first "50256" must be retained because it marks the end of a particular text, while the remaining "50256" token IDs are unnecessary and can be replaced with "-100". Retaining that first "50256" helps the LLM learn to generate end-of-text tokens as the signal that the response is complete.

Listing 06.05.1 (excerpt): This dataset class tokenizes text and builds each training example by taking an input chunk and a target chunk shifted one position ahead. Source: rasbt/LLMs-from-scratch, `ch02/01_main-chapter-code/ch02.ipynb`, Apache-2.0, commit cdbd33e.

```
from torch.utils.data import Dataset, DataLoader


class GPTDatasetV1(Dataset):
    def __init__(self, txt, tokenizer, max_length, stride):
        self.input_ids = []
        self.target_ids = []

        # Tokenize the entire text
        token_ids = tokenizer.encode(txt, allowed_special={"<|endoftext|>"})
        assert len(token_ids) > max_length, "Number of tokenized inputs must at least be

        # Use a sliding window to chunk the book into overlapping sequences of max_lengt
        for i in range(0, len(token_ids) - max_length, stride):
            input_chunk = token_ids[i:i + max_length]
            target_chunk = token_ids[i + 1: i + max_length + 1]
            self.input_ids.append(torch.tensor(input_chunk))
            self.target_ids.append(torch.tensor(target_chunk))

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, idx):
        return self.input_ids[idx], self.target_ids[idx]

def create_dataloader_v1(txt, batch_size=4, max_length=256,
                         stride=128, shuffle=True, drop_last=True,
                         num_workers=0):

    # Initialize the tokenizer
```

6.6 Ignoring padding in the loss with -100

Why mark some target positions so the loss will skip them at all? After creating the target token IDs by shifting the inputs and adding an additional padding token at the end, the remaining step is to replace padding tokens in the targets with placeholders. If a target contains several 50256 token IDs, the first is kept and all remaining 50256 values

are replaced with -100. If a target contains several “50256” values, the first one is kept because it indicates the end of text, and all remaining “50256” values are replaced with “-100”. If a target has only the end-of-text “50256”, that first “50256” remains unchanged. This keeps one end-of-text token while removing the extra end-of-text padding tokens from consideration.

The choice of -100 comes from the default ignore_index in PyTorch’s cross-entropy loss implementation. Cross-entropy loss is calculated from a logits tensor of predictions and a target tensor of true answers. In PyTorch, the default “ignore_index” for cross-entropy loss is “-100”. That means any target entry labeled “-100” is ignored when the loss is computed. This is why the extra padded target positions are changed to “-100”: those added tokens do not matter and should be excluded from the training loss. Excluding them makes training more efficient and avoids unnecessarily including tokens that are not important.

A small loss example makes the effect clear. With two examples, cross-entropy on the logits tensor and target tensor gives a loss of “1.11269”. Adding a third example changes the loss because one more training example is included. But if that third target is changed to “-100” while the predictions stay the same, the loss becomes the same as the first loss, as if the third example had no effect. The reason is that cross-entropy ignores the entry whose target value is “-100”.

The function takes a batch, a padding token ID, and an ignore index, which is “-100”. It first produces the input tensor and the target tensor as before, and then it modifies only the target tensor. To do that, it creates a mask over the target positions that contain the padding token ID, skips the first such position, and replaces the remaining padded target positions with the ignore index “-100”. With this modified collate function, the inputs stay unchanged, while in the targets every “50256” after the first one is replaced by “-100”. In that sense, the collate function implements the whole batching recipe on the whiteboard: padding, creating target token IDs, and replacing padded target tokens with the placeholder value “-100”, also called the ignore index.

Listing 06.06.1 (excerpt): This collate function pads each sequence with token 50256, builds shifted input and target tensors, keeps the first 50256 in each target, and replaces any later 50256 target positions with -100. Source: rasbt/LLMs-from-scratch, `ch07/01_main-chapter-code/ch07.ipynb`, Apache-2.0, commit cdbd33e.

```
def custom_collate_fn(
    batch,
    pad_token_id=50256,
    ignore_index=-100,
    allowed_max_length=None,
    device="cpu"
):
    # Find the longest sequence in the batch
```

```
    batch_max_length = max(len(item)+1 for item in batch)

    # Pad and prepare inputs and targets
    inputs_lst, targets_lst = [], []

    for item in batch:
        new_item = item.copy()
        # Add an <|endoftext|> token
        new_item += [pad_token_id]
        # Pad sequences to max_length
        padded = (
            new_item + [pad_token_id] *
            (batch_max_length - len(new_item))
        )
        inputs = torch.tensor(padded[:-1])  # Truncate the last token for inputs
        targets = torch.tensor(padded[1:])  # Shift +1 to the right for targets

        # New: Replace all but the first padding tokens in targets by ignore_index
        mask = targets == pad_token_id
        indices = torch.nonzero(mask).squeeze()
        if indices.numel() > 1:
            targets[indices[1:]] = ignore_index
```

- *Checked by running the code:* `50256` = 50256 (the text says 50256)
- *Checked by running the code:* `50256` = 50256 (the text says 50256)
- *Checked by running the code:* `50256` = 50256 (the text says 50256)

6.7 The full batching recipe and optional masking choices

How does instruction fine-tuning data become batches the model can train on, and why is that worth spelling out carefully? Creating batches for instruction fine-tuning is very important because the process includes specific details such as padding tokens, using "-100", and creating target token IDs by shifting the input token IDs to the right by one. Batching the dataset involves five detailed steps. First, the data is formatted using a prompt template and converted to the Alpaca prompt template. Next, the formatted prompt is converted into token IDs. Because tokenized samples do not all have equal length, each batch is padded with token "50256" so that every sample in that batch has the same length as the longest sample in the batch. After that, target token IDs are created by shifting the input token IDs to the right by one. Finally, all tokens with value 50256 except the first are replaced with -100. The first "50256" is kept because it indicates end of text, while the other replaced positions are excluded from the training loss because PyTorch cross-entropy loss uses an ignore index of "-100".



Figure 6.1: Instruction fine tuning batches are prepared by formatting prompts, tokenizing them, padding each batch, shifting inputs to form targets, and replacing extra padding targets with the ignore index.

There is also an optional masking choice beyond padding. Some researchers also mask the target token IDs so that only the response part matters during training. Here, masking the target token IDs means masking out the target token IDs that correspond to the instructions. This masking is implemented by replacing those target token IDs with "-100". When the non-response parts of the target are replaced with "-100", those tokens are ignored in the loss calculation. If the target token IDs corresponding to the instruction are masked out, the LLM cross-entropy loss is calculated only for the generated response target IDs. Masking out instruction tokens trains the model to focus on generating accurate responses rather than memorizing instructions, which can help reduce overfitting.

Whether that extra masking should be used remains unsettled. It is not yet confirmed which approach works best, and researchers are divided on whether masking instructions is universally beneficial during instruction fine-tuning. A recent paper titled "instruction tuning with loss over instructions" reported that not masking the instruction benefits LLM performance. So the choice between masking and not masking is still an open research question.

6.8 Loading pretrained GPT-2 weights for fine-tuning

How does a model move from a freshly defined GPT architecture to a useful starting point for instruction fine-tuning? Loading a pre-trained LLM means using previously trained weights with the constructed GPT architecture. Those trainable parts include the query, key, and value weight matrices in multi-head attention, the weights in the feed-forward network, the scale and shift parameters in layer normalization, and the token and positional embedding layers. When all of these trainable weights are added together, the total comes to more than 1 million trainable parameters. In pre-training, these parameters are trained on a huge amount of data.

OpenAI pre-trained GPT-2 on a huge amount of data, optimized its trainable weights, and released those weights publicly so that users can access and use them. Those GPT-2 weights were released for multiple model sizes, including 124 million, 345 million, 774 million, and 1558 million parameters, and they can be downloaded from folders corresponding to those parameter sizes. Before fine-tuning, these weights can be downloaded and reused because they already capture useful information, including semantic meaning

in token embeddings, while training the whole network again from scratch would take much longer and require more computation. Reusing GPT-2 weights helps because the model starts from a much more knowledgeable state instead of a random initialization.

Before loading a pre-trained LLM, the workflow required downloading the dataset, converting it into the Alpaca prompt style, batching it, and creating data loaders. With the data pipeline in place, the next step is to load the model and continue from there. The process is to load the pre-trained weights first and then train on the curated dataset using the training, testing, and validation data loaders. During fine-tuning, the weights loaded from GPT-2 still change, but starting from a knowledgeable state makes training on the specific dataset more efficient and lowers the computational time. Fine-tuning happens after pre-training because pre-training first brings the model to a knowledgeable state, and then training on a specific dataset further improves its weights and parameters.

For this instruction-following task, model size matters. A pre-trained LLM by itself does not follow instructions very well, which is why it must be trained with a specific dataset. For this task, GPT-2 with 355 million parameters is used because the smaller 124 million parameter model does not perform very well. Using GPT-2 medium with 355 million parameters requires about 1.42 GB of storage space on the local machine. The GPT-2 medium configuration has context size 1024, 24 transformer blocks, and 16 attention heads in each transformer block. In the base configuration, the vocabulary size is 50,257, the context length is 1024, the dropout rate is zero, and the query-key-value bias is set to true. The value 50,257 is the vocabulary size on which GPT-2 was trained.

> **Note.** The course's reference code differs here: The lecture uses GPT-2-style settings here (dropout 0, qkv bias true); the reference code switches to 0.1 dropout and disables qkv bias for the book's implementation.

The choice of size can be adjusted to the machine that will load the model. If a laptop has limited memory or processing speed, GPT-2 small with 124 million parameters can be used instead, and it should take about one-third to one-half as much time to load as GPT-2 medium. If a laptop has high processing speed and GPU access, GPT-2 large or GPT-2 XL can be used, and they should give much better results than smaller models.

6.9 Downloading and mapping GPT-2 parameters into the model

How do the downloaded GPT-2 checkpoint files become weights that a GPT implementation can actually use? The "download_and_load_gpt2" function accesses a URL and downloads seven files onto the local machine, but those files still require preprocessing steps. Simply downloading the seven files is not enough, because the downloaded parameters must be converted from the TensorFlow checkpoint into a specific dictionary. That dictionary makes the parameters much easier to handle and access. This conversion

matters because directly using only the seven downloaded files makes it difficult to integrate those parameters with the GPT code written earlier.

Once that setup is clear, the code can be read as a two-stage loading process. Running the download code creates a folder called "355 million", downloads the seven files, and returns a "params" dictionary. This params dictionary consists of the transformer block parameters, token embeddings, positional embeddings, and final normalization layer parameters. The code first imports the "download_and_load_gpt2" function from the "GPT_download3.py" file and stores the results in "settings" and "params". For the shown model, the download is 1.42 GB, so it can take a long time if the internet connection is weak, and the device should have memory available.

> **Note.** The course's reference code differs here: The reference code example uses the 124M model in a gpt2 directory; the lecture here describes downloading the 355M model instead.

After the files have been downloaded and organized, the next job is to place those values into the model itself. The "load_weights_into_gpt" function maps the downloaded parameters from the "params" dictionary into the corresponding trainable parts of the GPT architecture. You can think of this whole loading block as an assignment step that assigns the downloaded parameters to the model. After running "settings, params = download_and_load_gpt2", the next step is to run "load_weights_into_gpt" to load the downloaded weights from the 355 million model into the GPT architecture, and then set the model to evaluation mode.

6.10 Checking the pretrained model before fine-tuning

How close is a pre-trained model to the target behavior before any instruction fine-tuning, and why is it worth checking now? Before fine-tuning, it is useful to check a pre-trained LLM on one validation task by comparing its output with the expected response, even though the model has not been trained or fine-tuned on this dataset yet. A concrete validation example uses the instruction to convert the active sentence "The chef cooks the meal every day" into the passive voice. For this example, the correct output is "The meal is cooked by the chef every day." Because the model has not been trained on this data at all, the goal is not to get the right answer yet, but to see how far its response is from the desired one.

To do that, generation uses the previously defined "generate" function. That function takes the maximum number of new tokens to generate and then produces new tokens based on the model and its weights. In this setup, the model uses GPT-2 Medium pre-trained weights, the maximum number of new tokens is set to 35, the context length is 1024, the input text and model are passed in, and the generated token IDs are converted back into text. One detail matters for instruction-following: the generated text includes the original input together with the model's continuation. For instruction-following use,

the part that matters is only the model-generated response, not a reprint of the input instruction. To print only that response, subtract the length of the input instruction from the start of the generated text and print what remains.

In this example, the model's response is "the chef cooks the meal every day," which repeats the original sentence instead of carrying out the instruction. The pre-trained model is not able to correctly follow the instruction, because it repeats the original input sentence, repeats part of the instruction, and fails to convert the active sentence to passive voice. Without fine-tuning, simply loading pre-trained weights does a poor job on this instruction-following task and can even reproduce formatting such as "### instruction" in the response itself. Fine-tuning is needed to improve the model's ability to comprehend and appropriately respond to these requests. A 774 million parameter GPT-2 model still does not produce a coherent response without fine-tuning.

Listing 06.10.1: This code extracts the generated response by slicing off the original input text, removing the response marker, stripping whitespace, and printing the result. Source: rasbt/LLMs-from-scratch, ch07/01_main-chapter-code/ch07.ipynb, Apache-2.0, commit cdbd33e.

```
response_text = (
    generated_text[len(input_text):]
    .replace("### Response:", "")
    .strip()
)
print(response_text)
```

Not executed: error in cell 80.

6.11 Evaluating a fine-tuned model: goals, examples, and response extraction

How can a fine-tuned model be judged once dataset preparation and fine-tuning are complete, and why is this step necessary? By this point, the instruction fine-tuning pipeline has finished preparing the dataset and fine-tuning the model, and the next stage is to evaluate the large language model. The model was initialized with publicly available GPT-2 weights so that it started from a good foundational base before instruction fine-tuning. In the demonstrated run, training was limited to one epoch because of memory and compute limitations on the CPU. Running for more epochs can improve response quality, and two epochs instead of one is given as a concrete example. For the passive-voice test, the fine-tuned model produced "the meal is prepared by the chef every day," which is better than random output but not the correct answer.

> **Note.** The course's reference code differs here: The lecture example says the model answered "prepared," while the reference's printed example shows the correct passive form "cooked every day by the chef."

The difficulty is not generating a sentence, but deciding how good that sentence is. Evaluating a large language model is hard because performance is not a simple yes-or-no classification problem. Instead, evaluation requires comparing a generated sentence with a target sentence and deciding how close they are. That makes instruction-tuned model evaluation harder than spam-versus-not-spam classification, where accuracy is easy to compute from whether the predicted label matches the correct label. Because of this, large language model evaluation is treated as an important and rapidly developing field. The evaluation process here is organized into three stages: extracting responses, qualitative evaluation, and scoring the responses.

The first practical step is to gather the model's answers from the test set before trying to score them. To evaluate on the test set, the model's response is generated for every test input and collected for manual analysis and later comparison with the ground-truth outputs. A small demonstration loops over the test data, takes the first three entries, generates outputs with the fine-tuned model, converts token IDs to text, removes the instruction and input portion from the generated text, and prints the instruction, the correct response, and the model response. For the prompt asking to rewrite a sentence using a simile, the correct response is "the car is as fast as lightning," while the model response "the car is as fast as a bullet" is very close. For the question asking which cloud type is typically associated with thunderstorms, the model response is not really doing what it is asked to do. For the question about the author of Pride and Prejudice, the correct response is Jane Austen, but the model answered "George Bernard Shaw," which is clearly wrong. From these three samples, the first answer is close to correct, the second does not answer correctly, and the third is fully wrong. Qualitative inspection can reveal whether outputs seem correct or incorrect, but it does not by itself provide a numerical evaluation score.

Next, collect model responses for all instructions in the test dataset and store them in a separate file for evaluation. This preparation creates a new file called "instruction data with response.json" for record keeping. That file contains the instruction, the input, the true output, and the model response. The original "instruction data.json" file is shown as containing only the instruction, input, and true output, while the new file is shown as also including the model response. Keeping both the true output and the model response in one place makes later evaluation easier because each pair can be compared directly and assigned a score. The code iterates over all instruction inputs in the test data, generates responses for them, and writes the results to the "instruction data with response" file. When this code runs, the generate function is called on all instruction-input pairs in the test data, the responses are generated, and only the responses are collected and appended to the instruction data with response file. The resulting file is shown as the original instruction-input-output dictionary with an added model response field for every

example. After this process, the test data dictionary contains the instruction, input, output, and model response.

> **Note.** The course's reference code differs here: An earlier cell generates responses over the test set and writes instruction-data-with-response.json; this later optional cell only reloads that file.

After the evaluation file is prepared, preserving the trained model avoids repeating the expensive fine-tuning run. The fine-tuned model should be saved so the session can be resumed later without fine-tuning again. The model is saved with torch.save and model.state_dict, which ensures that all trained parameters are stored. In a future session, "load_state_dict" is used to load the saved parameter file. At this stage, responses have been collected in "instruction data with response.json", three evaluation frameworks have been discussed, and the LLM-based comparison framework has been selected. Earlier work has already included response extraction and qualitative evaluation by looking at outputs and responses, but numerical scoring has not yet been implemented at this point. Across the full pipeline, the work includes preparing the dataset, fine-tuning the large language model, extracting responses, doing qualitative evaluation, and quantitative scoring using AMA.

Listing 06.11.1: This code loops over the test data, formats each prompt, generates text with the model, strips the prompt and response marker from the generated text, stores the result in each test entry, and writes the augmented test data to instruction-data-with-response.json. Source: rasbt/LLMs-from-scratch, `ch07/01_main-chapter-code/ch07.ipynb`, Apache-2.0, commit cdbd33e.

```
from tqdm import tqdm

for i, entry in tqdm(enumerate(test_data), total=len(test_data)):

    input_text = format_input(entry)

    token_ids = generate(
        model=model,
        idx=text_to_token_ids(input_text, tokenizer).to(device),
        max_new_tokens=256,
        context_size=BASE_CONFIG["context_length"],
        eos_id=50256
    )
    generated_text = token_ids_to_text(token_ids, tokenizer)
    response_text = generated_text[len(input_text):].replace("### Response:", "").strip()

    test_data[i]["model_response"] = response_text
```

```
with open("instruction-data-with-response.json", "w") as file:
    json.dump(test_data, file, indent=4)  # "indent" for pretty-printing
```

Not executed: error in cell 98.

6.12 Evaluation frameworks for instruction-tuned LLMs

How should an instruction-fine-tuned model be evaluated once it can produce responses, and why is that choice important? Three common evaluation methods for instruction-fine-tuned LLMs are MMLU, human preference comparison, and using another larger language model to evaluate the model's performance. One very common method is to give the LLM a general knowledge test by benchmarking it on known responses, and this is called measuring Massive Multitask Language Understanding, or MMLU. An MMLU score is used as a measure of how the LLM is performing on general knowledge tasks. A summarized description on screen defines MMLU as a benchmark for measuring language model knowledge and for evaluating a text model's ability to learn and apply knowledge from various domains. MMLU consists of 57 tasks that cover STEM, humanities, social sciences, and other areas, with difficulty ranging from elementary to professional levels. A custom instruction-fine-tuned LLM can be evaluated with MMLU by testing its performance on those 57 tasks, and many papers in the instruction-finetuned LLM space report the MMLU score. Benchmarks such as MMLU help you learn a lot through the process, and the MMLU paper is highly impactful and highly relevant.

Human preference comparison evaluates LLMs by having humans look at answers from multiple LLMs and compare their performance. This method puts a human in the loop, and the human uses their own intuition and understanding to benchmark or compare LLMs. It can also be described as asking humans to rate LLM output.

Another common approach is to use another large language model to compare the expected output with the model response and assign a score. The setup is to keep the instruction, input, true output, and model response together, then ask a large language model to compare the true output with the model response and assign a score. This approach is fairly straightforward because it is fully automated. At the same time, it is like taking the easy way out because the evaluation criteria used by the evaluator LLM are not known, so the method depends on trusting that LLM. After responses are extracted from the fine-tuned LLM, another large language model can automatically evaluate them. Using another LLM for this step can efficiently assess response quality without extensive human involvement, saving time and resources while still producing meaningful performance indicators. Another larger pre-trained LLM can compare the model response with the true response and assign a score. In the example implementation, the chosen evaluator is the Meta Llama 8B Instruct model, used as an automated and simpler evaluation method.

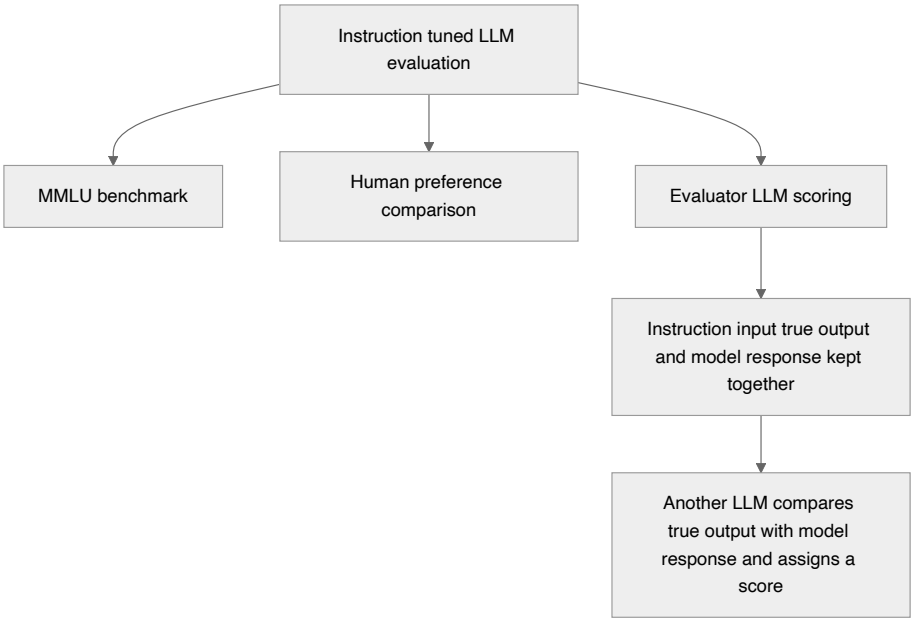


Figure 6.2: Instruction tuned models can be evaluated with a benchmark suite, human comparisons, or an evaluator LLM that compares model responses with reference outputs and assigns scores.

What kind of result does this automated scoring produce, and what are its limits? The goal here is to move beyond qualitative inspection and quantify the responses with a score. By asking the evaluator to respond with an integer number only, the evaluation produces a quantitative metric. That quantitative score is still not as robust as classification accuracy, because the assigned number remains subjective. LLM evaluation remains an open research area because there is substantial subjectivity in the choice of metrics and in judging responses, unlike problems where accuracy is a single value based on correct and wrong answers.

Listing 06.12.1: This loop builds a prompt containing the input, the correct output, and the model response, then asks another model to score the response for a few test entries and prints the results. Source: rasbt/LLMs-from-scratch, `ch07/01_main-chapter-code/ch07.ipynb`, Apache-2.0, commit cdbd33e.

```
for entry in test_data[:3]:
    prompt = (
        f"Given the input `{format_input(entry)}` "
        f"and correct output `{entry['output']}`, "
        f"score the model response `{entry['model_response']}`"
        f" on a scale from 0 to 100, where 100 is the best score. "
    )
```

```
    print("\nDataset response:")
    print(">>", entry['output'])
    print("\nModel response:")
    print(">>", entry["model_response"])
    print("\nScore:")
    print(">>", query_model(prompt))
    print("\n-------------------------")
```

Not executed: error in cell 118.

6.13 Using Ollama and Llama 3 as an automated evaluator

How can a local model be turned into an automatic evaluator for instruction-following outputs, and why is that useful here? Ollama is software, or an application, for efficiently running large language models on a laptop. It lets a user run models such as Llama 3, Mistral, and Gemma 2 on a PC or laptop. Ollama is used for inference only, not for pre-training. It is only a tool for generating text with LLM inference and does not support training or fine-tuning. It is also described as a very commonly used LLM inference application. The evaluation setup uses an instruction-fine-tuned Llama 3 model, accessed through Ollama. That model is chosen because it has already been fine-tuned on many instructions and is already trained to follow instructions well. The idea is to use Llama 3 to evaluate an LLM by comparing the correct output with the model response and assigning a score. Because the evaluator model is already trained to follow instructions, it is expected to do a great job on the new instruction of producing an evaluation score.

To install Ollama, go to "ollama.com", click download, choose the version for your operating system, and follow the installation steps. After installation, open a terminal and run "ollama run llama3". On Windows, first run "ollama serve" and then run "ollama run llama3". Running Llama 3 this way downloads files of about 4.7 GB, and the model takes substantial space and memory because it has 8 billion parameters. During the download, instructions are printed in the terminal, and after success, right-side arrows appear to show that Llama 3 is loaded. Once it is loaded, you can interact with Llama 3 by asking questions. A simple check is to ask a question and confirm that a response comes back. You must keep Ollama running while executing the later code, because if you shut down the terminal, Ollama stops and the code will not execute. Before using Ollama to evaluate test responses, run the verification code and confirm that it prints "Ollama running = true"; if it prints false, Ollama is not running successfully. Before later code cells, check again that Ollama is still active by confirming that the earlier test prints "Ollama running = True".

Instead of going to the terminal each time, a Python function can interact with the Llama model through the API. The "query_model" function takes a prompt and a model, sends

an API request to that model based on the prompt, and returns the generated response. In a simple demo, "query_model" is called with a prompt such as "what do llamas eat" and a model such as Llama 3, and then the response is printed. If no model is specified, "query_model" uses Llama 3 by default. Querying local models this way can require substantial compute and memory, can be slow on CPU, and may require freeing 15 to 20 GB of memory for the code to run.

With that in place, you can now turn to the evaluation prompt. In this setup, the prompt sent through "query_model" asks the evaluator model to compare the output and the model response and assign a score. To evaluate the responses generated by the fine-tuned model, a prompt sent through query_model asks Llama 3 to rate those responses, with 100 as the best score. The evaluation prompt includes the instruction and input, the correct output, and the model response, and asks for a score where 100 is the best score. The example evaluation code tests three data points and prints each model response together with the score assigned by a Llama 8 billion model. In the first example, the evaluator gives 85 out of 100 because the response uses a simile correctly and captures the intended meaning, even though it is less vivid than the reference answer. In the second example, the evaluator gives 20 out of 100 because the response does not directly answer the question, is largely irrelevant, and contains inaccuracies. In the third example, the evaluator gives 0 out of 100 because the answer is factually incorrect and irrelevant.

These examples show what this style of model-based evaluation can and cannot provide. The evaluator is described as doing an awesome job at qualitative evaluation, and its quantitative scores kind of make sense even if the exact numbers are not fully clear. It can provide reasonable evaluations and assign partial credit as well as zero when an answer is incorrect. If you want only a numeric metric instead of a qualitative explanation, change the prompt so the evaluator responds with an integer number only. A function is also available that iterates over the entire test set, computes this kind of score for every instruction, and then calculates an average score. That full-test-set function is not run on the shown machine because of hardware limitations, while faster hardware such as an M3 MacBook Air or a GPU can run it much more quickly. Ollama is not entirely deterministic, so your scores may differ slightly, but the method still provides a framework for quantitative evaluation. When the full evaluation is run, the fine-tuned model achieves an average score above 50, which can be used as a benchmark for comparing other models or training configurations. That average is described as just above 50, around 55%, and not very good, because some answers are correct and some are wrong.

Listing 06.13.1: This loop builds an evaluation prompt from each test entry, prints the correct and model responses, and queries Llama 3 for a 0-to-100 score on the first three examples. Source: rasbt/LLMs-from-scratch, `ch07/01_main-chapter-code/ch07.ipynb`, Apache-2.0, commit cdbd33e.

```
for entry in test_data[:3]:
    prompt = (
        f"Given the input `{format_input(entry)}` "
```

```
        f"and correct output `{entry['output']}`, "
        f"score the model response `{entry['model_response']}`"
        f" on a scale from 0 to 100, where 100 is the best score. "
    )
    print("\nDataset response:")
    print(">>", entry['output'])
    print("\nModel response:")
    print(">>", entry["model_response"])
    print("\nScore:")
    print(">>", query_model(prompt))
    print("\n-------------------------")
```

Not executed: error in cell 118.

6.14 Improving results and extending the approach

Why might an instruction-tuned model still fail even when it often gives useful answers? One failure mode appears when the model repeats the input instead of carrying out the requested rewrite. For example, with the input sentence "the lecture was delivered in a clear manner," the actual output is "the lecture was delivered clearly," but the model response remains "the lecture was delivered in a clear manner." That behavior is a mistake, although the model is also good in many cases, which is why the evaluation accuracy was around 55%.

Several changes can improve performance. One option is to adjust hyperparameters such as the learning rate, batch size, or number of epochs. Another option is to increase the size of the training dataset. The dataset used here contained 1100 instruction-input-output pairs, and with 85% used for training, the model was in effect trained on about 800 pairs. Those pairs were described as not enough. The Stanford Alpaca repository contains a dataset of 52,000 instruction-output pairs. That is about 50 times larger than the dataset used here, and using such a dataset may produce better responses.

Data size is not the only lever. You can also experiment with different prompts or instruction formats to guide the model responses more effectively. Another change is to use a larger pre-trained model, because a larger model may have greater capacity to capture complex patterns and generate more accurate responses. The model used here was "gpt2 medium" with 355 million parameters. Larger alternatives mentioned were "gpt2 large" with 774 million parameters and "gpt2 xl" with more than a billion parameters. These larger models should be used only when sufficient compute power or GPU access is available. For example, "gpt2 xl" was described as having 48 transformers and 25 attention heads in each transformer.

The fine-tuning method used here was standard full fine-tuning: all parameters in the architecture were updated based on the specific dataset. Parameter-efficient fine-tuning refers to methods such as "LoRA" that are designed to make fine-tuning more efficient than normal fine-tuning. These methods lead to a large computational speedup.

Key takeaways

- Pre-trained LLMs are good at text completion, but they do not follow specific instructions very well, so they need fine-tuning on a dataset of instructions and desired responses.
- The instruction dataset used here has 1,100 instruction input/output pairs from various fields, and it was split into training, testing, and validation sets.
- Before training, each instruction-input-output entry is converted into the Alpaca prompt format and then tokenized into token IDs.
- To make batches, token ID sequences are padded with 50256 so every sample in a batch has the same length, and the targets are created by shifting the inputs one position to the right.
- In the targets, the first 50256 end-of-text token is kept, but later padding 50256 tokens are replaced with -100 so PyTorch cross-entropy ignores them in the loss.
- Starting from pre-trained GPT-2 weights makes fine-tuning more efficient than training from scratch, but a pre-trained model alone still does a poor job on instruction-following tasks, so it needs fine-tuning.
- Evaluating an instruction-tuned model is hard because responses must be compared with target answers, and common approaches include MMLU, human judgment, and using another LLM to score responses automatically.

A Sources and coverage

A.1 Coverage map

How many teaching points each lecture (rows) contributes to each chapter (columns).

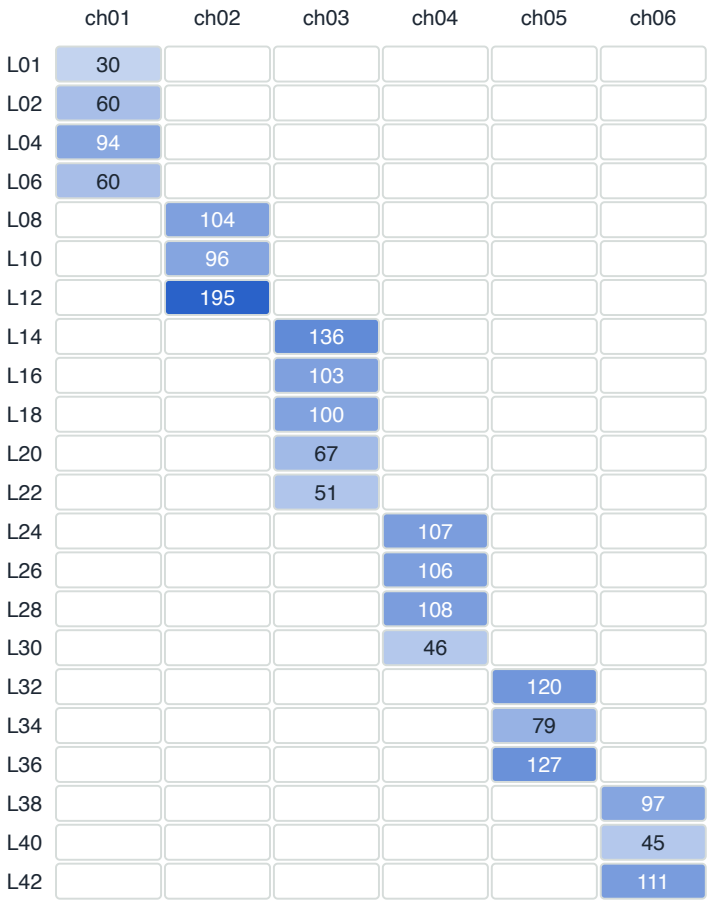


Figure A.1: Teaching points by lecture and chapter.

L01: Lecture 1: Building LLMs from scratch: Series introduction, automatic captions; 35 teaching points extracted.

L02: Lecture 2: Large Language Models (LLM) Basics, automatic captions; 67 teaching points extracted.

L04: Lecture 4: What are transformers?, automatic captions; 103 teaching points extracted.

L06: Lecture 6: Stages of building an LLM from Scratch, automatic captions; 61 teaching points extracted.

L08: Lecture 8: The GPT Tokenizer: Byte Pair Encoding, automatic captions; 109 teaching points extracted.

L10: Lecture 10: What are token embeddings?, automatic captions; 104 teaching points extracted.

L12: Lecture 12: The entire Data Preprocessing Pipeline of Large Language Models (LLMs), automatic captions; 216 teaching points extracted.

L14: Lecture 14: Simplified Attention Mechanism - Coded from scratch in Python | No trainable weights, automatic captions; 147 teaching points extracted.

L16: Lecture 16: Causal Self Attention Mechanism | Coded from scratch in Python, automatic captions; 108 teaching points extracted.

L18: Lecture 18: Multi Head Attention Part 2 - Entire mathematics explained, automatic captions; 110 teaching points extracted.

L20: Lecture 20: Layer Normalization in the LLM Architecture, automatic captions; 70 teaching points extracted.

L22: Shortcut connections in the LLM Architecture, automatic captions; 60 teaching points extracted.

L24: Coding the 124 million parameter GPT-2 model, automatic captions; 112 teaching points extracted.

L26: Measuring the LLM loss function, automatic captions; 111 teaching points extracted.

L28: Coding the entire LLM Pre-training Loop, automatic captions; 111 teaching points extracted.

L30: Top-k sampling in Large Language Models, automatic captions; 47 teaching points extracted.

L32: Loading pre-trained weights from OpenAI GPT-2, automatic captions; 120 teaching points extracted.

L34: Dataloaders in LLM Classification Finetuning | Python Coding | Hands on LLM project, automatic captions; 84 teaching points extracted.

L36: Coding a fine-tuned LLM spam classification model | From Scratch, automatic captions; 128 teaching points extracted.

L38: Data Batching in LLM instruction fine-tuning | Hands on project | Live Python coding, automatic captions; 99 teaching points extracted.

L40: Instruction fine-tuning: Loading pre-trained LLM weights, automatic captions; 49 teaching points extracted.

L42: Evaluating fine-tuned LLM using Ollama, automatic captions; 119 teaching points extracted.

A.2 Where each section comes from

Every factual sentence cites specific caption lines; these are the lecture time ranges each section draws on (links open the video at that moment).

Foundations and Course Roadmap

- Why learn LLMs from scratch: L01 0:54–3:06, 10:28–12:18, 13:38–15:03 · L02 31:40–32:56
- LLMs, generative AI, and the broader AI stack: L01 8:09–8:47 · L02 2:33–4:05, 6:00–11:12, 18:24–25:28
- What modern LLMs do and how they differ from earlier NLP: L02 11:05–14:02 · L06 12:29–13:07, 17:57–18:33
- Applications and motivation: L01 3:44–3:55, 4:58–6:07, 8:20–10:06 · L02 4:07–6:29, 25:49–32:38
- Open and closed models, and learning access: L01 6:45–7:49, 12:21–12:32
- Transformers as the key modern architecture: L02 14:41–14:54, 15:59–16:57 · L04 1:36–5:32, 9:00–10:31, 12:02–12:16, 13:33–17:17, 19:59–20:53, 35:58–37:37 · L06 15:54–17:51
- Tokenization, embeddings, and sequence representation: L04 7:01–13:28, 17:38–18:16 · L06 4:36–5:54
- Attention and long-range dependencies: L04 3:37–3:58, 20:56–25:33, 37:39–37:45
- BERT, GPT, and other model families: L04 25:43–35:18, 38:12–39:51
- The build pipeline: stage one to stage three: L06 2:26–4:32, 5:55–11:17, 13:15–15:48
- Prerequisites, delivery, and current evidence gaps: L01 6:10–6:16, 12:30–14:22 · L06 18:46–19:47

From Text to Model Inputs

- Why preprocessing exists: L12 0:13–2:03, 1:33:14–1:33:34 · L10 0:32–2:26
- Word-based tokenization foundations: L08 1:21–1:46, 2:57–3:59, 5:28–5:37 · L12 2:22–2:30, 4:00–4:09, 12:45–12:57, 14:05–15:43, 29:47–29:51

- Building a simple word tokenizer in Python: L12 3:28–3:55, 5:06–21:17 · L08 43:16–43:31, 47:00–47:16
- Limits of word tokenization: L08 4:10–6:22, 9:55–10:57 · L12 21:37–22:26, 29:53–31:28
- Unknown and boundary tokens in a word tokenizer: L12 13:21–14:42, 22:24–28:46 · L08 43:55–44:33
- Character and subword tokenization: L08 6:29–16:02, 25:01–25:29, 34:37–34:50, 35:58–36:32, 39:35–40:40 · L12 31:23–34:17
- Byte pair encoding as compression: L08 16:36–21:30
- Byte pair encoding for subword vocabularies: L08 0:33–0:45, 16:11–16:26, 21:46–40:00, 44:45–45:03, 46:30–46:33, 47:38–47:43, 49:02–49:31, 50:33–50:55 · L12 2:30–2:39, 34:21–36:18, 38:03–39:27, 40:44–42:43, 44:20–45:02, 46:34–47:00, 48:03–48:07
- Using the GPT-2 tokenizer with tiktoken: L08 40:59–43:08, 45:26–51:02 · L12 43:01–46:17
- Why token embeddings are needed: L10 0:25–1:38, 4:21–6:06, 8:04–13:56, 15:40–16:37, 25:35–25:52, 55:30–55:46, 59:14–1:00:00 · L12 2:39–2:50, 1:04:42–1:08:54, 1:23:03–1:24:07
- What embeddings capture: L10 6:19–7:54, 13:38–26:26, 55:30–56:28 · L12 1:08:50–1:09:24, 1:14:15–1:14:26
- Embedding matrices and lookup layers: L10 26:33–33:46, 36:50–46:45, 48:22–50:43, 57:03–57:27 · L12 1:09:49–1:18:12, 1:19:44–1:21:55
- Embedding layers versus one-hot plus linear layers: L10 50:43–54:55, 58:21–58:31
- Positional embeddings and input embeddings: L12 2:05–2:50, 48:40–49:21, 1:21:43–1:22:22, 1:23:53–1:33:00
- Creating next-token training examples: L12 49:53–1:03:20, 1:18:34–1:19:04

Attention and Transformer Building Blocks

- From tokens to embeddings and context: L14 2:34–7:40, 9:56–12:18 · L16 1:47–4:31
- Simplified self-attention intuition: L14 7:22–8:20, 12:32–16:01, 18:22–26:08
- Normalization and softmax for attention weights: L14 26:33–42:49
- Computing context vectors: L14 42:55–43:42, 44:44–51:12
- Vectorizing self-attention over all tokens: L14 51:34–1:02:03, 1:03:19–1:03:53, 1:05:30–1:08:13, 1:09:42–1:12:55
- Why trainable Q, K, V are needed: L14 14:05–14:29, 1:13:16–1:17:59 · L16 4:10–9:23, 16:10–17:06
- Causal attention and masking: L16 9:30–11:06, 12:41–16:39, 17:44–32:36
- Dropout, batching, and implementation details for causal attention: L16 32:54–55:31
- Efficient multi-head attention with weight splits: L18 0:51–5:48, 7:16–7:32, 9:42–33:06, 34:43–54:46, 56:02–58:19
- Layer normalization in transformers: L20 1:06–1:26, 2:28–5:58, 7:13–8:25, 9:27–35:43, 36:56–38:22

- Shortcut connections and gradient flow: L22 0:23–0:35, 1:57–6:55, 8:32–8:35, 10:02–31:45

Assembling and Pretraining a GPT Model

- What the model is and how the full GPT pipeline is assembled: L24 0:16–0:21, 1:27–4:01, 6:20–6:28, 59:00–1:00:04 · L26 1:42–2:56, 21:55–22:00 · L28 30:25–30:45
- Configuration, tokenizer, and prerequisite vocabulary concepts: L26 5:20–7:08, 21:30–22:57, 24:12–24:18 · L24 10:36–11:40, 14:11–14:24, 40:23–41:55 · L28 31:02–31:50
- Inputs, batches, embeddings, and dimensions: L26 8:51–9:04 · L24 7:31–15:56, 17:03–17:22, 39:01–39:23, 46:27–48:48
- Dropout, layer normalization, and transformer block internals: L24 17:29–31:34, 43:14–46:18 · L26 22:26–22:30
- Stacking blocks, final normalization, output head, and logits: L26 7:28–8:00, 27:15–27:25, 39:46–40:27, 47:12–47:22 · L24 3:47–4:20, 32:48–38:46, 51:03–51:38
- Coding the model and parameter accounting: L24 5:24–5:45, 42:23–42:50, 44:56–45:00, 48:51–50:23, 51:43–57:53 · L28 17:12–20:31, 33:47–33:51
- Next-token prediction tasks, targets, and basic generation from logits: L26 8:29–15:45, 16:47–21:04, 23:03–26:05, 27:18–30:13
- Loss function: from target probabilities to cross-entropy and perplexity: L26 0:20–0:39, 3:07–4:33, 8:23–8:41, 11:55–12:02, 31:28–31:52, 33:02–45:47, 47:31–54:50
- Preparing data for pre-training: L28 0:12–11:07, 13:31–14:05
- Backpropagation, optimizer steps, and the training loop: L28 11:14–16:59, 20:33–24:02, 27:32–28:59, 32:07–33:54
- Evaluation mode, monitoring progress, and overfitting: L28 12:44–13:19, 24:16–30:08, 33:11–41:31
- Decoding logits into text: greedy, temperature, and top-k sampling: L30 1:06–19:07, 21:21–22:16
- Interpreting decoding behavior and a gap in the course narrative: L30 0:26–0:36, 19:13–22:27

Using Pretrained GPT-2 and Fine-Tuning for Classification

- Why load GPT-2 weights and what prerequisites are assumed: L32 0:15–2:24, 4:20–4:29, 44:03–44:12, 48:53–49:42 · L34 0:12–1:10, 3:12–3:32 · L36 45:59–46:04, 48:20–48:38
- Installing tools and downloading released GPT-2 files: L32 4:20–8:49, 11:17–13:04, 26:24–26:40, 28:43–30:09, 32:28–32:39
- Understanding GPT-2 release files, settings, and tokenization metadata: L32 7:22–7:37, 8:51–11:22, 13:08–13:26, 27:38–27:46, 30:27–30:47
- The parameter dictionary and how GPT-2 weights map onto the architecture: L32 13:47–28:25, 30:25–32:14
- Updating the scratch GPT model and assigning pretrained weights: L32 32:47–43:06
- Verifying generation and identifying what can be changed: L32 43:27–49:03

- Preparing the spam-classification dataset: L34 1:29–3:09, 26:12–26:51 · L36 1:10–2:51
- Tokenization, padding, truncation, and dataset design for classification: L34 3:52–23:14, 29:26–30:27
- Building and checking the data loaders: L34 22:53–29:21 · L36 2:13–3:12
- Adapting GPT for classification and making predictions: L36 3:41–9:11, 45:33–45:42, 47:49–47:58
- Accuracy and loss before training: L36 9:36–23:54
- Training loop for fine-tuning: L36 23:21–33:40
- Results, overfitting checks, and what was trained: L36 33:42–38:07, 45:44–46:28, 48:00–49:26
- Inference on new messages and saving the fine-tuned model: L36 38:39–48:04, 49:07–49:13

Instruction Fine-Tuning and Evaluation

- Why instruction fine-tuning is needed and what data it uses: L38 0:32–1:54, 3:45–4:08, 5:28–5:48 · L40 0:35–1:10, 18:28–18:42 · L42 1:07–2:36, 46:37–46:47
- Formatting instruction data into Alpaca-style prompts: L38 2:13–5:11, 9:59–10:32, 24:22–26:56
- Tokenization and variable-length sequences: L38 6:25–11:23, 25:34–26:34
- Batch padding with the end-of-text token: L38 11:33–14:47, 27:14–31:49
- Shifted targets for next-token prediction: L38 15:30–20:06, 22:04–22:19, 32:26–37:45, 45:15–45:40
- Ignoring padding in the loss with -100: L38 20:00–22:18, 23:32–23:43, 37:02–38:46, 39:59–45:09
- The full batching recipe and optional masking choices: L38 22:29–23:43, 46:03–51:00
- Loading pretrained GPT-2 weights for fine-tuning: L40 1:03–7:49, 12:08–12:52
- Downloading and mapping GPT-2 parameters into the model: L40 8:01–12:18
- Checking the pretrained model before fine-tuning: L40 13:08–18:22
- Evaluating a fine-tuned model: goals, examples, and response extraction: L42 0:52–1:00, 2:17–4:40, 5:42–12:59, 18:20–23:50, 51:41–52:04
- Evaluation frameworks for instruction-tuned LLMs: L42 13:02–13:21, 14:32–17:58, 23:30–24:31, 42:46–44:14, 50:10–51:10
- Using Ollama and Llama 3 as an automated evaluator: L42 24:35–43:00, 44:21–46:24
- Improving results and extending the approach: L42 45:58–49:51

A.3 Points not included

Teaching points the outline left out, with the editor's reason:

- All videos in the series and the detailed lecture notes will be available for free. *(Course logistics/free availability, off-topic for concept outline.)*

- The main objective of the course is to put learners in a different position than other people by understanding this material better than anyone else. *(Motivational aside, not concept content.)*
- The course will follow the table of contents shown and convert it into a detailed video lecture series. *(Course logistics about following table of contents.)*
- The instructor plans to release all of the course content completely for free. *(Course logistics/free release, duplicate in spirit of points covered elsewhere in the book.)*
- The next lecture will cover an introduction to large language models and the things shown on the board. *(Lecture navigation/logistics.)*
- This lecture will cover what a large language model is, what "large" means in that term, how modern large language models differ from earlier NLP models, what makes LLMs so good, how terms like LLM, generative AI, deep learning, machine learning, and artificial intelligence differ, and some applications of LLMs. *(Lecture agenda, not concept content.)*
- Deep neural networks have many applications, including image detection, text generation, self-driving cars, and detecting whether a patient has a brain tumor. *(Generic examples of deep learning applications; supporting aside not needed in final concept structure.)*
- The lecturer demonstrates ChatGPT with a prompt to draft an email to a friend about booking movie tickets, and ChatGPT begins drafting the email and even includes emojis. *(Duplicate demonstration of chatbot/email capability already covered by points covered elsewhere in the book.)*
- The Transformer schematic contains components such as input embedding, multi-head attention, feed-forward layers, add and norm, and output embedding. *(Visual schematic reference duplicated by placed transformer concept points covered elsewhere in the book.)*
- The lecturer says the whiteboard approach is intended to provide a visual flavor and visual intuition for how things work. *(Teaching method/logistics.)*
- The next lesson is about the stages of building LLMs, with a few more lectures on basics before moving into coding. *(Lecture navigation/logistics.)*
- Learners should make sure they understand the material, take notes, and ask questions in the comments. *(Study advice/course logistics.)*
- This lecture gives a basic introduction to transformers without going into their mathematical or coding details. *(Lecture framing/logistics.)*
- The original transformer paper proposed a neural network architecture whose full details are substantial enough that each aspect would need separate lectures. *(Lecture framing about scope/detail, not concept content.)*
- A simplified transformer schematic is shown as eight steps, and understanding these eight steps gives the intuition of the transformer architecture. *(Visual reference duplicated by placed transformer workflow points.)*
- In the example shown, the transformer takes English input text and after eight steps converts it into German. *(Visual example duplicated by placed translation workflow points covered elsewhere in the book.)*
- For decoding, the decoder receives both the vector embedding and the partial

output text, and it is trained to predict the next output word. *(Duplicate of placed decoder procedure points covered elsewhere in the book.)*

- The simplified explanation is meant only as an introductory view, because attention has not yet been covered and the full transformer architecture is more complicated. *(Lecture framing/logistics.)*
- The transformer schematic includes attention blocks such as masked multi-head attention and multi-head attention. *(Visual schematic reference duplicated by placed attention/transformer points.)*
- In the simplified Transformer architecture, an input example is pre-processed by converting words or sentences into words and token IDs, passed into the encoder to produce vector embeddings, and then those embeddings are fed to the decoder along with partial output text so that it generates the translated sentence one word at a time. *(Duplicate of placed transformer workflow points covered elsewhere in the book.)*
- Transformers can also be used for computer vision tasks such as image classification and image segmentation. *(Duplicate of points covered elsewhere in the book on non-language transformer uses.)*
- The lecturer shows a diagram of the Transformer architecture and points out the attention blocks as the main source of its power. *(Visual reference duplicated by placed attention and transformer points covered elsewhere in the book.)*
- The lecturer shows the 1994 paper that introduced the byte pair algorithm for data compression. *(visual reference to the original paper; not needed in a concept outline)*
- The lecturer states that GPT-2 or GPT-3 used roughly 50,000 to 57,000 tokens after byte pair encoding. *(duplicate of placed vocabulary-size points covered elsewhere in the book)*
- The lecture taught what byte pair encoding is, how to install tiktoken and use the GPT-2 byte pair tokenizer, how it handles unknown words, and that its vocabulary size is about 50,000. *(lecture recap/logistics summary, not content)*
- A schematic is shown to illustrate that the course has covered tokenizing input text into token IDs, including a custom tokenizer in the previous lecture and byte pair encoding in this lecture, and has not yet covered token embeddings. *(course-progress visual reference, not book content)*
- The lecture also covered intuitive whiteboard iterations of the byte pair encoder and coding a byte pair encoder in Python using the tiktoken library. *(lecture recap, duplicate of placed BPE and tiktoken material)*
- In the example sentence, the words are tokenized, arranged in ascending order, and then assigned token IDs. *(duplicate of placed token-ID construction points covered elsewhere in the book)*
- For simplicity, the lecturer uses a word-based tokenizer in the example. *(teaching simplification aside, not core content)*
- In the worked example, the selected input token IDs are represented as a torch tensor because tensors are needed for backpropagation and optimization. *(implementation aside about tensors; off-topic for the conceptual outline)*

- The lecturer shows an image of an embedding weight matrix and token IDs, where each requested ID maps to a highlighted row and the collected rows form the output embedding matrix. *(visual reference for embedding lookup; covered by placed points covered elsewhere in the book)*
- For the example token IDs corresponding to the words "fox jumps over dog," the embedding layer returns the vector embeddings for all of those IDs. *(example duplicate of embedding lookup behavior already covered by points covered elsewhere in the book)*
- The lecture does not cover how to train embeddings, only what token embeddings are and what the embedding layer weight matrix is. *(course-scope note, not content)*
- The lecturer says that the most important thing is to keep showing up for the lectures, not lose interest or motivation, and continue learning along with him. *(course motivation/logistics)*
- The lecturer says he looks forward to seeing everyone in the next lecture. *(course logistics)*
- This lecture belongs to stage one of building an LLM, specifically the data preparation and sampling part, while the three stages are building the basics, pre-training, and fine-tuning. *(course-structure logistics, off-topic)*
- The dataset used for demonstration is the book "The Verdict," shown on screen with its cover and text. *(visual reference to demonstration dataset, not conceptually necessary)*
- Printing the first 50 vocabulary items shows special characters first with token IDs such as 0, 1, and 2, followed by other tokens with their IDs. *(printed output visual reference, duplicate of placed vocabulary-construction points)*
- In the example vocabulary shown, the lecturer identifies "e" followed by "s" as the most common pair, merges it into "es," and then continues with later merges such as "est." *(visual example duplicate of placed BPE merge steps points covered elsewhere in the book)*
- After several iterations of merging the most common pairs, the final token list shown contains a mixture of characters, subwords, and whole words. *(visual reference duplicate of placed BPE outcome points covered elsewhere in the book)*
- The lecturer notes that this tokenizer has encode and decode methods similar to the previous tokenizer class. *(duplicate of placed tokenizer encode/decode points covered elsewhere in the book)*
- The screenshot shows that byte pair encoding represents a new unknown word as tokens made of subwords or characters, which is why decoding can recover the same sentence. *(visual reference duplicate of placed unknown-word handling points covered elsewhere in the book)*
- The vocabulary sizes associated with GPT tokenizers increase for later models such as GPT-3 and GPT-4. *(aside about later GPT tokenizer sizes; off-topic)*
- This part of the data loader is detailed but very important. *(teaching emphasis aside, not content)*
- Understanding datasets and data loaders is not strictly required before understanding token embeddings. *(course-guidance aside, not core material)*

- For simplicity, the lecturer shows an example with a vocabulary size of six and a vector dimension of three, using input IDs 2, 3, 5, and 1. *(visual/simple example duplicate of placed embedding example points covered elsewhere in the book)*
- If you pass a single token ID such as 3 into the embedding layer, it returns the embedding vector for that ID, which corresponds to the appropriate row of the matrix. *(code example duplicate of placed lookup procedure points covered elsewhere in the book)*
- If you pass multiple input IDs into the embedding layer at once, it returns the token embedding vectors for all those input IDs based on the embedding layer weight matrix. *(code example duplicate of placed batching lookup points covered elsewhere in the book)*
- The lecturer points out that in the shown batch, each row is one input sequence of four token IDs, and there are eight rows in total. *(visual reference duplicate of placed batch-shape points covered elsewhere in the book)*
- In the sentences "the cat sat on the mat" and "on the mat the cat sat," the word "cat" would have the same token embedding even though it appears in different positions. *(example duplicate of placed claim points covered elsewhere in the book and purpose points covered elsewhere in the book)*
- At this stage, the token embeddings form a tensor of shape 8 by 4 by 256. *(duplicate of placed token-embedding shape points covered elsewhere in the book)*
- The lecturer uses the terms positional embedding and positional encoding interchangeably. *(duplicate of placed terminology note points covered elsewhere in the book)*
- The lecture covered tokenization, including word-based, subword-based, and character-based tokenizers. *(lecture recap duplicate of placed tokenization sections)*
- The lecture covered the byte pair encoding tokenizer, which is used by GPT. *(lecture recap duplicate of placed BPE sections)*
- The lecture was designed as a condensed overview of the full LLM data preprocessing pipeline. *(lecture-format summary, not content)*
- The teaching approach combines whiteboard explanation with coding in Google Colab to strengthen both theoretical and coding foundations. *(teaching-method note, off-topic)*
- This lecture introduces a simplified self-attention mechanism without trainable weights, and later lectures will add trainable weights, causal attention, and multi-head attention. *(course logistics / roadmap statement, not conceptual content needed in the book outline)*
- A 3D plot is shown to visualize the embedding vectors in three-dimensional space. *(visualization aside; the underlying embedding concept is already covered by points covered elsewhere in the book)*
- In the shown example, higher attention is paid to the words "journey" and "starts," while lower attention is paid to "one." *(example-specific visual repetition of placed attention-weight interpretation points covered elsewhere in the book)*
- The lecturer visually shows embedding vectors for different words together with their relative importance values. *(visual aid only; core idea already covered by*

placed procedure points for weighting embeddings)

- In the illustrated example, the word "journey" has the highest importance because it is the query, while "starts" and "step" have lower importance and "one," "with," and "your" have less importance. *(example-specific visual repetition of placed points covered elsewhere in the book)*
- The lecturer shows the embedding vector for "journey" in green and the corresponding context vector in red in three-dimensional space. *(visual aid only; contrast between embeddings and context vectors is already captured by points covered elsewhere in the book)*
- The lecturer says the computed context vector is also shown on the screen as a red arrow and contrasted with the other vectors. *(visual aid only; duplicate of placed context-vector explanation)*
- The displayed attention weights have been rounded, so they may not be exactly identical to previously computed values. *(rounding note about displayed numbers; not conceptually important)*
- Without trainable weights, the vectors for "mat" and "warm" might be at a 90-degree angle, illustrating that they may not be semantically related even though "mat" is important in this context. *(duplicate example detail of points covered elsewhere in the book about context importance not matching semantic similarity)*
- The next lecture will cover a simplified self-attention mechanism with trainable weights and will introduce the concepts of key, query, value, causal attention, and multi-head attention. *(course logistics / future-lecture preview)*
- This lecture has covered simplified self-attention in detail to build intuition before moving on to key, query, and value. *(course logistics / recap statement)*
- The lecture series first covered simplified self-attention without trainable weights, then self-attention with trainable weights, and this lecture introduces causal attention before moving on to multi-head attention. *(course logistics / series roadmap)*
- For example, when the query is 'journey', only the attention to 'your' and 'journey' is kept, and the attention scores for later words are masked out. *(example-specific illustration of causal masking; duplicate of general rule in points covered elsewhere in the book)*
- For a token like 'with', attention is kept only for earlier tokens, while for the last token nothing is masked, and for the first token everything after it is masked. *(example-specific illustration of causal masking; duplicate of general rule in points covered elsewhere in the book)*
- In the running example, the input sequence is the six words "your journey begins with one step." *(example sentence reference only; not needed for conceptual organization)*
- Using the negative-infinity mask before softmax kills two birds with one stone: softmax turns the masked entries into zero and also ensures that each row sums to one. *(analogy phrasing duplicating the already placed technical points covered elsewhere in the book)*
- For simplicity, the lecturer suggests thinking of tokens as words, such as "the," "cat," and "sleeps." *(minor analogy about thinking of tokens as words; duplicate of*

placed token simplification points covered elsewhere in the book)

- The lecturer says you can think of two attention heads as two people paying attention to each token. *(analogy about heads as people; intuitive aside not needed once head splitting is explained formally)*
- For the example sentence "the cat sleeps," the second row corresponds to the query token "cat," and its entries represent attention between cat and "the," cat and "cat," and cat and "sleeps." *(example-specific row interpretation duplicating general statement points covered elsewhere in the book)*
- Because the attention weights in a row sum to one, they can be interpreted as how much attention a token pays to earlier tokens and itself, such as cat paying mostly attention to "the" and only a small amount to cat in the example shown. *(example-specific interpretation duplicating general probabilistic reading in points covered elsewhere in the book)*
- The example uses small embedding sizes and few attention heads only to keep the outputs readable. *(example-size caveat; not part of the core concept)*
- GPT-3 uses many more attention heads than this example, and GPT models generally have D_in equal to D_out. *(scale aside about GPT-3; not needed for the conceptual outline)*
- Understanding dimensions and linear algebra is central to understanding and modifying attention mechanisms and to becoming a strong machine learning engineer. *(motivational aside about linear algebra importance; duplicate of placed prerequisite note points covered elsewhere in the book)*
- Higher-dimensional matrices become much easier to understand once you write them down and understand what is going on. *(study advice / motivational aside)*
- You should write things down, such as on a whiteboard or a piece of paper, because doing so helps you remember them for a longer period of time. *(study advice, off-topic for the book content)*
- The next lecture will start building the LLM model. *(course logistics / future-lecture preview)*
- Layer normalization is used not only in GPT and LLM architectures but also very frequently in many deep learning architectures. *(broad aside about use in many architectures; not needed for the chapter's conceptual flow)*
- Turning off scientific notation in the printed output makes the normalized means appear as zero and the variances appear as one for both batches. *(printing-format note; off-topic)*
- Layer normalization is one of the building blocks needed to understand the GPT architecture, alongside components such as the GPT backbone, GELU activation, feed-forward networks, shortcut connections, and the transformer block. *(course-roadmap statement listing building blocks; duplicate of placed component sections)*
- In the example network without shortcut connections, the displayed gradient magnitudes become much smaller in earlier layers, illustrating the vanishing gradient problem. *(visual demonstration duplicated by placed claims points covered elsewhere in the book about vanishing gradients without shortcuts)*
- In the example network with shortcut connections, the displayed gradient magnitude

in the first layer is much larger than without shortcut connections, indicating that vanishing gradients are reduced. *(visual demonstration duplicated by placed claims points covered elsewhere in the book about improved gradients with shortcuts)*

- The shown loss-landscape comparison suggests that without skip connections there are many local minima, peaks, and valleys, whereas with skip connections the landscape appears smoother with fewer local minima. *(visual comparison of loss landscape; duplicate of placed claim points covered elsewhere in the book)*
- The lecturer shows a comparison in which the loss-function landscape without skip connections has many oscillations, while the landscape with skip connections is much smoother. *(visual comparison of loss landscape; duplicate of placed claim points covered elsewhere in the book)*
- In this example, layer_sizes has six elements indexed from 0 to 5. *(indexing detail for the example list; off-topic implementation minutia)*
- In the lecture demonstration, the network without shortcut connections showed gradients vanishing toward almost zero near the first layer, while the network with shortcut connections did not. *(example result duplicated by placed claims points covered elsewhere in the book)*
- The previous lectures covered layer normalization, GELU activation, and the feed-forward neural network as building blocks of the Transformer block. *(course logistics / recap of previous lectures)*
- This lecture covered shortcut connections in detail, and the next lecture will cover the Transformer block itself. *(course logistics / future-lecture preview)*
- The lecturer aims for each lecture to provide intuitive understanding, theoretical understanding, and coding knowledge. *(teaching-philosophy aside, off-topic)*
- To predict the next word for a given input context, one looks at the vocabulary column with the highest logit value, and that column should correspond to the correct next token. *(Duplicate of placed points covered elsewhere in the book about selecting the highest-probability/highest-logit token for prediction.)*
- Once the visual intuition for the GPT blocks is clear, the model is ready to be implemented in code by arranging those blocks together. *(Duplicate of placed points covered elsewhere in the book/points covered elsewhere in the book about implementing the model once the block structure is clear.)*
- Writing every step of the transformer code as a visual flow map on a whiteboard or in a notebook can greatly strengthen understanding and confidence about the subject. *(Course study advice; not book content.)*
- The lecturer is showing on the screen a visual flow map of the transformer similar to the one written on the whiteboard. *(On-screen visual reference; not substantive content.)*
- The next lecture will probably be the last lecture in the GPT architecture series and will generate text from the final output tensor obtained today. *(Course logistics about the next lecture.)*
- In the illustrated example, each row has three tokens because the context size is assumed to be three. *(Minor example detail tied to a toy context size; not needed once general target construction points are placed.)*

- Although the last next-token target is often the main interest, the earlier target positions are also important because they correspond to the earlier prediction tasks in the input. *(Supporting repetition of placed points covered elsewhere in the book about having a prediction task at each position.)*
- A previously defined function called "generate text simple" is used to take the model, start from four input tokens, and generate 10 new tokens. *(Course code logistics referencing a previously defined helper function; not conceptually needed.)*
- In this example, the vocabulary size is 50257 tokens. *(Duplicate of placed vocabulary-size points covered elsewhere in the book/points covered elsewhere in the book.)*
- In this lecture, the code example used only two sample inputs. *(Minor course-example logistics.)*
- Training a GPT model instance means creating an instance of the previously defined GPT model class. *(Duplicate of placed points covered elsewhere in the book about the GPT model class.)*
- The lecturer emphasizes that this LLM training loop was built from scratch without using libraries such as LangChain. *(Course aside about not using external libraries; off-topic for the conceptual outline.)*
- The next lecture will cover decoding strategies such as temperature scaling to control randomness so the model predicts new words instead of merely memorizing text. *(Course logistics previewing the next lecture.)*
- The next lecture will load pretrained weights from OpenAI, examine some other pretraining strategies, and later move on to fine-tuning and applications. *(Course logistics about future topics.)*
- In the previous lecture the dataset was downloaded and preprocessed, and in this lecture data loaders were created for the training, validation, and testing datasets. *(course logistics recap from previous lecture)*
- The next stage will initialize the LLM model, load pretrained weights, and modify the model. *(course logistics/forward pointer to later stage)*
- The subsequent lectures will evaluate the fine-tuned model. *(course logistics/forward pointer)*
- The lecture approach combines whiteboard teaching with coding. *(teaching style, off-topic for book content)*
- The aim is to train machine learning engineers with strong fundamental and theoretical understanding, not just skill in using ChatGPT. *(course motivation, off-topic for technical outline)*
- This part of the project calculates classification loss and accuracy, implements the training loop and testing loop, and completes the fine-tuning project. *(section overview/logistics rather than concept content)*
- The lecturer indicates on the whiteboard that masking is done by replacing target-text token IDs with -100. *(duplicate visual reference of masking procedure already covered by placed points covered elsewhere in the book)*
- In this implementation series, the lecturer will not apply masking and will leave it as an optional exercise. *(course logistics/implementation choice; says masking is left as an exercise, not conceptual content)*

- This lecture covers step four of the instruction fine-tuning hands-on project, which is loading a pre-trained large language model. *(course logistics; identifies lecture step number rather than teaching content)*
- If the GPT-2 parameters have already been downloaded, the output will indicate that the files already exist. *(minor runtime note about existing downloaded files; off-topic for concept outline)*
- This lecture covered loading a pre-trained LLM, and the fine-tuning process will be covered in the next lecture. *(course logistics about what the next lecture covers)*
- The shared code file can be used to perform a wide range of instruction fine-tuning tasks. *(general remark about shared code usability; too broad/off-topic)*
- The lecturer shows the MMLU paper and points to a list of 57 tasks or subject areas, such as abstract algebra, anatomy, astronomy, business ethics, biology, chemistry, computer science, law, management, marketing, and public relations. *(visual reference duplicating placed MMLU content in points covered elsewhere in the book)*
- The lecturer would like someone to run the MMLU test code on the instruction fine-tuned LLM if possible. *(course logistics/request to viewers rather than course concept)*
- Creating the instruction data with response file can take time; in the lecturer's case it took around 10 to 15 minutes. *(runtime estimate; peripheral logistics)*
- The Ollama download file shown for macOS is 177 MB, and the installation process is simple and does not take much time. *(minor installation logistics for macOS download size)*
- Downloading the Llama 3 model through Ollama took around 15 to 20 minutes for the lecturer. *(download-time logistics)*
- On limited hardware, multiple simultaneous queries can cause the laptop to hang, so the lecturer could handle at most about two queries in one session. *(hardware-specific anecdote; peripheral to the conceptual workflow)*
- On the lecturer's laptop, running the evaluation for three test samples initially caused a crash and then took about 35 minutes, making evaluation of the full test set impractical. *(hardware/runtime anecdote; not central to the evaluation method)*
- Through this process, the lecture also introduced a new tool called Ollama. *(course wrap-up remark about introducing Ollama; duplicate of placed Ollama content)*

B Evaluation report

Status: **REVIEWED RELEASE**.

Measure	Value
Words	52243
Sections	91
Factual sentences	2237
Supported on first check	93.5%
Supported after one rewrite	97.9%
Paragraphs rewritten	192
Open review-queue sentences	0
Reviewer decisions (accepted / edited / removed)	3 / 46 / 4
Planted false sentences caught	106 / 120
Real sentences flagged in the plant run	3 / 120
Teaching points extracted / placed / dropped	2170 / 2042 / 128
Core points dropped	3
Diagrams kept / dropped by the checker	16 / 2
Code listings from the reference	46
Cross-check vs reference (agrees / contradicts / not covered)	918 / 73 / 1052
Open conflicts with the reference	0
Board frames read / figures kept after checking against speech	16 / 6
Listings executed / numeric claims checked by running code	37 of 46 ran; 69 passed, 0 failed
Evaluator agent	passed: support 0.965, material errors 1.5/100, core ideas missed 0.0426, quality 3.66/5
Evaluator method	200 sampled sentences, each judged 3 times (majority counts; the runs disagreed on 10% of them). Core ideas are searched across the whole book; missed near the sampled clip alone: 12.8%

B.1 Planted claims by type

Type	Caught / planted
number	22 / 24
term swap	21 / 24
reversed cause	19 / 24
unsupported addition	24 / 24
wrong attribution	20 / 24

B.2 Planted claims the checker missed

- (number) Deep learning is therefore a subset of machine learning because machine learning includes neural network methods and also 3 methods like decision trees that are not neural networks.
- (reversed cause) The code prints that the encoded text has length 5145 tokens, so it reads the raw text and encodes it with the byte pair tokenizer from tiktoken.
- (term swap) In “encode”, the input text is split on the chosen characters, whitespace is removed, and the remaining strings are hashed in the vocabulary to return a list of token IDs.
- (reversed cause) Understanding how attention works is therefore important for understanding dimensions, batches, three-dimensional tensors, matrices, and linear algebra fundamentals.
- (term swap) Softmax also handles extreme cases better, because after normalization a very large value becomes dominant, making its normalized result very close to one and the smaller values very close to zero.
- (wrong attribution) This layer normalization step is the only step in this flow where the dimensions change, from four tokens by 768 to four tokens by 50257.
- (reversed cause) Using the model for a classification task allows this classification head to be attached on top of the GPT architecture.
- (term swap) For example, one message can produce probabilities 0.99 and 0.01 after sigmoid, and then the predicted class is index 0, which means no spam.
- (wrong attribution) At the model level, the code passes all samples in a batch through the model, takes the logits of the last output token, applies argmax for the whole batch, and compares the predicted labels with the target labels.
- (reversed cause) Those GPT-2 weights were released for multiple model sizes, including 124 million, 345 million, 774 million, and 1558 million parameters, and they can be downloaded because the folders correspond to those parameter sizes.
- (wrong attribution) When an instance of this instruction dataset class is created, the data loader first stores the dataset in “self.data”.
- (number) The reason for the shift is that instruction fine-tuning still uses the 2-token prediction task.

- (wrong attribution) For example, with the input sentence “the lecture was delivered in a clear manner”, the expected rewrite was “the lecture was delivered clearly”, but the benchmark response remained “the lecture was delivered in a clear manner”.
- (reversed cause) Running Llama 3 this way downloads files of about 4.7 GB, and the model has 8 billion parameters because it takes substantial space and memory.